

# Jaybird JDBC Driver Java Programmer's Manual

Roman Rokytskyy, Mark Rotteveel

# Table of Contents


1. Introduction . . . 1
    1.1. This manual . . . 1
    1.2. History . . . 1
    1.3. Jaybird Architecture . . . 1
    1.4. Jaybird Distribution . . . 2
    1.5. Quality Assurance . . . 3
    1.6. Useful resources . . . 3
    1.7. Contributing . . . 4

User Manual . . . 7

2. Obtaining a connection . . . 9
    2.1. Obtaining connection java.sql.DriverManager . . . 9
    2.2. Driver types . . . 15
    2.3. Connection Pooling . . . 21
    2.4. The javax.sql.DataSource implementation . . . 23
    2.5. The javax.sql.ConnectionPoolDataSource implementation . . . 24
    2.6. The javax.sql.XADataSource implementation . . . 24
3. Handling exceptions . . . 25
    3.1. Working with exceptions . . . 25
    3.2. Warnings . . . 27
    3.3. java.sql.SQLException in Jaybird . . . 28
    3.4. SQL states . . . 30
    3.5. Useful Firebird error codes . . . 30
4. Executing statements . . . 35
    4.1. The java.sql.Statement interface . . . 35
    4.2. Statement behind the scenes . . . 38
    4.3. The java.sql.PreparedStatement interface . . . 40
    4.4. The java.sql.CallableStatement interface . . . 43
    4.5. Batch Updates . . . 49
    4.6. Escape syntax . . . 52
5. Working with result sets . . . 57
    5.1. ResultSet properties . . . 57
    5.2. ResultSet manipulation . . . 59
6. Using transactions . . . 63
    6.1. JDBC transactions . . . 63
    6.2. Auto-commit mode . . . 64
    6.3. Read-only Transactions . . . 66
    6.4. Transaction Isolation Levels . . . 67
    6.5. Savepoints . . . 68

6.6. Transaction Parameter Buffer . . . 69
6.7. Table Reservation . . . 72
6.8. Transaction management statements . . . 74
7. Working with Services . . . 75
7.1. ServiceManager . . . 75
7.2. Backup and restore . . . 77
7.3. User management . . . 85
7.4. Database maintenance . . . 87
7.5. Table statistics of a connection (experimental) . . . 93
7.6. Database statistics . . . 93
8. Working with Events . . . 97
8.1. Database events . . . 97
8.2. Posting events . . . 98
8.3. Subscribing to events . . . 98
Reference Manual . . . 103
9. Connection reference . . . 105
9.1. Authentication plugins . . . 105
9.2. Wire encryption support . . . 107
9.3. Wire compression . . . 108
9.4. Database encryption support . . . 109
9.5. Default holdable result sets . . . 110
9.6. Firebird auto commit mode (experimental) . . . 111
9.7. Process information . . . 112
9.8. Data type bind support . . . 113
9.9. Enabling unsupported protocol versions . . . 114
9.10. Opt-in feature for package information in DatabaseMetaData . . . 115
9.11. Client info properties . . . 116
9.12. Create database if it does not exist . . . 119
9.13. Reporting of `SQLWarnings` . . . 120
9.14. Custom socket factory . . . 120
9.15. Schema support . . . 121
10. Statement reference . . . 125
10.1. Async fetch . . . 125
10.2. Generated keys retrieval . . . 126
10.3. Inline blobs . . . 132
10.4. Connection property ignoreProcedureType . . . 133
10.5. Maximum field size (`maxFieldSize`) . . . 134
10.6. Connection property scrollableCursor . . . 135
10.7. Connection property useServerBatch . . . 136
10.8. Connection property serverBatchBufferSize . . . 136
10.9. Connection property allowTxStmts . . . 137

10.10. Connection property extendedMetadata . . . 138
10.11. Schema support . . . 138
11. General . . . 139
11.1. Logging . . . 139
12. Datatype reference . . . 143
12.1. Binary types BINARY/VARBINARY . . . 143
12.2. Type BOOLEAN . . . 143
12.3. Date/time types . . . 145
12.4. Decimal floating point type DECFLOAT . . . 151
12.5. Exact numeric types DECIMAL/NUMERIC . . . 155
12.6. Type INT128 . . . 156
Appendices . . . 157
Appendix A: Extended connection properties . . . 159
A.1. Authentication and security properties . . . 159
A.2. Other properties . . . 160
A.3. Transaction isolation levels . . . 165
Appendix B: System properties . . . 167
B.1. Logging . . . 167
B.2. Process information . . . 167
B.3. Character set defaults . . . 167
B.4. Default `asyncFetch` value . . . 168
B.5. Default `enableProtocol` value . . . 168
B.6. Default inline blob property values . . . 168
B.7. Default `reportSQLWarnings` value . . . 168
B.8. Wire protocol buffer sizes . . . 169
B.9. Close connection on fatal error . . . 170
B.10. Other properties . . . 170
B.11. Useful Java system properties . . . 171
Appendix C: Data Type Conversion Table . . . 173
C.1. Mapping between JDBC, Firebird and Java Types . . . 173
C.2. Data Type Conversions . . . 173
Appendix D: Character Encodings . . . 175
D.1. Encodings Types . . . 175
D.2. Encodings in Java . . . 175
D.3. Available Encodings . . . 178
Appendix E: Supported JDBC Scalar Functions . . . 181
E.1. Numeric Functions . . . 181
E.2. String Functions . . . 182
E.3. Time and Date Functions . . . 183
E.4. System Functions . . . 184
E.5. Conversion Functions . . . 185

Appendix F: Jaybird versions . . . . . 189
F.1. Jaybird 6 . . . . . 189
F.2. Jaybird 5 . . . . . 192
F.3. Jaybird 4 . . . . . 195
Appendix G: License . . . . . 199

# Chapter 1. Introduction

Jaybird is a JDBC (Java DataBase Connectivity) driver to connect to the Firebird [https://firebirdsql.org] database server.

Jaybird is a JDBC driver that provides both Type 4 (pure Java) and Type 2 (native binding) support. The Type 2 driver includes support for Firebird Embedded.

## 1.1. This manual

This manual covers Jaybird 6 and may use Java 17 and Firebird 5.0 specific features, but most examples and information also apply to previous versions of Jaybird, Firebird, and Java. Features that were removed in Jaybird 5 or earlier are not covered.

New or removed features are tagged with the version that introduced a feature (e.g. ***Since:*** *Jaybird 6*) or removed feature (e.g. ***Removed in:*** *Jaybird 6*). This tagging is only done for features introduced (or removed) *after* Jaybird 5.0.0, or in Firebird 5.0 or later.

This manual may contain documentation of features of unreleased Jaybird and Firebird versions to simplify manual maintenance.

## 1.2. History

When Borland released an open-source version of the InterBase RDBMS, it included sources for a Type 3 (middleware) JDBC driver called InterClient.[1] However, due to some inherent limitations of the InterBase (and later Firebird) client library, it was decided that the Type 3 driver was a dead end. Instead, the Firebird team decided to develop a pure Java implementation of the wire protocol. This implementation became the basis for Jaybird, a Type 2 and Type 4 JDBC driver for the Firebird relational database.

## 1.3. Jaybird Architecture

The Jaybird driver consists of three layers, each responsible for a part of the functionality.

- The GDS-ng layer represents a Java translation of the Firebird API. It is represented by a number of interfaces and classes from the `org.firebirdsql.gds` package (and sub-packages).

  This API is implemented by a number of plugins that provide the pure Java, native, and embedded implementations of the driver.

- The XCA layer represents the heart of the driver. Here all connection and transaction management happens. Additionally, this layer adapts the GDS-ng API for use by the JDBC layer.

  The XCA layer is a replacement of the JCA layer of earlier versions, and should be considered an internal implementation detail.

- The JDBC layer is an implementation of the JDBC Specification [https://jcp.org/en/jsr/detail?id=221].

Separately, the Services API — also layered on GDS-ng — allows you to manage databases and the server itself. It offers a Manager component to create and drop databases. Other classes provide access to the Firebird Services Manager for database backup/restore, tracing, statistics gathering, etc.

## 1.4. Jaybird Distribution

Jaybird 6 supports Firebird 3.0 and higher on Java 17 and higher. See Jaybird versions for detailed information on supported Java and Firebird versions per Jaybird version.

The latest version of Jaybird can be downloaded from https://firebirdsql.org/en/jdbc-driver/.

### 1.4.1. Maven

You can also use Maven, or Gradle, to automatically download Jaybird and its dependencies.

Jaybird 6 is available from Maven Central:

| | |
|---|---|
| **groupId** | `org.firebirdsql.jdbc` |
| **artifactId** | `jaybird` |
| **version** | `6.0.7` |

For example:

*Main Jaybird artifact*

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird</artifactId>
  <version>6.0.7</version>
</dependency>
```

If you want to use ChaCha64 wire encryption, you need to add `chacha64-plugin` as a dependency:

*ChaCha64 wire encryption plugin*

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>chacha64-plugin</artifactId>
  <version>6.0.7</version>
</dependency>
```

If you want to use native or embedded (JDBC Type 2), you need to add `jaybird-native` as a dependency:

*Jaybird native artifact (native and embedded protocol)*

```
<dependency>
```

```
    <groupId>org.firebirdsql.jdbc</groupId>
    <artifactId>jaybird-native</artifactId>
    <version>6.0.7</version>
</dependency>
```

The Firebird client library is available as a Maven dependency, see Maven dependency for native client.

In the future, a Maven dependency for Firebird embedded may become available as well. An experimental version — for Windows — is available from https://github.com/mrotteveel/jaybird-embedded-win32-x86-64.

### 1.4.2. Gradle

See also Maven.

Examples:

*Main Jaybird artifact*

```
implementation 'org.firebirdsql.jdbc:jaybird:6.0.7'
```

*ChaCha64 wire encryption plugin*

```
implementation 'org.firebirdsql.jdbc:chacha64-plugin:6.0.7'
```

*Jaybird native artifact (native and embedded protocol)*

```
implementation 'org.firebirdsql.jdbc:jaybird-native:6.0.7'
```

## 1.5. Quality Assurance

The Jaybird developers uses JUnit test cases to assure the quality of the released driver. During development unit tests are extensively used. Committing a code change to the source control is not allowed until it passes all existing unit tests. Each reproducible bug usually gets its own test case. This guarantees that a clean check out can be compiled and will not contain any previously discovered and fixed bugs.

Currently, there are more than 10,000 test cases covering most of the driver code. A subset of the tests are automatically run — through a GitHub Action — for each push to the repository.

## 1.6. Useful resources

### 1.6.1. JDBC

For JDBC documentation, see Java SE Technologies - Database [https://www.oracle.com/java/technologies/javase/javase-tech-database.html].

For the JDBC specification, see JSR 221: JDBC™ 4.0 API Specification [https://jcp.org/en/jsr/detail?id=221] (covers JDBC 4.0 - 4.5, e.g. JDBC 4.5 is “Maintenance Release 5”).

### 1.6.2. Firebird

General information about the Firebird database is available from the Firebird website [https://firebirdsql.org/].

For information about using SQL in Firebird, see the Firebird 5.0 Language Reference [https://firebirdsql.org/file/documentation/html/en/refdocs/fblangref50/firebird-50-language-reference.html] and other documentation available from the Reference Manuals [https://firebirdsql.org/en/reference-manuals/] section of the Firebird website.

### 1.6.3. Jaybird Support

Support for Jaybird is available through the following channels:

- The firebird-java Google Group [https://groups.google.com/g/firebird-java] and corresponding mailing list firebird-java@googlegroups.com

  You can subscribe to the mailing list by sending an email to firebird-java+subscribe@googlegroups.com (this does not require a Google account **if** you confirm by email reply). Alternatively, you can join the group at the firebird-java Google Group [https://groups.google.com/g/firebird-java] (this requires a Google account).

- On Jaybird Frequently Asked Questions [https://firebirdsql.org/docs/drivers/java/faq.html]
- On Jaybird wiki [https://github.com/FirebirdSQL/jaybird/wiki/] (“archived”)

## 1.7. Contributing

There are several ways you can contribute to Jaybird or Firebird in general:

- Participate on the mailing lists [https://firebirdsql.org/en/mailing-lists/]
- Report bugs or submit patches on the tracker (see Reporting Bugs)
- Create pull requests on GitHub (https://github.com/FirebirdSQL/jaybird)
- Become a developer (for Jaybird contact us on firebird-java [https://groups.google.com/g/firebird-java], for Firebird in general, use firebird-devel [https://groups.google.com/g/firebird-devel])
- Become a paying Associate, Partner or sponsor of the Firebird Foundation [https://firebirdsql.org/en/firebird-foundation/] (see also Support Firebird: Donation and Sponsorship Opportunities [https://firebirdsql.org/en/donate])

### 1.7.1. Reporting Bugs

The developers follow firebird-java [https://groups.google.com/g/firebird-java]. Join the list and post information about suspected bugs. List members may be able to help out to determine if it is an actual bug, provide a workaround and get you going again, whereas bug fixes might take a while.

You can report bugs in the Jaybird bug tracker [https://github.com/FirebirdSQL/jaybird/issues] on GitHub.

When reporting bugs, please provide a minimal, but complete reproduction, including databases and source code to reproduce the problem. Patches or pull requests to fix bugs are also appreciated. Make sure the patch is against a recent master version of the code. You can also fork the jaybird repository [https://github.com/FirebirdSQL/jaybird] and create pull requests.

[1] For those interested in software archaeology, you can find the open sourced Interclient sources archived on https://github.com/FirebirdSQL/x-cvs-interclient

# User Manual

# Chapter 2. Obtaining a connection

Jaybird is a regular JDBC driver and supports two primary ways to obtain connection: via `java.sql.DriverManager` and via an implementation of the `javax.sql.DataSource` interface.

## 2.1. Obtaining connection java.sql.DriverManager

`java.sql.DriverManager` is a database connection factory introduced with JDBC 1. It is based on the concept of the JDBC URL, a string that uniquely identifies the database to connect. The driver manager then checks which driver(s) — if any — can establish a connection.

There is also support to specify additional connection parameters, like username and password.

A JDBC URL consists of three parts:

```
jdbc:firebird://localhost:3050/c:/database/example.fdb
```

- `jdbc`
  JDBC protocol
- `firebird` (also supported: `firebirdsql`)
  JDBC subprotocol, identifies driver to use, in this case Jaybird
- `//localhost:3050/c:/database/example.fdb`

  This is a driver-specific part, and identifies the database for the driver to connect, in the case of Jaybird, the primary URL format is `//[<host>:[<port>]]/<path to database>`.

  See also Driver types for more in-depth coverage of the syntax of Jaybird JDBC URLs.

The first part — `jdbc:firebird:` or `jdbc:firebirdsql:` — is required by JDBC. It specifies the so-called protocol and subprotocol for the JDBC connection. In other words, it specifies the type of connection that the application wants to obtain, in this example, a connection to a Firebird database.

An example of obtaining a connection is shown below.

*Simple example to obtain a JDBC connection*

```
package hello;

import java.sql.*;

public class HelloServer {

  public static void main(String[] args) throws Exception {
    Class.forName("org.firebirdsql.jdbc.FBDriver"); ①

    try (Connection connection = DriverManager.getConnection(
          "jdbc:firebird://localhost:3050/c:/db/employee.fdb",
```

```
            "SYSDBA", "masterkey")) { ②
        // use connection here
      }
    }
  }
```

① This loads the driver (optional, see also next paragraphs)

② Creates a connection

The first line of this code used to be important — it tells Java to load the Jaybird JDBC driver. As required by the JDBC specification, at this point the driver registers itself with `java.sql.DriverManager`.

Since Java 6 (JDBC 4), explicitly loading the driver using `Class.forName("org.firebirdsql.jdbc.FBDriver")` is no longer necessary, except when the driver is not on the system (initial) classpath. Examples where it may be necessary to explicitly load the driver are web applications that include the driver in the deployment (e.g. in `WEB-INF/lib` of the WAR). There, the driver is not on the system classpath, so it will need to be loaded explicitly.



Explicitly loading the driver with `Class.forName("org.firebirdsql.jdbc.FBDriver")` may also aid in debugging "*No suitable driver found for jdbc:firebird...*" errors:

1. is the driver not found on the classpath, or
2. it was found, but an error occurred during loading or initialization,
3. or — if the error goes away — it was not loaded by automatic driver loading.

We will leave out usages of `Class.forName` in further examples; they will work because of automatic driver loading.

### Registering the JDBC driver

There are several ways to register a JDBC driver:

1. `DriverManager` loads the drivers from the system classpath. This happens automatically.
2. The application explicitly loads the driver's class.

   ```
   Class.forName("org.firebirdsql.jdbc.FBDriver");
   ```

   This is only necessary if the automatic loading (see previous item) is not available. This can be necessary because the driver JAR is loaded dynamically, through a different classloader, etc).

   The JDBC specification requires that during class initialization the driver registers itself with `DriverManager`.

3. The JDBC driver class name is listed in the `jdbc.drivers` system property. Multiple drivers can be separated using a colon (':').

You can specify the value of this property during JVM startup:

```
java\
  -Djdbc.drivers=foo.Driver:org.firebirdsql.jdbc.FBDriver\
  -classpath jaybird-full-{jaybird-example-version}.jar;C:/myproject/classes\
  my.company.SomeJavaExample
```

The second statement of the example tells `java.sql.DriverManager` to open a database connection to the Firebird server running on localhost, and the path to the database is `c:/database/employee.fdb`.

The connection specification consists of — optionally — the host name of the database server (defaults to localhost), and optionally you can specify a port (defaults to 3050). The host name can be specified using either its DNS name (for example `fb-server.mycompany.com` or just `fb-server`), or its IP address (e.g. `192.0.2.5`, or `[2001:db8::5]` for an IPv6 address).

After the server name and port, the alias or path to the database is specified. We suggest to specify a database alias instead of the absolute database path. For more information about using aliases, see the documentation of Firebird server.

The format of the path depends on the platform of the Firebird server.

On Windows, the path must include the drive letter and path, for example `c:/database/employee.fdb`, which points to the employee database that can be found in the `database` directory of drive `C:`. Java (and Firebird) supports either `/` or `\` (escaped as `\\`) as path separator on the Windows platform. On Unix and Linux platforms, you can use only `/` as the path separator.

On Unix platforms the path must include the root, as the path is otherwise interpreted relative to a server-dependent folder. Having to include the root has the effect that a database in `/var/firebird/employee.fdb` needs to use a double `//` after the host name (and port) in the connection string: `jdbc:firebird://localhost//var/firebird/employee.fdb`.

It is possible to specify a relative path, but as this depends on the server configuration, this may be confusing or easily lead to errors. We recommend to not use a relative path, and instead use an alias (see also `databases.conf` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fbconf/fbconf-databases.html] in the *Firebird Configuration Reference*.

### 2.1.1. Specifying extended properties

The JDBC API provides a method to specify additional connection properties, for example a client encoding:

*Obtaining JDBC connection with additional connection properties*

```
package hello;

import java.sql.*;
import java.util.*;
```

```
public class HelloServerWithEncoding {

  public static void main(String[] args) throws Exception {
    var props = new Properties();
    props.setProperty("user", "SYSDBA");
    props.setProperty("password", "masterkey");
    props.setProperty("encoding", "UTF8");

    try (Connection connection = DriverManager.getConnection(
            "jdbc:firebird://localhost:3050/C:/db/employee.fdb",
            props)) {
        // use connection here
    }
  }
}
```

The `user` and `password` properties are defined in JDBC. Nearly all other property names — like `encoding` here — are driver-specific.

Additional properties, for example the SQL role for the connection, can be added to the `props` object. The list of properties available in Jaybird can be found in Extended connection properties.

It is not always possible to use the above described method to add properties. Jaybird also supports specifying connection properties in the JDBC URL.

*Extended JDBC URL format*

```
jdbc:firebird://host[:port]/<path to db>?<properties>
<properties> ::= <property>[{& | ;}<properties>]
<property>   ::= <name>[=<value>]
```

The example below shows the specification for extended JDBC properties in the URL.

In this case extended properties are passed together with the URL using the HTTP-like parameter passing scheme: first comes the main part of the URL, then '?', then name-value pairs separated with '&' or ';'. The following example is equivalent to the previous example.

*Specifying extended properties in the JDBC URL*

```
import java.sql.*;

...

Connection connection = DriverManager.getConnection(
    "jdbc:firebird://localhost:3050/C:/db/employee.fdb?encoding=UTF8",
    "SYSDBA",
    "masterkey");
```

#### URL encoding in query part of JDBC URL

UTF-8 URL encoded values (and keys) can be used in the query part of the JDBC URL.

This can be used to include otherwise unsupported characters in a connection property value:

- `;` escaped as `%3B`
- `&` escaped as `%26`
- `+` escaped as `%2B`

  A `+` in the query part means *space* (0x20), so occurrences of `+` (*plus*) need to be escaped; make sure to do this for *base64* encoded values of `dbCryptConfig`, or better yet use *base64url* encoding instead.

- `%` escaped as `%25`.

  A `%` in the query part introduces an escape, so occurrences of `%` (*percent*) need to be escaped.

- Optionally, a *space* (0x20) can be escaped as `+`

URL encoding can also be used to encode any Unicode codepoint in the query string. Jaybird will always use UTF-8 for decoding.

Invalid URL encoded values will throw a `SQLNonTransientConnectionException`.

The support for URL encoding *only* applies to the JDBC URL part after the first `?`. URL encoding should not be applied for connection properties set through `java.util.Properties` or on a `javax.sql.DataSource`.

### 2.1.2. Obtaining a connection via javax.sql.DataSource

The interface `javax.sql.DataSource` defines a simple API for a factory of `java.sql.Connection` objects. Data sources can be created and configured using code or bean introspection, looked up from JNDI, or injected by CDI or Spring.

Jaybird itself provides one `javax.sql.DataSource` implementation, `org.firebirdsql.ds.FBSimpleDataSource`, which is a plain factory of connections, without connection pooling.



If you need connection pooling, use a third-party connection pool library like HikariCP [https://brettwooldridge.github.io/HikariCP/], DBCP [https://commons.apache.org/proper/commons-dbcp/], or c3p0 [https://www.mchange.com/projects/c3p0/]. Application servers, and for example Tomcat, also provide built-in connection pool support. Consult their documentation for more information.

See also Connection Pooling.

A simple example of creating a data source and obtaining a connection via a `DataSource` object is shown below.

*Obtaining a JDBC connection from a DataSource*

```
package hello;

import java.sql.*;
import org.firebirdsql.ds.*;

public class HelloServerDataSource {

  public static void main(String[] args) throws Exception {
    var ds = new FBSimpleDataSource();
    ds.setUser("SYSDBA");
    ds.setPassword("masterkey");
    // in a single property
    ds.setDatabaseName("//localhost:3050/C:/database/employee.fdb");
    // or split out over serverName, portNumber and databaseName
    ds.setServerName("localhost");
    ds.setPortNumber(3050);
    ds.setDatabaseName("C:/database/employee.fdb");

    try (Connection connection = ds.getConnection()) {
      // use connection here
    }
  }
}
```

### Using JNDI to look up a javax.sql.DataSource

The JDBC 2.0 specification introduced a mechanism to obtain database connections without requiring the application to know any specifics of the underlying JDBC driver. The application is only required to know the logical name to find an instance of the `javax.sql.DataSource` interface using Java Naming and Directory Interface (JNDI). This was a common way to obtain connections in web and application servers before the introduction of CDI.

This code assumes that you have correctly configured the JNDI properties. For more information about configuring JNDI please refer to the documentation provided with your web or application server.

*Typical way to obtain JDBC connection via JNDI*

```
package hello;

import java.sql.*;
import javax.sql.*;
import javax.naming.*;

public class HelloServerJNDI {

  public static void main(String[] args) throws Exception {
    var ctx = new InitialContext();
```

```
    DataSource ds = (DataSource) ctx.lookup("jdbc/SomeDB");

    try (Connection connection = ds.getConnection()) {
      // do something here...
    }
  }
}
```

Usually, the binding between the `DataSource` object and its JNDI name happens in the configuration of your web or application server. However, under some circumstances (e.g. you are developing your own JNDI-enabled application server/framework), you may have to do this yourself. You can use this code snippet for this purpose:

*Programmatic way to instantiate javax.sql.DataSource implementation*

```
import javax.naming.*;
import org.firebirdsql.ds.*;
...
var ds = new FBSimpleDataSource();

ds.setDatabaseName("//localhost:3050/C:/database/employee.fdb");
ds.setUser("SYSDBA");
ds.setPassword("masterkey");

var ctx = new InitialContext();

ctx.bind("jdbc/SomeDB", ds);
```

The `DataSource` implementation supports all connection properties available to the `DriverManager` interface.



Manually binding to JNDI like shown above is uncommon. If you find yourself copying this code, rethink what you're doing.

In fact, use of JNDI is extremely uncommon these days.

## 2.2. Driver types

As mentioned in the section Jaybird Architecture, Jaybird supports multiple implementations of the GDS-ng API. The default Jaybird distribution contains two main categories of the implementations: the Type 4 (pure Java) implementation of the Firebird wire protocol, and a Type 2 implementation using a JNA (Java Native Access) [https://github.com/java-native-access/jna] proxy that can use a Firebird `fbclient` library.

The next sections provide a description of these types and their configuration with the corresponding JDBC URLs that should be used to obtain the connection of desired type. The type of the JDBC driver for the `javax.sql.DataSource` is configured via the property `type` (`setType(String)`).

### 2.2.1. PURE_JAVA (JDBC Type 4)

The `PURE_JAVA` type (JDBC Type 4) provides a pure Java implementation of the Firebird wire protocol. This type is recommended for connecting to a remote database server using TCP/IP sockets. No installation is required except adding the JDBC driver to the classpath. This type of driver provides the best performance when connecting to a remote server. Some Jaybird features are only available in the pure Java implementation.

To obtain a connection using the `PURE_JAVA` driver type you have to use a JDBC URL as shown in Obtaining connection java.sql.DriverManager.

The following JDBC URL syntax is supported (*serverName* became optional in Jaybird 5)

```
<pure-java-url> ::=
  jdbc:firebird[sql]:[java:]<database-coordinates>

<database-coordinates> ::=
    //[serverName[:portNumber]]/databaseName
  | <legacy-url>

<legacy-url> ::=
  [serverName[/portNumber]:]databaseName
```

If *serverName* is not specified, it defaults to `localhost`.
If *portNumber* is not specified, it defaults to `3050`.

In theory, even `<database-coordinates>` and *databaseName* are optional, but this requires specifying the database name using connection property `databaseName`, which is possible, but not recommended.

When using `javax.sql.DataSource` implementation, you can specify either `"PURE_JAVA"` or `"TYPE4"` as the driver type. This type is already used by default.

*Pure Java URL examples*

```
// Connect to db alias employee on localhost, port 3050
jdbc:firebird://localhost/employee
jdbc:firebird://localhost:3050/employee
jdbc:firebird:///employee

// Same using the legacy URL format
jdbc:firebird:localhost:employee
jdbc:firebird:localhost/3050:employee
jdbc:firebird:employee
```



The legacy URL format is the historic URL format used by Firebird itself. We recommend not using this format, as it can be ambiguous.

For example, say you have a server called `java`, then `jdbc:firebird:java:employee`

will attempt to open the `employee` db alias on `localhost`, not on `java`, as `java:` is interpreted as the optional sub-subprotocol, and not as the hostname. With `jdbc:firebird://java/employee`, this ambiguity does not exist.

### 2.2.2. NATIVE (JDBC Type 2)

***Since:*** *Jaybird 6* Native connections require the `jaybird-native` artifact on the classpath.

The `NATIVE` type (JDBC Type 2) uses a JNA proxy to access the Firebird client library and requires installation of the Firebird client. The `NATIVE` driver type is used to access a remote database server through TCP/IP, or — Windows-only — XNET or WNET. Performance of the `NATIVE` driver using TCP/IP is approximately 10% lower compared to the `PURE_JAVA` driver, but with XNET has up to 30% higher performance compared to the `PURE_JAVA` driver when connecting to a server on the same host. This is mostly due to the fact that TCP/IP stack is not involved in XNET.

In addition to Jaybird and — since Jaybird 6 — `jaybird-native`, this requires a native Firebird client library on the `PATH` or `LD_LIBRARY_PATH`, and JNA 5.19.1 needs to be on the classpath.

To create a connection using the `NATIVE` JDBC driver to connect to a remote server you have to use the following JDBC URL syntax:

```
<native-url> ::=
  jdbc:firebird[sql]:native:<database-coordinates>

<database-coordinates> ::=
    //[serverName[:portNumber]]/databaseName
  | <fbclient-url>

<fbclient-url>
    inet://serverName[:portNumber]/databaseName
  | inet4://serverName[:portNumber]/databaseName
  | inet6://serverName[:portNumber]/databaseName
  | wnet://[serverName[:portNumber]/]databaseName
  | xnet://databaseName
  | [serverName[/portNumber]:]databaseName
```

All URLs supported by fbclient can be used. The supported URLs depend on the fbclient version and the OS (e.g. XNET and WNET are Windows only, and WNET support has been removed in Firebird 5).



**LOCAL protocol removed in Jaybird 5**

The `LOCAL` protocol was removed in Jaybird 5, and is now simply an alias for `NATIVE`. To ensure local access, use a connection string using XNET (Windows only!):

```
jdbc:firebird:native:xnet://<path to database>
```

This requires a Firebird 3.0 or later `fbclient.dll`.

> Support for this type of URL was introduced in Jaybird 5, so this syntax cannot be used in earlier versions.
>
> As XNET is Windows only, on other platforms, consider using an EMBEDDED connection instead.

*Native URL examples*

```
// Connect to db alias employee on localhost, port 3050
jdbc:firebird:native://localhost/employee
jdbc:firebird:native://localhost:3050/employee
jdbc:firebird:native:///employee

jdbc:firebird:native:inet://localhost/employee
// Require IPv4
jdbc:firebird:native:inet4://localhost/employee
// Require IPv6
jdbc:firebird:native:inet6://localhost/employee
// Using WNET
jdbc:firebird:native:wnet://localhost/employee
// Using XNET
jdbc:firebird:native:xnet://employee

// Same using the legacy URL format
jdbc:firebird:native:localhost:employee
jdbc:firebird:native:localhost/3050:employee
// May use XNET, INET or embedded access
jdbc:firebird:native:employee
```

### Maven dependency for native client

You can use a library to provide the Firebird client library for the `NATIVE` protocol. For Windows, Linux, and macOS, you can add the `org.firebirdsql.jdbc:fbclient` [https://github.com/mrotteveel/jaybird-fbclient] dependency to your classpath. This dependency does not support the `embedded` protocol.

*Native libraries for all supported OS and architectures*

```
<dependency>
    <groupId>org.firebirdsql.jdbc</groupId>
    <artifactId>fbclient</artifactId>
    <version>5.0.4.0</artifactId>
</dependency>
```

Since version 5.0.1.1 of this library, you can also specify the desired OS, or OS and architecture using the `classifier`:

*Only native libraries for Linux (all supported architectures)*

```
<dependency>
```

```
    <groupId>org.firebirdsql.jdbc</groupId>
    <artifactId>fbclient</artifactId>
    <version>5.0.4.0</artifactId>
    <classifier>linux</classifier>
</dependency>
```

See mrotteveel/jaybird-fbclient [https://github.com/mrotteveel/jaybird-fbclient#os-specific-packages] for the available classifiers.

You can also download the library (see mrotteveel/jaybird-fbclient [https://github.com/mrotteveel/jaybird-fbclient#download] for download link) and add it your classpath.

See next sections for other solutions.

### Windows



We recommend using the solution documented in Maven dependency for native client.

On Windows, you need to make sure that `fbclient.dll` is located on the `PATH` environment variable. Alternatively, you can specify the directory containing this DLL in the `jna.library.path` system property or the connection property `nativeLibraryPath`[1].

For example, if you put a copy of `fbclient.dll` in the current directory you have to use the following command to start Java:

```
java -cp <relevant claspath> -Djna.library.path=. com.mycompany.MyClass
```

If your Java installation is 64-bit, you need a 64-bit `fbclient.dll`, for 32-bit Java, a 32-bit `fbclient.dll`.

### Linux



We recommend using the solution documented in Maven dependency for native client.

On Linux, you need to make sure that `libfbclient.so` is available through the `LD_LIBRARY_PATH` environment variable.

Usually shared libraries are stored in the `/usr/lib/` directory; however you will need root permissions to install the library there. Some distributions will only have, for example, `libfbclient.so.2.5`. In that case you may need to add a symlink from `libfbclient.so` to the client on your system.

Alternatively, you can specify the directory containing this DLL in the `jna.library.path` system property or the connection property `nativeLibraryPath`[1]. See the Windows example above for more details.

#### Limitations

Older versions of the Firebird client library — as far as we are aware, Firebird 2.1 or older — may not be thread-safe when connecting to a local database server using IPC. By default, Jaybird does not provide synchronization, but it can be enabled with the system property `org.firebirdsql.jna.syncWrapNativeLibrary` set to true. However, this synchronization is local to the classloader that has loaded the Jaybird classes.

To guarantee correct synchronization, the Jaybird driver must be loaded by the top-most classloader. For example, when using the Type 2 JDBC driver with a web or application server, you have to add the Jaybird classes to the main classpath (for example, to the `lib/` directory of your web or application server), but **not** to the web or Java EE/Jakarta EE application, e.g. the `WEB-INF/lib` directory.

### 2.2.3. EMBEDDED (JDBC Type 2)

***Since:*** *Jaybird 6* Embedded connections require the `jaybird-native` artifact on the classpath.

The Embedded server JDBC driver is a Type 2 JDBC driver that, rather than using the Firebird client library, loads the Firebird embedded server library instead. This is the highest performance type of JDBC driver for accessing local databases, as the Java application accesses the database file directly.

The following JDBC URL syntax is supported:

```
<embedded-url> ::=
  jdbc:firebird[sql]:embedded:dbname-or-alias
```

In practice, the URL accepts the same `<fbclient-url>` values as described for `NATIVE`. That is, the embedded server also acts as client library (i.e. you get the same Type 2 behavior as you would get with using "native").

This driver tries to load `fbembed.dll/libfbembed.so` (the name used in Firebird 2.5 and earlier) and `fbclient.dll/libfbclient.so`.

See also the NATIVE (JDBC Type 2) section.

When using Firebird 3.0 and higher embedded, you will need to make sure the necessary plugins like `engineNN.dll/libengineNN.so` (*NN* is 12 for Firebird 3.0, 13 for Firebird 4.0 and Firebird 5.0) are accessible to the client library, consult the Firebird 3.0 documentation for more information. For an example, see the article Jaybird with Firebird embedded example [https://www.lawinegevaar.nl/firebird/jaybird_embedded_example.html].

#### Limitations

Older versions of the Firebird embedded server for Linux are not thread safe; as far as we know this concerns Firebird 2.1 or older. Jaybird can provide the needed synchronization in Java code, as described for NATIVE type. This implies the same restrictions on the classloader that will load the Jaybird classes.

By default, the Firebird embedded library opens databases in exclusive mode. This means that this particular database is accessible only to one Java virtual machine. This can be changed with the `ServerMode` setting in `firebird.conf`.

### 2.2.4. OOREMOTE (Type 4)

***Removed in:*** *Jaybird 6*

The `OOREMOTE` type is a JDBC Type 4 driver specifically for use with OpenOffice.org and LibreOffice. It addresses some differences in interpretation of the JDBC specification, and provides alternative metadata in certain cases to allow OpenOffice.org and LibreOffice to function correctly.



This only describes connecting to Firebird using Jaybird from OpenOffice.org or LibreOffice. This is not about the Firebird embedded use that has been introduced in recent LibreOffice versions.

To obtain a connection you have to use following URL:

```
jdbc:firebirdsql:oo://host[:port]/<path to database>
jdbc:firebird:oo://host[:port]/<path to database>
```

All other notes for PURE_JAVA (JDBC Type 4) apply.



**OOREMOTE deprecated in Jaybird 5, and removed in Jaybird 6**

The OOREMOTE protocol implementation is deprecated and has been removed in Jaybird 6. To connect to Firebird, use LibreOffice Base with connection option “Firebird External”.

## 2.3. Connection Pooling

Each time a connection is opened via `DriverManager` or `FBSimpleDataSource`, a new physical connection to the server is opened. This physical connection is closed when the connection is closed. To avoid the overhead of creating connections, you can use a connection pool to maintain a cache (pool) of open physical connections that can be reused.

Jaybird does not provide its own connection pool implementation. If you need a `javax.sql.DataSource` implementation that provides a connection pool, either use the connection pool support of your application server, or consider using HikariCP [https://brettwooldridge.github.io/HikariCP/], DBCP [https://commons.apache.org/proper/commons-dbcp/], or c3p0 [https://www.mchange.com/projects/c3p0/].

### 2.3.1. HikariCP example

This example shows how to configure HikariCP [https://brettwooldridge.github.io/HikariCP/] to connect to Firebird.

```
package example;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import org.firebirdsql.ds.FBSimpleDataSource;

import java.sql.Connection;
import java.sql.SQLException;

public class HikariConnectExample {

  public static void main(String[] args) {
    HikariDataSource hikariDataSource = initDataSource();

    try (Connection connection = hikariDataSource.getConnection()) {
      // use connection
    } catch (SQLException e) {
      System.getLogger("HikariConnectExample")
          .log(System.Logger.Level.ERROR, "Could not connect", e);
    }

    hikariDataSource.close();
  }

  private static HikariDataSource initDataSource() {
    var firebirdDataSource = new FBSimpleDataSource();
    firebirdDataSource.setServerName("localhost");
    firebirdDataSource.setDatabaseName("employee");
    firebirdDataSource.setUser("sysdba");
    firebirdDataSource.setPassword("masterkey");
    firebirdDataSource.setCharSet("utf-8");

    var config = new HikariConfig();
    config.setDataSource(firebirdDataSource);
    return new HikariDataSource(config);
  }
}
```

HikariCP provides multiple ways to configure the connection. Some examples:

*Indirect use of* `FBSimpleDataSource`

```
private static HikariDataSource initDataSourceAlternative1() {
  var config = new HikariConfig();
  config.setDataSourceClassName("org.firebirdsql.ds.FBSimpleDataSource");
  config.setUsername("sysdba");
  config.setPassword("masterkey");
  config.addDataSourceProperty("serverName", "localhost");
  config.addDataSourceProperty("databaseName", "employee");
  config.addDataSourceProperty("charSet", "utf-8");
```

```
  return new HikariDataSource(config);
}
```

*Using the Jaybird JDBC driver instead of a data source*

```
private static HikariDataSource initDataSourceAlternative2() {
  var config = new HikariConfig();
  config.setDriverClassName("org.firebirdsql.jdbc.FBDriver");
  config.setJdbcUrl("jdbc:firebird://localhost/employee");
  config.setUsername("sysdba");
  config.setPassword("masterkey");
  config.addDataSourceProperty("charSet", "utf-8");
  return new HikariDataSource(config);
}
```

## 2.4. The javax.sql.DataSource implementation

Connection pool implementations, whether provided by a Java EE/Jakarta EE application server or a third-party library, are exposed as an implementation of the `javax.sql.DataSource` interface — just like "basic" data sources.

The most important method exposed by this interface is the `getConnection()` method, which returns a connection based on the configuration of the data source. For a "basic" (non-pooling) data source, this will create a new, physical, connection. For a connection pool, this will create a logical connection that wraps a physical connection from the pool.



The "user" of a connection should not care whether the connection is pooled or not: the connection should behave the same from the perspective of the user, and the user should use the connection in the same way. This allows for swapping between a non-pooling and pooling data source in an application without any changes to the code using the data source.

When the application ("user") is done with a connection, it should call `close()` on the connection. When from a non-pooling data source, the physical connection will be closed. For a logical connection from a connection pool, `close()` will invalidate the logical connection (which will make it behave like a closed connection), and return the underlying physical connection to the connection pool. The connection pool will either keep it in the pool for re-use, or close the physical connection (e.g. if the pool has too many idle connections).



Use a connection for the shortest scope, and time, necessary for correct behaviour. Get a connection, and close it as soon as you're done. When using a connection pool, this has the added benefit that just a few connections can serve the needs of the application.

## 2.5. The javax.sql.ConnectionPoolDataSource implementation

The `javax.sql.ConnectionPoolDataSource` interface represents a factory that creates `PooledConnection` objects for use by a connection pool. For example, application servers support the use of a `ConnectionPoolDataSource` to populate their connection pool.

A `PooledConnection` instance represents a physical connection to a database and is a source of logical connections that a connection pool can hand out to the application. Closing this logical connection returns the physical connection back into the pool.



**Not a connection pool**

Contrary to its name, a `ConnectionPoolDataSource` is not a connection pool! It is a factory for physical connections that can be **used** by a connection pool.

Jaybird provides `org.firebirdsql.ds.FBConnectionPoolDataSource` as an implementation of the `javax.sql.ConnectionPoolDataSource` interface.

## 2.6. The javax.sql.XADataSource implementation

The JDBC 2.0 specification introduced the `javax.sql.XADataSource` interface for access to connections that can participate in distributed transactions with a JTA-compatible transaction coordinator. This gives applications the possibility to use two-phase commit to coordinate commits to multiple resource managers (e.g. multiple databases).

Just like `javax.sql.ConnectionPoolDataSource`, applications normally don't access an `XADataSource` implementation directly, instead it is used as a factory of connections for an XA-enabled data source. To the application this is usually exposed as a `javax.sql.DataSource`.

Jaybird provides `org.firebirdsql.ds.FBXADataSource` as an implementation of the `javax.sql.XADataSource` interface.

[1] This connection property will only work for the **first** native connection made.

# Chapter 3. Handling exceptions

Exception handling is probably the most important aspect that directly affects the stability of the application. Correct handling of the error cases guarantees correct functioning of the client code as well as the database server.

All methods of the interfaces defined in the JDBC specification throw instances of `java.sql.SQLException` — or a subclass — to notify about error conditions that happen during request processing. `SQLException` is a checked exception, which forces Java programmers to either handle it with the try/catch clause or redeclare it in the method signature.

## 3.1. Working with exceptions

Exception handling becomes even more important if we consider that this topic is either ignored or presented in incorrect form in most JDBC tutorials. The official JDBC tutorial from Oracle [https://docs.oracle.com/javase/tutorial/jdbc/index.html] briefly mentions that exceptions should be handled by using try/catch blocks only at the end of the course, but neither reasons for doing this nor the best practices are presented.

There are good reasons to think about exception handling in your applications before you start coding. First of all, it is very hard to change the exception handling pattern in existing code. The changes will affect all layers above the place where the changes in exception handling are made and the new application must be thoroughly tested after the change.

Another reason was already mentioned on the beginning of this chapter: instances of `java.sql.SQLException` are the only way for the RDBMS server or JDBC driver to notify about the error condition that happened during request processing. By checking the error code which is sent with the exception, an application can try to recover from the error.

And last but not least, there is resource management. When an exception happens in a method, the execution flow of Java code differs from the normal flow, and only correctly coded application will ensure that all allocated resources will be released. The resources in our case are JDBC connections, statements, result sets, etc. All these objects not only take memory in the Java Virtual Machine of the application, but also consume memory and possibly hold locks on the server, which, worst case, can lead to an unintended Denial-of-Service attack, as the database server can no longer service requests.

A good exception handling strategy requires you do distinguish three kinds of error conditions:

- errors that the database access layer can detect and correctly handle; for example, the application might decide to re-execute the business transaction if the database server returned a deadlock error;
- errors that database access layer can detect, but is unable to handle; usually those are all database errors that do not have special handling routines;
- errors that database access layer cannot detect without additional code unrelated to the functionality of this layer; basically, all runtime exceptions fall into this category.

The handling strategy then consists of

- processing the selected error codes for cases described above;
- converting the generic `SQLException` into a generic business error in the application (this can be throwing some generic exception defined in the application, but can also be an entry in the application event log and short message that asks to retry the operation later);
- some emergency tactics, since the error that happened (e.g. `NullPointerException` or `OutOfMemoryError`) was not considered while the application was created, thus possibly leaving it in an unknown state; further operation should be considered dangerous and the corresponding execution branch has to be halted.

The problem of resource management can be solved if resource allocation and release happens in the same code block and is protected with a try-with-resources block. The code to recover from error conditions should use try/catch blocks. An example of such error and resource handling code is presented below.

*Typical resource allocation and error handling patterns*

```
String updateString = "update COFFEES " +
    "set SALES = ? where COF_NAME like ?";

try (PreparedStatement updateSales = con.prepareStatement(updateString)) {
  int [] salesForWeek = {175, 150, 60, 155, 90};
  String [] coffees = {"Colombian", "French_Roast",
      "Espresso", "Colombian_Decaf",
      "French_Roast_Decaf"};

  int len = coffees.length;

  for(int i = 0; i < len; i++) {
    updateSales.setInt(1, salesForWeek[i]);
    updateSales.setString(2, coffees[i]);

    try {
      updateSales.executeUpdate();
    } catch(SQLException ex) {
      if (ex.getErrorCode() == ...) {
        // do something
      } else {
        throw new BusinessDBException(ex);
      }
    }
  }
}
```

The nested try/catch block shows you an example of handling a deadlock error if it happens (first scenario according to our classification), otherwise the exception is converted and passed to the upper layers (second scenario). As you see, there is no special treatment to the third scenario.

A possible bug in the JDBC driver could have generated runtime exception in the `PreparedStatement.executeUpdate()` method, which would lead to the statement handle leakage if

the try-with-resource block had not been used to do the resource cleanup. As a rule of thumb, always declare and allocate resources in a try-with-resources block: the resource will be automatically closed/freed at the end of the block, even if exceptions occur.

Such coding practice might look weird, because on first sight the whole purpose of using the `PreparedStatement` is neglected: the statement is prepared, used only once and then deallocated. However, when this practice is combined with connection and statement pooling, it brings enormous advantage to the application code. The code becomes much more manageable — resource allocations and release happen in the same method and the software developer does not need to remember the places where the same prepared statement might be used. A statement pool will either reuse the statement or it will prepare a new one, if it detects that all pooled prepared statements are currently in use. As a side effect, the application will always use the minimum number of statements handles, which in turn reduces the used resources on the database side.



Jaybird currently provides no statement pooling itself, availability will depend on the connection pool library used. Consult the documentation of your connection pool to see if — and how — it provides statement pooling.

## 3.2. Warnings

Some errors returned by Firebird are treated as warnings. They are converted into instances of `java.sql.SQLWarning` class in the JDBC layer. These exceptions are not thrown from the driver methods, but added to a connection instance.



Currently, no warning is added to `Statement` or `ResultSet` objects.

Each next warning is appended to the tail of the warning chain. To read the warning chain, use the code presented below.

*Example how to work with warnings*

```
import java.sql.*;
....
SQLWarning warning = connection.getWarnings();
while (warning != null) {
  .... // do something with the warning
  warning = warning.getNextWarning();
}
```

or

*Alternative example how to work with warnings*

```
import java.sql.*;
....
for (Throwable throwable : connection.getWarnings()) {
  if (throwable instanceof SQLWarning) {
```

```
    SQLWarning warning = (SQLWarning) throwable;
    .... // do something with the warning
  }
}
```

This second example will iterate over the first warning, all its causes (if any), and then on to other warnings (if any), and so on.

To clear existing warning, call `Connection.clearWarnings()` method.

***Since:*** *Jaybird 6* Some applications log or report `SQLWarning` in a way that can be confusing or annoying. You can disable registration of `SQLWarnings` globally or per connection, see Reporting of `SQLWarnings` for more information.

## 3.3. java.sql.SQLException in Jaybird

**TODO** Information in section is possibly outdated or incomplete

An `SQLException` is a special exception that is thrown by the JDBC connectivity component in case of an error. Each instance of this exception is required to carry the vendor error code (if applicable) and a SQL state according to the X/Open SQLstate or SQL:2003 specifications. Firebird and Jaybird use SQL:2003 SQL state codes.[1]

When multiple SQL errors happened, they are joined into a chain. Usually the most recent exception is thrown to the application, the exceptions that happened before can be obtained via `SQLException.getNextException()` method. Alternatively, `SQLException.iterator()` can be used to walk over all exceptions in the chain and their causes.

The JDBC specification provides an exception hierarchy that allows an application to react on the error situations using regular exception handling rather than checking the error code. Error codes may still be necessary for handling specific error cases.

The JDBC 4.3 specification defines the following exception hierarchy:[2]

- `java.sql.SQLException` — root of all JDBC exceptions
  - `java.sql.BatchUpdateException` — thrown when batch of the statements did not execute successfully; contains the result of batch execution.
  - `java.sql.SQLClientInfoException` — thrown when client info properties could not be set.
  - `java.sql.SQLNonTransientException` — thrown when retrying the same action without fixing the underlying cause would fail.
    - `java.sql.SQLDataException` — thrown for data-related errors, for example conversion errors, too long values. (SQLstate class 22)
    - `java.sql.SQLFeatureNotSupportedException` — thrown to indicate that an optional JDBC feature is not supported by the driver or the data source (Firebird). (SQLstate class `0A`)
    - `java.sql.SQLIntegrityConstraintViolationException` — thrown for constraint violations. (SQLstate class 23)

        - `java.sql.SQLInvalidAuthorizationSpecException` — thrown for authorization failures. (SQLstate class 28)
        - `java.sql.NonTransientConnectionException` — thrown for connection operations that will not succeed on retry without fixing the underlying cause. (SQLstate class 08)
        - `java.sql.SQLSyntaxErrorException` — thrown for syntax errors. (SQLstate class 42)
    - `java.sql.SQLRecoverableException` — thrown when an action might be retried by taking recovery actions and restarting the transaction.
    - `java.sql.SQLTransientException` — thrown when the action might succeed if it is retried without further recovery steps.
        - `java.sql.SQLTimeoutException` — thrown when the `queryTimeout` or `loginTimeout` has expired.
        - `java.sql.SQLTransactionRollbackException` — thrown when the statement was automatically rolled back because of deadlock or other transaction serialization failures. (SQLstate class 40)
        - `java.sql.SQLTransientConnectionException` — thrown for connection operations that might succeed on retry without any changes. (SQLstate class 08)
    - `java.sql.SQLWarning` should only be used to signal warnings, it should never be thrown by a JDBC driver.
        - `java.sql.DataTruncation` — thrown when a data truncation error happens, can also be used as a warning.



Unfortunately Jaybird does not yet fully use this exception hierarchy, we are working to improve this with the next versions of Jaybird.

Each of three layers in Jaybird use exceptions most appropriate to the specific layer.

- Subclasses of `java.sql.SQLException` are thrown by the JDBC, XCA, and GDS layers. In addition to the JDBC-provided exceptions, Jaybird has a few subclasses that might be interesting to an application:
    - ***Removed in:*** *Jaybird 6* `org.firebirdsql.jdbc.FBDriverConsistencyCheckException` — this exception is thrown when driver detects an internal inconsistent state. SQL state is `HY000`.
    - `org.firebirdsql.jdbc.FBDriverNotCapableException` — this exception is thrown when an unsupported method is called, extends `java.sql.SQLFeatureNotSupportedException`. SQL state is `0A000`.
    - `org.firebirdsql.jdbc.FBSQLParseException` — this exception is thrown when incorrect escaped syntax is detected when parsing JDBC escapes. SQL state is `42000`.
    - `org.firebirdsql.jdbc.field.TypeConversionException` — this exception is thrown when the driver is asked to perform a type conversion that is not defined in the JDBC specification, or otherwise not supported by Jaybird. For a table of allowed type conversions see Data Type Conversion Table.
- Subclasses of `javax.transaction.XAException` are thrown when an XA protocol error happens in the XCA layer. Similar to the previous case, `XAException` can wrap the `SQLException` or

`GDSException`, which are extracted during exception conversion to preserve the error code.

## 3.4. SQL states

Jaybird supports the SQLstate values from the SQL:2003 standard,[1] however only few states nicely map into the Firebird error codes.

Applications can use the SQLstate codes in the error handling routines which should handle errors that are returned from different databases. But since there is little agreement between RDBMS vendors, this method can be used only for very coarse error distinction.

## 3.5. Useful Firebird error codes

Contrary to the SQLstates, the Firebird native error codes are extremely useful to determine the type of an error that happened.

Here you can find a short list of error codes, symbolic names of a corresponding constant in a `org.firebirdsql.gds.ISCConstants` class, the error message and short explanation of an error.

**TODO** Needs revising now Jaybird tries to pull the most important error code to the top

### 3.5.1. DDL Errors

DDL errors happen during execution of DDL requests, and two primary error codes are used in Firebird while executing the DDL operations. There are few other rare cases not mentioned here, but the corresponding error messages contain enough information to understand the reason of an error.

| | | |
|---|---|---|
| 335544351L | `isc_no_meta_update` | `"unsuccessful metadata update"`<br><br>This error is returned when the requested DDL operation cannot be completed, for example the application tries to define a primary key that will exceed the maximum allowed key size. |
| 335544510L | `isc_lock_timeout` | In combination with `isc_obj_in_use` (335544453L), this means that the DDL command tries to modify an object that is used in some other place, usually in another transaction. The complete error message will contain the name of the locked object. |
| 335544569L | `isc_dsql_error` | If the third error code is either `isc_dsql_datatype_err` or `isc_dsql_command_err`, then additional error codes and arguments specify the reason why the operation has failed. |

### 3.5.2. Lock Errors

Lock errors are reported by Firebird primarily when the application tries to modify a record which is already modified by a concurrent transaction. Depending on the transaction parameters such error can be reported either right after detection or after waiting some defined timeout hoping that concurrent transaction will either commit or rollback and eventually release the resource. More information on transaction locking modes can be found in section Using transactions.

| | | |
|---|---|---|
| 335544345L | isc_lock_conflict | "lock conflict on no wait transaction"<br><br>This error is returned when a "no wait" transaction needs to acquire a lock but finds another concurrent transaction holding that lock.<br><br>Instead of waiting the predefined timeout hoping that concurrent transaction will either commit or rollback, an error is returned to notify an application about the situation. |
| 335544510L | isc_lock_timeout | "lock time-out on wait transaction"<br><br>Similar to the isc_lock_conflict, but this error is returned when the lock timeout that was specified for the current transaction expired while waiting for a lock.<br><br>Another source of this error are DDL operations that try to obtain a lock on a database object that is currently used in some other place. |
| 335544336L | isc_deadlock | "deadlock"<br><br>Two transactions experience a deadlock when each of them has a lock on a resource on which the other is trying to obtain a lock. |

### 3.5.3. Referential Integrity Errors

Referential integrity constraints ensure that the database remains in a consistent state after the DML operation and/or whole transaction is completed. Three primary error codes are returned when the defined constraints are violated. The error messages are self-explanatory.

| | | |
|---|---|---|
| 335544665L | isc_unique_key_violation | violation of PRIMARY or UNIQUE KEY constraint "{0}" on table "{1}" |
| 335544558L | isc_check_constraint | Operation violates CHECK constraint {0} on view or table {1} |
| 335544466L | isc_foreign_key | violation of FOREIGN KEY constraint "{0}" on table "{1}" |

### 3.5.4. DSQL Errors

This group contains secondary codes for the primary error code `isc_dsql_error` (`335544569L`), that has a message `"Dynamic SQL Error"`.

In most situations, Jaybird 3 and higher will put this secondary error code in the `SQLException` instead of `isc_dsql_error`.

| | | |
|---|---|---|
| 335544573L | `isc_dsql_datatype_err` | `"Data type unknown"`<br><br>Usually this error is reported during DDL operation when the specified data type is either unknown or cannot be used in the specified statement. However it can also happen in DML operation, e.g. when an `ORDER BY` clause contains unknown collation, or if a parameter is used in a `SELECT` clause without explicit cast. |
| 335544570L | `isc_dsql_command_err` | `"Invalid command"`<br><br>Error happens either during parsing the specified SQL request or by handling the DDL command. |

### 3.5.5. Other Errors

This table contains other errors that might be interesting to the application developer, however they do not fall into any of the previous categories.

| | | |
|---|---|---|
| 335544321L | `isc_arith_except` | `"arithmetic exception, numeric overflow, or string truncation"`<br><br>Happens at runtime when an arithmetic exception happens, like division by zero or the numeric overflow (e.g. number does not fit the 64 bits limit).<br><br>Another source of this error are string operations, like string concatenation producing a too long string, impossibility to transliterate characters between character sets, etc.<br><br>Future versions of Firebird will provide a secondary code to distinguish the exact reason of an error. |

| | | |
|---|---|---|
| 335544348L | isc_no_cur_rec | "no current record for fetch operation"<br><br>Happens when the application asks Firebird to fetch a record, but no record is available for fetching.<br><br>Java applications should never get this error, since checks in the JDBC driver should prevent the application from executing a fetch operation on the server side. |
| 335544374L | isc_stream_eof | "attempt to fetch past the last record in a record stream"<br><br>Application tries to execute fetch operation after all records have already been fetched.<br><br>Similar to the previous error, Java applications should not get this error due to the checks that happen before issuing the fetch request to the server. |
| 335544517L | isc_except | "exception {0}"<br><br>An custom exception has been raised on the server. Java application can examine the underlying GDSException to extract the exception message. |
| 335544721L | isc_network_error | Unable to complete network request to host "{0}"<br><br>This error is thrown when Jaybird cannot establish a connection to the database server due to a network issues, e.g. host name is specified incorrectly, Firebird has not been started on the remote host, firewall configuration prevents client from establishing the connection, etc. |

[1] it is possible sometimes X/Open SQLstates are used

[2] excluding those defined for javax.sql.rowset

# Chapter 4. Executing statements

After obtaining a connection, the next thing to do is to execute an SQL statement. The JDBC specification distinguishes three kinds of statements:

1. Regular statements to execute SQL statements without parameters,
2. prepared statements to execute SQL statements with parameters,
3. and callable statements to execute stored procedures.

## 4.1. The java.sql.Statement interface

The `java.sql.Statement` interface is the simplest interface to execute SQL statements. It distinguishes three types:

- statements that return a result set, or, in other words, queries;
- statements that change the state of the database but return no result set;
- `INSERT` statements (or other statements with similar behaviour) that return the values of the columns which were generated by the database engine while inserting the record.

Let's check one of the typical usages shown below. In general the usage pattern of the statement consists of three steps.

*Typical way to execute query to get information about the user*

```
try (Statement stmt = connection.createStatement(); ①
     ResultSet rs = stmt.executeQuery(
         "SELECT firstName, lastName FROM users" +
         " WHERE userId = 5") ②
){
  if (rs.next()) {
    String firstName = rs.getString(1);
    String lastName = rs.getString(2);
  }
} ③
```

① Create a `Statement` object by calling the `createStatement()` method of the `Connection` object.

② Use the Statement object by calling its methods, in our case we execute a simple query `SELECT firstName, lastName FROM users WHERE userId = 5`. Processing of the query result will be discussed in details in Working with result sets.

③ Close the result set and statement to release all allocated resources. In our example this is done using the try-with-resources block. With try-with-resources, Java takes care of closing resources in the right order, even if exceptions occur, or if a resource was not allocated (say, if `executeQuery` throws an exception).

As the connection object is the factory for the statement objects, this puts a constraint on the object lifetime: statements are bound to the connection; when the connection is closed, all statements that

were created by that connection become invalid and the resources allocated by them are released. However, despite that these resources are released when the connection closes, it is strongly recommended to use the try-with-resources block, to guarantee that resources are released as soon as possible because of reasons that will be discussed later.

A statement can be executed using the following methods:

- `Statement.executeQuery(String)` — executes a `SELECT` or other result set producing statement, and returns a result set. If the specified statement does not produce a result set, an `SQLException` is thrown after statement execution[1].
- `Statement.executeUpdate(String)` — executes other DML (Data Manipulation Language, e.g. `INSERT`, `UPDATE`, `DELETE`) or DDL (Data Definition Language[2]) statements and returns the number of updated rows — as `int`. If the specified statement is a query or otherwise produces a result set, an `SQLException` is thrown.
- `Statement.executeLargeUpdate(String)` — same as previous, except it returns the update count as `long`.
- `Statement.execute(String)` — executes a statement and returns `true` when the statement returned a result set, or `false` if the result has no result set, but has an update count or nothing as its result. You can use `Statement.getResultSet()` to get the result of the executed query, or you can use `Statement.getUpdateCount()` or `getLargeUpdateCount()` when you have executed an update statement.

  Formally, this method is also used for statements with multiple results (result sets and update counts), but Firebird only supports at most one result set and at most one update count.

These `execute` methods have several variants for additional features covered in the reference section Generated keys retrieval.

A statement is closed by calling the `Statement.close()` method, or by using a try-with-resources which calls `close()` behind the scenes. After a close, the statement object is invalid and cannot be used any more.

It is allowed to use the same statement object to execute different statements, one by one. The code below contains a short example which first performs a select to find the ID of the user 'Joe Doe', and if the record is found, it enables his account.



Concatenation of values into a query string as done in this example is not a good practice as it can leave your code vulnerable to SQL injection.

In this specific case it is safe to do as the values are integers. In general: don't do this, use a prepared statement with parameters instead.

*Using the same statement object multiple times to enable user account*

```
try (Statement stmt = connection.createStatement();
     ResultSet rs = stmt.executeQuery(
         "SELECT userId FROM users " +
         "WHERE lastName = 'Doe' AND firstName = 'Joe'")) {
  if (rs.next()) {
```

```
    int userId = rs.getInt(1);

    int rowsUpdated = stmt.executeUpdate(
        "UPDATE accounts SET accountEnabled = 1 " +
        "WHERE userId = " + userId);

    if (rowsUpdated == 0)
      rowsUpdated = stmt.executeUpdate(
          "INSERT INTO accounts (userId, enabled) " +
          "VALUES (" + userId + ", 1)");

    if (rowsUpdated != 1)
      throw new SomeException(
          "User was not updated correctly.");
  }
}
```

The way the code is constructed is quite tricky because of the result set lifetime constraints that are defined by the JDBC specification, please read the chapter Working with result sets for more details.

However, here it is done intentionally to emphasize that a single object is used to execute `SELECT` and `UPDATE`/`INSERT` statements. It also shows how to check whether the executed statement modified the expected number of rows — the application first tries to update the account and only if no rows were updated, it inserts new record into the `accounts` table.



This example of 'try update, then insert' approach can be better handled using `MERGE` [https://firebirdsql.org/file/documentation/html/en/refdocs/fblangref40/firebird-40-language-reference.html#fblangref40-dml-merge] or `UPDATE OR INSERT` [https://www.firebirdsql.org/file/documentation/html/en/refdocs/fblangref40/firebird-40-language-reference.html#fblangref40-dml-update-or-insert].

When an application needs to execute DDL statements, it is recommended to use the `Statement.execute(String)` method, as in this case the amount of modified records makes little sense. The next example shows the creation of database tables using this method.

*Example of creating database tables*

```
try (Statement stmt = connection.createStatement()) {
  stmt.execute("CREATE TABLE customer(" +
      "customerId INTEGER NOT NULL PRIMARY KEY, " +
      "firstName VARCHAR(20) NOT NULL, " +
      "lastName VARCHAR(40) NOT NULL)");
}
```

As mentioned earlier, the `Statement.execute(String)` method can also be used to execute statements of an unknown type.

```
try (Statement stmt = connection.createStatement()) {
```

```
    boolean hasResultSet = stmt.execute(sql);
    if (hasResultSet) {
      ResultSet rs = stmt.getResultSet();
      ...
    } else {
      int updateCount = stmt.getUpdateCount();
      ...
    }
  }
```

According to the JDBC specification, the `getResultSet()` and `getUpdateCount()` methods should only be called once per result.

***Removed in:*** *Jaybird 5.0.5* In Jaybird 5.0.4 and earlier, calling the `getResultset()` method a second time throws an exception.
***Since:*** *Jaybird 5.0.5* Since Jaybird 5.0.5, calling `getResultSet()` multiple times returns the same instance of `ResultSet` it returned from the first call to `getResultSet()` or from `executeQuery()`.

# 4.2. Statement behind the scenes

The previous examples requires us to discuss the statement object dynamics, its lifecycle and how it affects other subsystems in details.

## 4.2.1. Statement dynamics

When a Java application executes a statement, a lot more operations happen behind the scenes:

1. A new statement object is allocated on the server. Firebird returns an identifier of the allocated object, a statement handle, that must be used in next operations.
2. An SQL statement is compiled into an executable form and is associated with the specified statement handle.
3. Jaybird asks the server to describe the statement and Firebird returns information about the statement type and possible statement input parameters (we will discuss this with prepared statements) and output parameters, namely the result set columns.
4. If no parameters are required for the statement, Jaybird tells Firebird to execute the statement, passing the statement handle to the corresponding method.

After this Jaybird has to make a decision depending on the operation that was called.

- If `Statement.execute()` was used, Jaybird only checks the statement type to decide whether it should return `true`, telling the application that there is a result set for this operation, or `false`, if the statement does not return any result set.
- If `Statement.executeUpdate()` was called, Jaybird asks Firebird to give the information about the number of affected rows. This method can be called only if the statement type tells that no result set can be returned by the statement.

  When called for queries, an exception is thrown even though the statement was successfully

executed on the server.

- If `Statement.executeQuery()` was called and the statement type indicates that a result set can be returned, Jaybird constructs a `ResultSet` object and returns it to the application. No additional checks, like whether the result set contains rows, are performed, as that is the responsibility of the `ResultSet` object.

  If this method is used for statements that do not return result set, an exception is thrown even though the statement was successfully executed on the server.



The described behaviour may change in the future by throwing the exception **before** executing the statement.

When an application does not need to know how many rows were modified, it should use the `execute()` method instead of `executeUpdate()`. This saves an additional call to the server to get the number of modified rows which can increase performance in situations where network latency is comparable with the statement execution times.

The `execute()` method is also the only method that can be used when the application does not know what kind of statement is being executed (for example, an application that allows the user to enter SQL statements to execute).

After using the statement object, an application should close it. Two different possibilities exist: to close the result set object associated with the statement handle and to close the statement completely.

If, for example, we want to execute another query, it is not necessary to completely release the allocated statement. Jaybird only needs to compile a new statement on the existing statement handle before using it, in other words we can skip step 1 (allocating a new statement handle). This saves us one round-trip to the server over the network, which might improve application performance.

If we close the statement completely, the allocated statement handle is no longer usable. In theory, Jaybird could allocate a new statement handle, however the JDBC specification does not allow use of a `Statement` object after `close()` has been called.

### 4.2.2. Statement lifetime and DDL

Step 2 (compiling the SQL statement) in the previous section is probably the most important, and usually, most expensive part of the statement execution life cycle.

When Firebird server receives a "prepare statement" call, it parses the SQL statement and converts it into the executable form: BLR, or Binary Language Representation. BLR contains low-level commands to traverse the database tables, conditions that are used to filter records, defines the order in which records are accessed, indices that are used to improve the performance, etc.

When a statement is prepared, it holds references to all database object definitions that are used during that statement execution. This mechanism preserves the database schema consistency, it saves the statement objects from "surprises" like accessing a database table that has been removed by another application.

However, holding references to those database objects has one very unpleasant effect: it is not possible to upgrade the database schema, if there are active connections to the database with open statements referencing objects being upgraded. In other words, if two application are running and one is trying to modify a schema, table, view, procedure, function, package or trigger definition while another one is accessing those objects, the first application will receive an error 335544453 "*object is in use*".

To avoid this problem, it is strongly recommended to close a statement as soon as it is no longer needed. This invalidates the BLR and release all its references to database objects, making them available for modification.

Special care should be taken when statement pooling is used. In that case, statements are not released even if the `close()` method is called. Likely, the only possibility to close pooled statements is to close the pooled connections. Please check the documentation of your connection pool for more information.

## 4.3. The java.sql.PreparedStatement interface

As we have seen, Jaybird already performs internal optimization when it comes to multiple statement execution — it can reuse the allocated statement handle in subsequent calls. However, this improvement is very small and sometimes can even be negligible compared to the time needed to compile a SQL statement into its BLR form.

The `PreparedStatement` interface addresses such inefficiencies. An object that implements this interface represents a precompiled statement that can be executed multiple times. If we use the execution flow described in the "Statement dynamics" section, it allows us to go directly to step 4 for repeated executions.

However, executing the same statement with the same values rarely makes sense, unless we want to fill the table with the same data, which usually is not the case. Therefore, JDBC provides support for parametrized statements — SQL statements where literals are replaced with question marks ('?'), so-called positional parameters. The application then assigns values to the parameters before executing the statement.

Our first example in this chapter can be rewritten as shown below. At first glance the code becomes more complicated without any visible advantage.

*Example for user account update rewritten using prepared statements*

```
try (PreparedStatement stmt1 = connection.prepareStatement(
          "SELECT userId FROM users WHERE " +
          "lastName = ? AND firstName = ?")) {
  stmt1.setString(1, "Doe");
  stmt1.setString(2, "Joe");
  try (ResultSet rs = stmt1.executeQuery()) {

    if (rs.next()) {
      int userId = rs.getInt(1);

      try (PreparedStatement stmt2 =
```

```
                connection.prepareStatement(
                    "UPDATE accounts SET accountEnabled = 1 " +
                    "WHERE userId = ?" )) {
          stmt2.setInt(1, userId);

          int rowsUpdated = stmt2.executeUpdate();

          if (rowsUpdated == 0) {
            try (PreparedStatement stmt3 =
                     connection.prepareStatement(
                         "INSERT INTO accounts " +
                         "(userId, enabled) VALUES (?, 1)")) {
              stmt3.setInt(1, userId);
              rowsUpdated = stmt3.executeUpdate();
            }
          }
          if (rowsUpdated != 1)
            throw new SomeException(
                "User was not updated correctly.");
        }
      }
    }
  }
```

- First, instead of using just one statement object we have to use three, one per statement.
- Second, before executing the statement we have to set parameters first. As is shown in the example, parameters are referenced by their position. The `PreparedStatement` interface provides setter methods for all primitive types in Java as well as for some widely used SQL data types (BLOBs, CLOBs, etc). The `NULL` value is set by calling the `PreparedStatement.setNull(int)` method.
- Third, we are now forced to use four nested try-with-resources blocks, which makes code less readable.

So, where's the advantage? First of all, prepared statements parameters protect against SQL injection as the values are sent separately from the statement itself. It is not possible to change the meaning of a statement due to incorrect string concatenation, so data leaks or other problems caused by SQL injection can be avoided. Second of all, the driver handles conversion of Java object types to the correct format for the target datatype in Firebird: you don't need to convert a Java value to the correct string literal format for Firebird's SQL dialect.

To address some identified problems, we can redesign our application to prepare those statements before calling that code (for example in a constructor) and close them when the application ends. In that case the code can be more compact (see the next example). Unfortunately, the application is now responsible for prepared statement management. When a connection is closed, the prepared statement object will be invalidated, but the application will not be notified. And when the application uses similar statements in different parts of the application, the refactoring might affect many classes, possibly destabilizing the code. So, the refactoring on this example is not something we want to do.

*Rewritten example to let application manage prepared statements*

```
// prepared statement management
PreparedStatement queryStmt =
    connection.prepareStatement(queryStr);
PreparedStatement updateStmt =
    connection.prepareStatement(updateStr);
PreparedStatement insertStmt =
    connection.prepareStatement(insertStr);

.....................

// query management
queryStmt.clearParameters();
queryStmt.setString(1, "Doe");
queryStmt.setString(2, "Joe");
try (ResultSet rs = queryStmt.executeQuery()) {
  if (rs.next()) {
    int userId = rs.getInt(1);

    updateStmt.clearParameters();
    updateStmt.setInt(1, userId);
    int rowsUpdated = updateStmt.executeUpdate();

    if (rowsUpdated == 0) {
      insertStmt.clearParameters();
      insertStmt.setInt(1, userId);
      rowsUpdated = insertStmt.executeUpdate();
    }

    if (rowsUpdated != 1)
      throw new SomeException(
          "User was not updated correctly.");
  }
}
.....................

// prepared statement cleanup
insertStmt.close();
updateStmt.close();
queryStmt.close();
```

The answer to the advantage question is hidden in the `prepareStatement(String)` call. Since the same statement can be used for different parameter values, the connection object could have a possibility to perform prepared statement caching. A JDBC driver can ignore the request to close the prepared statement, save it internally and reuse it each time application asks to prepare an SQL statement that is known to the connection.



Jaybird currently does not perform statement caching

# 4.4. The java.sql.CallableStatement interface

The `CallableStatement` interface extends `PreparedStatement` with methods for executing and retrieving results from stored procedures. It was introduced in the JDBC specification to unify access to the stored procedures across different database systems. The main difference to `PreparedStatement` is that the procedure call is specified using the portable escape syntax[3]:

*Unified escape syntax for stored procedure execution*

```
procedure call ::= {[?=]call <procedure-name>(<params>)}
params ::= <param> [, <param> ...]
```

Each stored procedure is allowed to take zero or more input parameters, similar to the `PreparedStatement` interface. After being executed, a procedure can either return data in the output parameters or it can return a result set that can be traversed. Though the interface is generic enough to support database engines that can return both and have multiple result sets. These features are of no interest to Jaybird users, since Firebird does not support them.

The IN and OUT parameters are specified in one statement. The syntax above does not allow to specify the type of the parameter, therefore additional facilities are needed to tell the driver which parameter contains output values, the rest are considered to be IN parameters.



It is perfectly fine to execute stored procedures using a normal `Statement` or `PreparedStatement`. In fact, most of the time it's simpler to do so.

## 4.4.1. Firebird stored procedures

Firebird stored procedures represent a piece of code written in the PSQL language that allows SQL statement execution at the native speed of the engine and provides capabilities for a limited execution flow control. The PSQL language is not a general purpose language, therefore its capabilities are limited when it comes to interaction with other systems. UDR-based procedures can be used instead for such purposes.

Firebird stored procedures can be classified as follows:

- Procedures that do not return any results. These are stored procedures that do not contain the `RETURNS` keyword in their header.
- Procedures that return only a single row of results. These are stored procedures that contain the `RETURNS` keyword in their header, but do not contain the `SUSPEND` keyword in their procedure body. These procedures can be viewed as functions that return multiple values. These procedures are executed by using the `EXECUTE PROCEDURE` statement.
- Procedures that return a result set, also called "selectable stored procedures" (some other DBMSes call these "table-valued functions"). These are stored procedures that contain the `RETURNS` keyword in their header and the `SUSPEND` keyword in their procedure body, usually within a loop. Selectable procedures are executed using the `"SELECT * FROM myProcedure(…)"` SQL statement. It is also allowed to use the `EXECUTE PROCEDURE` statement, however that might produce strange results, since for selectable procedures it is equivalent to executing a `SELECT`

statement, but doing only one fetch after the select. If the procedure implementation relies on the fact that all rows that it returns must be fetched, the logic will be broken.

Consider the following stored procedure that returns factorial of the specified number.

*Source code for the procedure that multiplies two integers*

```
CREATE PROCEDURE factorial(
  max_value INTEGER
) RETURNS (
  factorial INTEGER
) AS
  DECLARE VARIABLE temp INTEGER;
  DECLARE VARIABLE counter INTEGER;
BEGIN
  counter = 0;
  temp = 1;
  WHILE (counter <= max_value) DO BEGIN
    IF (counter = 0) THEN
      temp = 1;
    ELSE
      temp = temp * counter;
    counter = counter + 1;
  END
  factorial = temp;
END
```

This procedure can be executed using the EXECUTE PROCEDURE call. When it is done in isql, the output looks as follows.

*Output of the EXECUTE PROCEDURE call in isql*

```
SQL> EXECUTE PROCEDURE factorial(5);

   FACTORIAL
============
         120
```

Now let's modify this procedure to return each intermediate result to the client.

*Modified procedure that returns each intermediate result*

```
CREATE PROCEDURE factorial_selectable(
  max_value INTEGER
) RETURNS (
  row_num INTEGER,
  factorial INTEGER
) AS
  DECLARE VARIABLE temp INTEGER;
  DECLARE VARIABLE counter INTEGER;
```

```
BEGIN
  counter = 0;
  temp = 1;
  WHILE (counter <= max_value) DO BEGIN
    IF (row_num = 0) THEN
      temp = 1;
    ELSE
      temp = temp * counter;
    factorial = temp;
    row_num = counter;
    counter = counter + 1;
    SUSPEND;
  END
END
```

If you create this procedure using the isql command line tool and then execute `SELECT * FROM factorial_selectable(5)`, the output will be like this:

*Output of the modified procedure*

```
SQL> SELECT * FROM factorial_selectable(5);

     ROW_NUM    FACTORIAL
============ ============
           0            1
           1            1
           2            2
           3            6
           4           24
           5          120
```

### 4.4.2. Using the CallableStatement

Let's see how the procedures defined above can be accessed from Java.

First, we can execute this procedure from the first example in the previous section using `EXECUTE PROCEDURE` and `PreparedStatement`, however this approach requires some more code for result set handling.

*Example of using the PreparedStatement to call executable procedure*

```
try (PreparedStatement stmt = connection.prepareStatement(
         "EXECUTE PROCEDURE factorial(?)")) {

  stmt.setInt(1, 2);

  try (ResultSet rs = stmt.executeQuery()) {
    rs.next(); // move cursor to the first row

    int result = rs.getInt(1);
```

```
  }
}
```

However, the standard for calling stored procedures in JDBC is to use the `CallableStatement`. The call can be specified using the escaped syntax, but native Firebird `EXECUTE PROCEDURE` syntax is also supported.

*Accessing the executable procedure via CallableStatement*

```
try (CallableStatement stmt = connection.prepareCall(
        "{call factorial(?,?)}")) {

  stmt.setInt(1, 2);
  stmt.registerOutParameter(2, Types.INTEGER);

  stmt.execute();

  int result = stmt.getInt(2);
}
```

Please note the difference in the number of parameters used in the examples. The first example contained only IN parameter on position 1 and the OUT parameter was returned in the `ResultSet` on the first position, so it was accessed via index 1.

The latter example additionally contains the OUT parameter in the call. We have used the `CallableStatement.registerOutParameter` method to tell the driver that the second parameter in our call is an OUT parameter of type INTEGER. Parameters that were not marked as OUT are considered by Jaybird as IN parameters. Finally, the `EXECUTE PROCEDURE factorial(?)` SQL statement is prepared and executed. After executing the procedure call, we get the result from the appropriate getter method.

It is worth mentioning that the stored procedure call preparation happens in the `CallableStatement.execute` method, and not in the `prepareCall` method of the `Connection` object. Reason for this deviation from the specification is that Firebird does not allow to prepare a procedure without specifying parameters and set them only after the statement is prepared. Another side effect of this issue is, that it is allowed to intermix input and output parameters, for example in the "IN, OUT, IN, OUT, OUT, IN" order. Not that it makes much sense to do this, but it might help in some cases when porting applications from another database server.

It is also allowed to use a procedure call parameter both as an input and output parameter. It is recommended to use this only when porting applications from the database servers that allow INOUT parameter types, such as Oracle.



Future Jaybird versions may become stricter in how they handle IN and OUT parameters. They will likely require a strict ordering of parameters to match the actual procedure definition, so interleaving IN and OUT, or marking a parameter position as both IN and OUT may no longer be supported.

Our recommendation: always use the order of IN parameters in the procedure

definition, followed by the OUT parameters in the `RETURNS` clause of the definition.

Mixing order of parameters will then probably require use of Firebird 6.0 and its native `CALL` statement. This is not final yet, and we may offer a backwards compatibility option, for a transition period of at least one major Jaybird version.

The actual stored procedure call using the `CallableStatement` is equivalent to the call using the prepared statement as shown in the first example. There is no measurable performance differences when using the callable statement interface.

The JDBC specification allows another syntax for the stored procedure calls:

*Calling stored procedure using different syntax*

```
try (CallableStatement stmt = connection.prepareCall(
         "{?= call factorial(?}")) {

  stmt.registerOutParameter(1, Types.INTEGER);
  stmt.setInt(2, 2);

  stmt.execute();

  int result = stmt.getInt(1);
}
```

Note, that input parameter now has index 2, and not 1 as in the previous example. This syntax seems to be more intuitive, as it looks like a function call. It is possible to use this syntax for stored procedures that return more than one parameter by combining code from the second and the last examples.



Future Jaybird versions may become stricter in how they handle IN and OUT parameters, and especially the `{?=call ...}` syntax may be restricted to only support procedures with a single OUT parameter (i.e. a single parameter in the `RETURNS` clause of its definition).

This is not final yet, and we may offer a backwards compatibility option, for a transition period of at least one major Jaybird version.

Firebird stored procedures can also return result sets. This is achieved by using the `SUSPEND` keyword inside the procedure body. This keyword returns the current values of the output parameters as a single row to the client.

The following example is more complex and shows a stored procedure that computes a set of factorial of the numbers up to the specified number of rows.

The `SELECT` SQL statement is the natural way of accessing the selectable procedures in Firebird. You "select" from such procedures using the `Statement` or `PreparedStatement` objects.

With minor issues it is also possible to access selectable stored procedures through the

`CallableStatement` interface. The escaped call must include all IN and OUT parameters. After the call is prepared, parameters are set the same way. However, the application must explicitly tell the driver that selectable procedure is used and access to the result set is desired. This is done by calling a Jaybird-specific method as shown in the example below. When this is not done, the application has access only to the first row of the result set. **TODO** Outdated?

The getter methods from the `CallableStatement` interface will provide you access only to the first row of the result set. To get access to the complete result set you have to either call the `executeQuery` method or the `execute` method followed by `getResultSet` method.

*Example of using selectable stored procedure via escaped syntax*

```
import java.sql.*;
import org.firebirdsql.jdbc.*;
...
try (CallableStatement stmt = connection.prepareCall(
         "{call factorial(?, ?, ?)}")) {

  FirebirdCallableStatement fbStmt =
      (FirebirdCallableStatement)stmt;
  fbStmt.setSelectableProcedure(true);

  stmt.setInt(1, 5);
  stmt.registerOutParameter(2, Types.INTEGER); // first OUT
  stmt.registerOutParameter(3, Types.INTEGER); // second OUT

  try (ResultSet rs = stmt.executeQuery()) {

    while(rs.next()) {
      int firstCol = rs.getInt(1);             // first OUT
      int secondCol = rs.getInt(2);            // second OUT
      int anotherSecondCol = stmt.getInt(3);   // second OUT
    }
  }
}
```

Note that OUT parameter positions differ when they are accessed through the `ResultSet` interface (the `firstCol` and `secondCol` variables in our example). They are numbered in the order of their appearance in the procedure call starting with 1.

When OUT parameter is accessed through the `CallableStatement` interface (the `anotherSecondCol` parameter in our example), the registered position should be used. In this case the result set can be used for navigation only.

### 4.4.3. Describing Output and Input Parameters

The `PreparedStatement.getMetaData` method is used to obtain description of the columns that will be returned by the prepared SELECT statement. The method returns an instance of `java.sql.ResultSetMetaData` interface that among other descriptions provides the following:

- column type, name of the type, its scale and precision if relevant;
- column name, its label and the display size;
- name of the table, to which this column belongs;
- information whether the column is read-only or writable, whether it contains signed numbers, whether it can contain NULL values, etc.

Additionally, the JDBC 3.0 specification introduced the interface `java.sql.ParameterMetaData` that provides similar information for the input parameters of both `PreparedStatement` and `CallableStatement` objects.



Due to the implementation specifics of the escaped syntax support for callable statements, it is not allowed to call `getParameterMetaData` before all OUT parameters are registered. Otherwise, the driver will try to prepare a procedure with an incorrect number of parameters and the database server will generate an error.

## 4.5. Batch Updates

Batch updates are intended to group multiple update operations to be submitted to a database server to be processed at once. Firebird 3.0 and earlier did not provide support for such functionality, so Jaybird 4 and earlier emulate it by issuing separate update commands.

Firebird 4.0 added support for server-side batch updates, which is supported by Jaybird for prepared statements, see Server-side Batch Updates.

### 4.5.1. Batch Updates with java.sql.Statement interface

The `Statement` interface defines three methods for batch updates: `addBatch`, `executeBatch` and `clearBatch`. You can add arbitrary `INSERT`/`UPDATE`/`DELETE` or DDL statement to the batch group. Adding a statement that returns a result set is an error.

*Example of batch updates using Statement object*

```
try (Statement stmt = connection.createStatement()) {
  stmt.addBatch("UPDATE products " +
      "SET amount = amount - 1 WHERE id = 1");
  stmt.addBatch("INSERT INTO orders(id, amount) VALUES(1, 1)");

  int[] updateCounts = stmt.executeBatch();
}
```

The JDBC specification recommends to turn auto-commit mode off (`false`) to guarantee standard behaviour for all databases. The specification explicitly states that behaviour in auto-commit case is implementation defined.

In auto-commit mode, Jaybird executes a batch in a single transaction, i.e. the "all-or-nothing" principle. A new transaction is started before the batch execution and is committed if there were no exceptions during batch execution, or is rolled back if at least one batch command generated an

error.

The `Statement.executeBatch` method submits the job to the database server. In case of successful execution of the complete batch, it returns an array of integers containing update counts for each of the commands. Possible values are:

- `0` or positive value — an update count for the corresponding update/DDL statement.
- `Statement.SUCCESS_NO_INFO` (`-2`) — driver does not have any information about the update count, but it knows that statement was executed successfully.

The `Statement.executeBatch` method closes the current result set if one is open. After execution, the batch is cleared. Calling `execute`, `executeUpdate` and `executeQuery` before the batch is executed does not have any effect on the currently added batch statements.

If at least one statement from the batch fails, a `java.sql.BatchUpdateException` is thrown. Jaybird will stop executing statements from batch after the first error. In auto-commit mode it will also roll back the transaction. An application can obtain update counts for the already executed statements using `getUpdateCounts` method of the `BatchUpdateException` class. The returned array will always contain fewer entries than there were statements in the batch, as it will only report the update counts of successfully executed statements.

### 4.5.2. Batch Updates with java.sql.PreparedStatement and java.sql.CallableStatement

Using batch updates with a prepared statement is conceptually similar to the `java.sql.Statement` approach. The main difference is that only one statement can be used, but with different sets of parameter values.

*Example of batch updates with PreparedStatement*

```
try (PreparedStatement stmt = connection.prepareStatement(
    "INSERT INTO products(id, name) VALUES(?, ?)")) {
  stmt.setInt(1, 1);
  stmt.setString(2, "apple");
  stmt.addBatch();

  stmt.setInt(1, 2);
  stmt.setString(2, "orange");
  stmt.addBatch();

  int[] updateCounts = stmt.executeBatch();
}
```

*Example of batch updates with CallableStatement*

```
try (CallableStatement stmt = connection.prepareCall(
    "{call add_product(?, ?)")) {
  stmt.setInt(1, 1);
  stmt.setString(2, "apple");
```

```
    stmt.addBatch();

    stmt.setInt(1, 2);
    stmt.setString(2, "orange");
    stmt.addBatch();

    int[] updateCounts = stmt.executeBatch();
}
```

### 4.5.3. Server-side Batch Updates

Jaybird supports server-side batch updates when connecting to Firebird 4.0 or higher. This support comes with a number of limitations:

- Only supported on the pure Java protocol, not on native or embedded.
- Only supported on `PreparedStatement`.
  - The `Statement` batch behaviour is not supported by server-side batch updates.
  - The `CallableStatement` implementation is more complex than prepared statement, so the decision was made not to reimplement this using server-side batch updates. The implementation might be rewritten in a future Jaybird version, if there is sufficient demand. As a workaround, use `execute procedure` or `{call procedure_name(...)}` from a `PreparedStatement`.
- Requesting generated-keys will fall back to emulated behaviour as server-side batches do not support returning values produced by the `RETURNING` clause.
- ***Since:*** *Jaybird 5.0.5* Using parameterless statements will fall back to emulated behaviour as server-side batches do not support executing parameterless statements.

  In Jaybird 5.0.0 - 5.0.4 this results in error "*Statement used in batch must have parameters*" (`isc_batch_param`, `335545186`). The workaround is to connect with property `useServerBatch` set to `false`, or to not use a prepared statement, or not use batch execution, for parameterless statements.

- Firebird 4.0 has additional facilities to send `BLOB` values as part of the batch update, but this is not yet used by Jaybird.

When server-side batch support is unavailable, either because the server doesn't support it, or because of above limitations, or if the connection property `useServerBatch` is `false`, Jaybird will fall back to the emulated behaviour.

By default, Jaybird will request the maximum server-side batch buffer size (256MB as of Firebird 4.0). A smaller buffer can be requested with connection property `serverBatchBufferSize` (value in bytes). Jaybird does not track the consumption of the server-side batch buffer. Attempting to execute a batch larger than this buffer will fail with error "Internal buffer overflow - batch too big" (`isc_batch_too_big`, `335545198`). The 256MB buffer used with Jaybird defaults on Firebird 4.0 is sufficient to accommodate several thousand rows at maximum row size of — for Firebird 5.0 and earlier — 64 KiB (a naive calculation says around 4000 rows, but this doesn't account for all overhead of a row).

# 4.6. Escape syntax

The escape syntax was introduced as a portable JDBC-specific syntax to represent parts of the SQL language that are (or were) usually implemented differently by database vendors. The escaped syntax is also used to define features that might not be implemented by the database server, but can have an appropriate implementation in the driver.

The JDBC specification defines escaped syntax for the following

- Scalar Functions
- Date and time literals
- Outer Joins
- Calling stored procedures
- LIKE escape character
- “Disable escape processing” escape

## 4.6.1. Scalar Functions

Escaped syntax for the scalar function call is defined as

```
{fn <function-name> (argument list)}
```

For example, `{fn concat('Firebird', 'Java')}` results in the code `('Firebird'||'Java')` in the actual statement sent to the server. "Supported JDBC Scalar Functions" provides a list of supported scalar functions.

## 4.6.2. Date and time literals

It is possible to include date and time literals in SQL statements. To guarantee that each database will interpret the literal identically, the JDBC specification provides the following syntax to specify them:

Date literal escaped syntax:

```
{d 'yyyy-mm-dd'}
```

Time literal escaped syntax:

```
{t 'hh:mm:ss'}
```

Timestamp literal syntax (fractional seconds part `'.f…'` can be omitted):

```
{ts 'yyyy-mm-dd hh:mm:ss.f...'}
```

### 4.6.3. Outer Joins

Due to the — historically — various approaches to specify outer joins (for example, the Oracle "(+)" syntax), the JDBC specification provides the following syntax:

```
{oj <outer join>}
```

where the outer join is specified as

```
<outer join> ::=
    <table name> {LEFT|RIGHT|FULL} OUTER JOIN
    {<table name> | <outer join>} ON <search condition>
```

An example SQL statement would look like this:

```
SELECT * FROM {oj tableA a
    LEFT OUTER JOIN tableB b ON a.id = b.id}
```

### 4.6.4. Stored procedures

The escaped syntax for stored procedures is described in detail in section The java.sql.CallableStatement interface.

### 4.6.5. LIKE escape character

The percent sign ('%') and underscore ('_') characters are wildcards in the LIKE predicate of SQL. To interpret them literally they must be escaped using an escape character. The escaped syntax for this case identifies which character is used as an escape character:

```
{escape '<escape character>'}
```

### 4.6.6. "Disable escape processing" escape

***Since:*** *Jaybird 5.0.12/6.0.5*

The "disable escape processing" escape ({\...\}) can be used to disable escape processing and parameter parsing within part of a statement text.

*Syntax*

```
<start>[<allowed-char>...]<end>

<start> := {\

<end> := \}
```

```
<allowed-char> :=
    <esc-backslash>
  | !! any other character than backslash !!

<esc-backslash> := \\
```

Within the escape, **all** occurrences of a backslash (\) **must** be escaped by doubling. Unescaped backslashes inside the escape will result in a `FBSQLParseException`.

*Some examples*

```
-- Input:
select {\"N\\A"\} from SOME_TABLE
-- Sent to server:
select "N\A" from SOME_TABLE

-- Input:
select {\{fn EXP(2)}\} from SOME_TABLE
-- Sent to server (results in a syntax error)
select {fn EXP(2)} from SOME_TABLE
```

As Java string literals, the escape requires a **lot** of backslashes, see the example repeated below as Java string literals.

*Same examples, but as Java string literals*

```
-- Input:
"select {\\\"N\\\\A\"\\} from SOME_TABLE"
-- Sent to server:
"select \"N\\A\" from SOME_TABLE"

-- Input:
"select {\\{fn EXP(2)}\\} from SOME_TABLE"
-- Sent to server (results in a syntax error)
"select {fn EXP(2)} from SOME_TABLE"
```

We think the use case for this escape is extremely limited, even non-existent, for Jaybird. Jaybird always offloads parameter parsing to Firebird, and Firebird currently has no syntax that could conflict with the definition of JDBC escapes.



**Possible ambiguity for comments, literals, and delimited identifiers**

Given the backslash must be escaped everywhere inside the escape, the definition in the JDBC 4.5 specification can be interpreted to mean that the escape can end inside a comment, literal, or delimited identifier (quoted identifier). Allowing this would conflict with the fact that other JDBC escapes are not parsed inside comments, literals, and delimited identifiers. Discussion with the JDBC spec lead and others did not resolve this ambiguity.

> For Jaybird, we decide to reject attempts to end the escape in a comment, literal, or delimited identifier. Attempts to do so (e.g. `{\ends/*in\}comment*/`) will raise a `FBSQLParseException`. Valid alternatives would be:
>
> - End it before the comment:
>   `{\ends\}/*in\}comment*/`
> - End it after the comment and escape the backslash in the comment:
>   `{\ends/*in\\}comment*/\}`
> - Or, don't use the escape:
>   `ends/*in\}comment*/`
>
> If a consensus is reached in the JDBC Expert Group, or a community consensus arises, we may revisit this decision.
>
> Further details can be found in jdp-2026-03: JDBC escape to disable escape processing [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2026-03-jdbc-escape-to-disable-escape-processing.adoc]

Given above ambiguity, and the lack of a real need to use it for Jaybird/Firebird, we recommend avoiding use of this escape unless absolutely necessary. If it is used, we recommend only using it for the shortest substring needed.

For example, the last example (passing the JDBC escape `{fn EXP(2)}` unprocessed to the server) could also be achieved by only disabling escape processing for the braces:

```
-- Input:
select {\{\}fn EXP(2){\}\} from SOME_TABLE
-- Sent to server (results in a syntax error)
select {fn EXP(2)} from SOME_TABLE
```

Alternatively, consider disabling escape processing for the `Statement` using `Statement.setEscapeProcessing(boolean)` [https://docs.oracle.com/en/java/javase/25/docs/api/java.sql/java/sql/Statement.html#setEscapeProcessing(boolean)] (not available for `PreparedStatement` and `CallableStatement`). Alternatively, starting with Jaybird 7, escape processing can be disabled by default by setting connection property `escapeProcessing` to `false` (also works for `PreparedStatement` and `CallableStatement`).

[1] This is an implementation detail and may change in the future to throw an exception *before* execution

[2] This term is used to group all statements that are used to manipulate database schema, i.e. creation of tables, indices, views, etc.

[3] The call escape syntax in limited form also works for `Statement` and `PreparedStatement`

# Chapter 5. Working with result sets

When a `SELECT` statement is executed, the results of the query are returned through an implementation of the `java.sql.ResultSet` interface.

## 5.1. ResultSet properties

### 5.1.1. ResultSet Types

The JDBC specification defines three types of result sets

`TYPE_FORWARD_ONLY`
: The result set is not scrollable, the cursor can only move forward.

  For isolation level `TRANSACTION_READ_COMMITTED`, the result set will return all rows that satisfy the search condition at the moment of — Firebird 4.0 and higher — query execution, or — Firebird 3.0 and earlier — fetch (which — simplified — will be every *fetch size* calls to `ResultSet.next()`). For other isolation levels, the result set will return rows that were visible at query execution.

  This is the default result set type.

`TYPE_SCROLL_INSENSITIVE`
: The result set is scrollable.

  The cursor can move back and forth, can be positioned on the specified row. Only rows that satisfy the condition at the time of query execution are visible.

`TYPE_SCROLL_SENSITIVE`
: This type is not supported by Firebird and Jaybird.

  Jaybird allows an application to ask for this type of result set, however in compliance with the JDBC specification, the type is “downgraded” to `TYPE_SCROLL_INSENSITIVE` and a corresponding warning is added to the connection object.

Due to missing support for scrollable cursors in Firebird 4.0 and earlier, support for scrollable results sets (`TYPE_SCROLL_INSENSITIVE`) is implemented by fetching the complete result set to the client. Scrolling happens in memory on the client. This can have adverse effect on the system memory usage and performance when the result set is large.

***Since:*** *Firebird 5.0*

Starting with Firebird 5.0, server-side scrollable cursors are supported, but not enabled by default. Jaybird will use scrollable non-holdable result set when connection property `scrollableCursor` is set to value `SERVER`, and the connection is a pure Java connection (not native or embedded). A future version may enable this behaviour by default.

### 5.1.2. ResultSet Concurrency

Result set concurrency specifies if a row in the result set object can be updated directly through the result set. Result sets that allow direct modification using the `ResultSet.updateXXX` methods are usually used in GUI applications which allow in-place editing of the underlying result set.

The result set concurrency is specified during statement creation and cannot be changed later. JDBC defines two types of result set concurrency, which are both supported by Jaybird:

`CONCUR_READ_ONLY`
: Read-only result set. This type is always supported. It tells the driver that direct update of the result set is not possible. All `ResultSet.updateXXX` methods throw an exception.

`CONCUR_UPDATABLE`
: Updatable result set. This is supported only under certain conditions that are needed for the driver to correctly construct a DML request that will modify exactly one row. These conditions are:

  - the `SELECT` statement references only one table;
  - all columns that are not referenced by the `SELECT` statement allow `NULL` values, otherwise it won't be possible to insert new rows;
  - the `SELECT` statement does not contain the `DISTINCT` predicate, aggregate functions, joined tables, or stored procedures;
  - the `SELECT` statement references all columns of the tables primary key definition or the `RDB$DB_KEY` column.

### 5.1.3. ResultSet Holdability

Result set holdability informs the driver whether result sets should be kept open across commits. `ResultSet.HOLD_CURSORS_OVER_COMMIT` tells the driver to keep the result set object open, while `ResultSet.CLOSE_CURSORS_AT_COMMIT` tells driver to close them on commit.

When an application calls `Connection.commit()`, the Firebird server closes all open result sets. It is not possible to tell the server to keep a result set open over commit unless "commit retaining" mode is used. This mode is global for the complete connection and is not suitable for holdability control on a statement level. Use of "commit retaining" mode also has an undesirable side effect for read-write transactions as it inhibits garbage collection. Because of these reasons "commit retaining" is not used in Jaybird during normal execution. If really needed, applications can commit the transaction keeping the result sets open by executing a `COMMIT RETAIN` SQL statement.

To support holdable result sets, Jaybird will cache all rows locally.
***Removed in:*** *Jaybird 6* In Jaybird 5 and earlier, for `TYPE_FORWARD_ONLY`, this is achieved by upgrading to `TYPE_SCROLL_INSENSITIVE`. See also ResultSet Types.
***Since:*** *Jaybird 6* Since Jaybird 6, the result set type is no longer upgraded to `TYPE_SCROLL_INSENSITIVE`, but all rows are still cached locally.



When connecting to Firebird 5.0 with Jaybird 5 or higher and connection property `scrollableCursor=SERVER`, a holdable result set will not use a server-side scrollable

cursor, but instead emulate by caching. Server-side scrollable cursors do not support the holdable behaviour.

## 5.2. ResultSet manipulation

`ResultSet` objects are created when either `Statement.executeQuery(String)` or `Statement.getResultSet()` methods are called, or when obtaining the generated keys from `Statement.getGeneratedKeys()`. `Statement.getResultSet()` is used in combination with `Statement.execute(String)` and can be called only once per result set (see the following two examples; ***Removed in:*** *Jaybird 5.0.5*).



***Removed in:*** *Jaybird 5.0.5* The implementation in Jaybird 5.0.4 and older does not allow calls to the `getResultSet()` method if the result set was already returned by `executeQuery` or a previous call to `getResultSet()`.

***Since:*** *Jaybird 5.0.5* Starting with Jaybird 5.0.5, calls to `getResultSet()` will return the current result set, even if the result set was already returned before by an `executeQuery` or the `getResultSet()` method.

*Using Statement.executeQuery(String) method*

```
try (Statement stmt = connection.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT * FROM myTable")) {
  // process result set
}
```

*Using Statement.getResultSet() method*

```
try (Statement stmt = connection.createStatement()) {
  boolean hasResultSet = stmt.execute("SELECT * FROM myTable");
  if (hasResultSet) {
    try (ResultSet rs = stmt.getResultSet()) {
      // process result set
    }
  }
}
```

### 5.2.1. Accessing the values in the result set

Depending on the type of the result set, it is possible to move the cursor either forward only (next example [#using-forward-only]), or using absolute and relative positioning (second example below [#using-scrollable-updatable]).

Values of the result set are obtained by calling the corresponding getter method depending on the type of column. For example, the `ResultSet.getInt(1)` method returns the value of the first column as an `int` value. If the value of the column is not `INTEGER`, Jaybird tries to convert it according to the conversions specified in Data Type Conversion Table. If conversion is not possible, an exception is

thrown.

There are two possibilities to obtain data from result set columns: by column label or by column position. Position of the first column is 1. Names supplied to getter methods are case-insensitive. The search only happens in column aliases — or in JDBC terminology the *column label*. If a column has no alias, the original column name is considered its alias. If there is more than one column matching the specified name (even if the original names were quoted), the first match is taken.

When getters for primitive types are used and the value in the result set is `NULL`, the driver returns the default value for that type. For example, `getInt()` method will return `0`. To know whether the value is really `0` or `NULL`, you have to call `ResultSet.wasNull()` method **after** calling the *get* method.

Getters that return object values (`getString`, `getDate`, `getObject`, etc.) will return a `null` value for columns containing `NULL`. Calling `wasNull` after object *get* methods is possible but unnecessary.

*Example of using forward-only result sets*

```
try (Statement forwardStatement = connection.createStatement();
     ResultSet rs = forwardStatement.executeQuery(
         "SELECT id, name, price FROM myTable")) {

  while(rs.next()) {
    int id = rs.getInt(1);
    String name = rs.getString("name");
    double price = rs.getDouble(3);
  }
}
```

### 5.2.2. Updating records in the result set

Updatable result sets are especially useful when the result of some query is displayed by the application which also allows the user to directly edit the data and post the changes to the database.

*Example of using scrollable and updatable result sets*

```
try (Statement scrollStatement = connection.createStatement(
         ResultSet.TYPE_SCROLL_INSENSITIVE,
         ResultSet.CONCUR_UPDATABLE);
     ResultSet rs = scrollStatement.executeQuery(
         "SELECT id, name, price FROM myTable");
  rs.absolute(1);                  // move to the first row
  rs.updateString(2, anotherName); // update the name
  rs.updateRow();                  // post changes to the db

  rs.moveToInsertRow();
  rs.updateInt(1, newId);
  rs.updateString(2, newName);
  rs.updateDouble(3, newPrice);
  rs.insertRow();
  rs.moveToCurrentRow();
```

```
  rs.relative(-2);
}
```

The code example above shows how to update the first row, insert a new row, and after that move two records backwards.

An application can also update the current row using so called “positioned updates” on named cursors. This technique can be used only with forward-only cursors, since the application can only update the row the server-side cursor points to. In case of scrollable cursors the complete result set is fetched to the client and then the server-side cursor is closed. The example below [#using-positioned-updates] shows how to use positioned updates.

First, the application has to specify the name of the cursor and the list of the columns that will be updated before the query is executed. This name is later used in the `UPDATE` statement as shown in the example.

*Example of using the positioned updates*

```
connections.setAutoCommit(false);
try (Statement selectStmt = connection.createStatement();
     Statement updateStmt = connection.createStatement()) {
  selectStmt.setCursorName("someCursor");

  try (ResultSet rs = selectStmt.executeQuery(
           "SELECT id, name, price FROM myTable " +
           "FOR UPDATE OF myColumn")) {

    while(rs.next()) {
      ...
      if (someCondition) {
        updateStmt.executeUpdate("UPDATE myTable " +
            "SET myColumn = myColumn + 1 " +
            "WHERE CURRENT OF " + rs.getCursorName());
      }
    }
  }
}
```

### 5.2.3. Closing the result set

A result set is closed by calling the `ResultSet.close()` method. This releases the associated server resources, and clears data held in the `ResultSet` object, making it available for garbage collection. It is strongly recommended to explicitly close result sets in auto-commit mode or `ResultSet.TYPE_SCROLL_INSENSITIVE` result sets, because this releases memory used for the cached data. Whenever possible, use try-with-resources.

The result set object is also closed automatically, when the statement that created it is closed or re-executed. In auto-commit mode, the result set is closed automatically if any statement is executed

on the same connection.

# Chapter 6. Using transactions

Transactions are used to group SQL statements into a single block that satisfies so-called ACID properties: Atomicity, Consistency, Isolation and Durability. In other words, all statements executed within a transaction either succeed and their results will be permanently stored in the database, or the effect of the statement execution will be undone.

## 6.1. JDBC transactions

Firebird supports multiple concurrent transactions over the same database connection. This allows applications that work via the native Firebird API to reduce the number of network connections required, which in turn can save resources on the server[1].

This model however is not used by all database engines, and the designers of the JDBC API have chosen a model where each database connection has only one active transaction. Also, unlike the Firebird model, where transactions require an explicit start, the JDBC specification requires the driver to start a transaction automatically as soon as a transactional context is needed.

The following code shows a very simple example of using transactions in JDBC where a hypothetical intruder increases salary of each employee twice and uses explicit transaction control in JDBC. They also try to hide their identity and if the operations succeeds, they commit the transaction, otherwise they roll the changes back.

*Example of explicit transaction control*

```
Connection connection = ...

connection.setAutoCommit(false); ①

try (Statement stmt = connection.createStament()) {
  stmt.executeUpdate("UPDATE employee " +
      " SET salary = salary * 2"); ②

  // ... do some more changes to database
  // to hide the identity of the person
  // that messed up the salary information
  // by deleting the audit trails logs, etc.

  stmt.executeUpdate("DELETE FROM audit_trails");

  connection.commit(); ③
} catch(SQLException ex) {
  connection.rollback(); ④
}
```

To use transactions, the application first switches auto-commit mode off (see below for more information), then creates a `java.sql.Statement` object, and executes an `UPDATE` statement. Please note, that there is no explicit transaction start: a new transaction will be started right before

executing the statement (step 2).

If we work with a database where not only referential integrity is preserved, but also reasonable security rules are encoded in the triggers, it will raise an error preventing cleaning the audit trails information. In this case the intruder chooses to undo all the changes they made, so that nobody notices anything. But if no security rules are implemented, they commit the transaction.



Firebird PSQL has an `IN AUTONOMOUS TRANSACTION` block [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-psql-coding.html#fblangref50-psql-inauton] that can be used to prevent such abuse and prevent audit-records from being wiped out by a transaction rollback. Alternatively, you can use an external table [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-ddl-table.html#fblangref50-ddl-tbl-external] to store the audit log, or use the trace facility [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fbconf/fbconf-trace.html#fbconf-trace] to log such operations.

When a connection uses explicit transaction control (auto-commit disabled), each transaction must be finished by calling the `commit()` or `rollback()` methods of the `Connection` object before the connection is closed. If a transaction was not finished, but the `close` method of the connection is called, the active transaction is rolled back automatically. If you do not commit or roll back the transaction, and do not close the connection explicitly, the rollback will only occur after the connection has been garbage collected, when the server detects the connection is gone.



Given the non-deterministic delay of garbage collection and detection of terminated connections by the server, make sure to explicitly end the transaction and close the connection. Do not rely on the garbage collector for this.

## 6.2. Auto-commit mode

Each newly created connection by default has auto-commit mode enabled. In other words, the duration of the transaction is limited by the duration of statement execution, or more formally — the transaction is ended when the statement is completed. The point when statement execution is considered complete, is defined in the specification as:

*Rules when the statement is completed in auto-commit mode*

> A `ResultSet` object is explicitly closed when
>
> - The `close` method on the `ResultSet` is executed, thereby releasing any external resources
> - The `Statement` or `Connection` object that produced the `ResultSet` is explictly [sic] closed
>
> A `ResultSet` object is implicitly closed when
>
> - The associated `Statement` object is re-executed

> - The `ResultSet` is created with a Holdability of `CLOSE_CURSORS_AT_COMMIT` and an implicit or explicit commit occurs
>
> Note – Some JDBC driver implementations may also implicitly `close` the `ResultSet` when the `ResultSet` type is `TYPE_FORWARD_ONLY` and the `next` method of `ResultSet` returns `false`.
>
> — JDBC 4.3 Specification

***Removed in:*** *Jaybird 6* In Jaybird 5 and earlier, in auto-commit mode, a `TYPE_FORWARD_ONLY` result set is implicitly closed when `next()` returned false. This was to conform to the requirements of JDBC 3.0, but since this was relaxed in JDBC 4.0 (see quote above), this behaviour was changed in Jaybird 6.

***Since:*** *Jaybird 6* In auto-commit mode, a result set will now remain open until explicitly closed using `ResultSet.close()`, when any statement is executed, when the auto-commit mode is disabled, or by the close of the `Statement` or `Connection`.

If there is an ongoing transaction and the value of the auto-commit property is changed, the current transaction is committed.

Note, when a connection is obtained via a `javax.sql.DataSource` object and container managed transactions are used (for example, the application is executing inside an EJB container, or the connection is participating in a JTA-managed distributed transaction), it is an error to call `setAutoCommit`.

Special care should be taken when using multiple statements in auto-commit mode. The earlier JDBC specifications did not fully define the rules for the statement completion as it did not define the behaviour of multiple `ResultSet` objects created using the same connection object in auto-commit mode.

Since Firebird does not allow a result set to remain open after the transaction ends, Jaybird 1.5.x and below cached the complete result set in memory when `SELECT` statements were executed and the transaction was committed. This had an adverse effect on allocated memory when the result set is big, especially when it contains `BLOB` fields. The JDBC 3.0 specification addressed this unclear situation (see above) and Jaybird 2.1 was improved to correctly handle this. This also reduces the memory footprint — the driver no longer caches non-scrollable and non-holdable result sets in memory.

However, some Java applications that do not conform to the current JDBC specifications might no longer work with Jaybird 2.1 and above unless additional steps are taken.

The piece of code below works perfectly with explicit transaction control. However, it won't work correctly with auto-commit with a driver — like Jaybird — that complies with the JDBC specification, when the `selectStmt` and `updateStmt` object are created by the same connection object (step 1). When the `UPDATE` is executed in step 3, the result set produced by the `SELECT` statement must

be closed before the execution. When the Java application tries to fetch the next record by calling the `rs.next()` method, it will receive an `SQLException` with a message "*The result set is closed*".

The correct solution to this situation is to fix the application by either using explicit transaction control, or by using two connection objects, one for the `SELECT` statement and one for the `UPDATE` statement.

*Non-compliant usage of nested statements in auto-commit mode*

```
try (Statement selectStmt = connection.createStatement(); ①
     Statement updateStmt = connection.createStatement();
     ResultSet rs = selectStmt.executeQuery(
         "SELECT * FROM myTable")) {
  while(rs.next()) { ②
    int id = rs.getInt(1);
    String name = rs.getString(2);

    updateStmt.executeUpdate("UPDATE anotherTable SET " +
        " name = '" + name + "' WHERE id = " + id); ③
  }
}
```

Unfortunately, not all applications can be changed, either because there is no source code available or, simply, because any change in the code requires complete release testing of the software. To address this, Jaybird provides the connection property `defaultHoldable` which makes result sets holdable by default. Holdable result sets are fully cached in memory, and will not be closed automatically when a transaction ends.[2] This property also affects the default holdability of result sets when auto-commit is disabled.

See Default holdable result sets for more information.

## 6.3. Read-only Transactions

A transaction can be declared read-only to reduce the possibility of lock conflicts. In general, this makes little sense for Firebird, because of its multi-generational architecture, where readers do not block writers and vice versa. However, in some cases it can be useful.

It is not allowed to connect with a read-write transaction to a database located on a read-only media, for example, a CD-ROM. The reason is that, to guarantee consistency of the read-write transactions, Firebird has to increase the transaction identifier when the transaction starts, and store the new value on the so-called Transaction Inventory Page even if no changes are made in that transaction. This requirement can be relaxed if a transaction is declared read-only and the engine ensures that no data can be modified.

Another reason is that long-running read-write transactions inhibit the process of collecting garbage, i.e. a process of identifying previous versions of the database records that are no longer needed and releasing the occupied space for the new versions. Without garbage collection the database size will grow very fast and the speed of the database operations will decrease, because the database engine will have to check all available record versions to determine the appropriate

one.

Therefore, if you are sure that application won't modify the database in the transaction, use the `setReadOnly` method of the `java.sql.Connection` object to tell the server that the transaction is read-only.

## 6.4. Transaction Isolation Levels

The SQL standard defines four isolation levels, each next one weaker than the previous. These isolation levels are also used in the JDBC specification:

*JDBC transaction isolation levels and their characteristics*

`TRANSACTION_SERIALIZABLE`

Transactions with this isolation level prohibit phantom reads, the situation when one transaction reads all rows satisfying a `WHERE` condition, another transaction inserts a row satisfying that condition, and the first transaction re-executes the statement.

`TRANSACTION_REPEATABLE_READ`

This isolation level prevents non-repeatable reads, a situation when a row is read in one transaction, then modified in another transaction, and later re-read in the first transaction. In this case different values had been read within the same transaction.

`TRANSACTION_READ_COMMITTED`

Transactions with this isolation level can see only committed records. However, it does not prevent so-called non-repeatable reads and phantom reads.

`TRANSACTION_READ_UNCOMMITTED`

The weakest isolation level, or better, a level without isolation. Such transactions can see the not yet committed changes from concurrently running transactions.

Firebird, however, defines other isolation levels: `read_committed`, `concurrency` and `consistency`. Only the `read_committed` isolation level can be mapped to the same level defined by the SQL standard. Dirty reads are prevented, non-repeatable reads and phantom reads can occur.

The `concurrency` isolation level is stronger than repeatable read isolation defined in the SQL standard and satisfies the requirements of a serializable isolation level, however, unlike RDBMSes with locking concurrency control, it guarantees better performance.

And finally, Firebird provides a `consistency` isolation level which in combination with table reservation feature guarantees a deadlock-free execution of transactions. A transaction will be prevented from starting if there is already another one with an overlapping sets of reserved tables. This isolation level guarantees truly serial history of transaction execution.

To satisfy the JDBC specification Jaybird provides the following default mapping of the JDBC transaction isolation levels to Firebird isolation levels:

- `TRANSACTION_READ_COMMITTED` is mapped to `read_committed` isolation level in Firebird — any changes made inside a transaction are not visible outside a transaction until the transaction is committed. A transaction in read-committed mode sees all committed changes made by other

transactions even if that happened after start of the current transaction.

- `TRANSACTION_REPEATABLE_READ` is mapped to `concurrency` isolation level in Firebird — any changes made inside this transaction are not visible outside a transaction until the transaction is committed. A transaction in repeatable-read sees only those changes that were committed before the transaction started. Any committed change in another transaction that happened after the start of this transaction is not visible in this transaction.
- `TRANSACTION_SERIALIZABLE` is mapped into `consistency` isolation level in Firebird — any modification to a table happens in serial way: all transactions wait until the current modification is done. This mode can be considered as a traditional pessimistic locking scheme, but the lock is placed on the whole table. See section Table Reservation for more information.

The default mapping is specified in the Jaybird code and can be overridden via the connection properties.

- via the `tpbMapping` property that specifies the name of the `ResourceBundle` with the new mapping of the isolation level;
- via the direct specification of the JDBC transaction isolation level. The following code contains an example of such operation, the values in the mapping are described in section Transaction Parameter Buffer.
- via the data source configuration.

*Overriding the default isolation level mapping*

```
Properties props = new Properties();
props.setProperty("user", "SYSDBA");
props.setProperty("password", "masterkey");
props.setProperty("TRANSACTION_READ_COMMITTED",
    "read_committed,no_rec_version,write,nowait");

Connection connection = DriverManager.getConnection(
    "jdbc:firebirdsql://localhost:3050/c:/example.fdb",
    props);
```

The property accepts the “short” names as shown, or the “long” names with the `isc_dpb_` prefix, as defined in `org.firebirdsql.jaybird.fb.constants.TpbItems` (Jaybird 5 and higher) or `org.firebirdsql.gds.ISCConstants` (older versions). For readability, we recommend using the “short” names.

The overridden mapping is used for all transactions started within the database connection. If the default mapping is overridden via the data source configuration, it will be used for all connections created by the data source.

## 6.5. Savepoints

Savepoints provide finer-grained control over transactions by providing intermediate steps within a larger transaction. Once a savepoint has been set, a transaction can be rolled back to that point without affecting preceding work.

To set a savepoint, use the following code:

*Example of using savepoints*

```
Connection connection = ...;
connection.setAutoCommit(false);

try (Statement stmt = connection.createStatement()) {
  stmt.executeUpdate(
      "INSERT INTO myTable(id, name) VALUES (1, 'John')");

  Savepoint savePoint1 =
      connection.setSavepoint("savepoint_1");

  stmt.executeUpdate(
      "UPDATE myTable SET name = 'Ann' WHERE id = 1");
  // ...

  connection.rollback(savePoint1);

  // at this point changes done by second update are undone
}
```

Note, rolling back to the savepoint automatically releases and invalidates any savepoints that were created after the released savepoint.

If the savepoint is no longer needed, you can use the `Connection.releaseSavepoint` method to release system resources. After releasing a savepoint, it is no longer possible to roll back the current transaction to that savepoint. Attempts to call the `rollback(Savepoint)` method with a released savepoint will result in an `SQLException`. Savepoints that have been created within a transaction are automatically released when that transaction is committed or rolled back.

## 6.6. Transaction Parameter Buffer

The behavior of Firebird transactions is internally controlled by the *Transaction Parameter Buffer* (TPB), which specifies different transaction properties:

- the transaction isolation level;
- the transaction's read-only or read-write mode;
- the lock conflict resolution mode — wait or no wait;
- the lock wait timeout;
- and, finally, the table reservations — their names and reservation modes.

The TPB is automatically generated depending on the transaction isolation level specified for the `java.sql.Connection` object. Additionally, if the connection is set to read-only mode, this is reflected in the TPB.

Usually there is no need to manipulate the TPB directly. However, the lock resolution mode as well

as table reservations cannot be specified by using the standard JDBC interfaces. For the cases where this is needed, Jaybird provides an extension of the JDBC standard.

*Example of specifying custom TPB*

```
FirebirdConnection fbConnection =
    connection.unwrap(FirebirdConnection.class);

TransactionParameterBuffer tpb =
    fbConnection.createTransactionParameterBuffer();

tpb.addArgument(TpbItems.isc_tpb_read_committed);
tpb.addArgument(TpbItems.isc_tpb_rec_version);
tpb.addArgument(TpbItems.isc_tpb_write);
tpb.addArgument(TpbItems.isc_tpb_wait);
tpb.addArgument(TpbItems.isc_tpb_lock_timeout, 15);

fbConnection.setTransactionParameters(tpb);
```

The above presents an example of populating the TPB with custom parameters.

The constants used in the examples in this chapter were introduced in Jaybird 5. Earlier versions have constants of the same name defined in `org.firebirdsql.gds.ISCConstants`, and similarly named constants in `org.firebirdsql.gds.TransactionParameterBuffer` (i.e. without `isc_tpb_` prefix and fully capitalized). Those older constants will be removed in Jaybird 6 in favour of `org.firebirdsql.jaybird.fb.constants.TpbItems`.

If accessing the Jaybird extension API is not possible or cumbersome, it is also possible to execute the `SET TRANSACTION` statement [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-transacs.html#fblangref50-transacs-settransac], **if** Connection property allowTxStmts is enabled.

### 6.6.1. Isolation level

Firebird supports three isolation levels: `read_committed`, `concurrency` and `consistency` which are represented by appropriate constants in the `TpbItems` class. The isolation level specifies the way the database engine processes the record versions on read operations. The `concurrency` isolation level is also often called `SNAPSHOT` and the `consistency` isolation level — `SNAPSHOT TABLE STABILITY`.

In `consistency` and `concurrency` modes, the Firebird database engine loads the different versions of the same record from disk and checks the “timestamps” of each version and compares it with the “timestamp” of the current transaction. The record version with the highest timestamp that is however lower or equal to the timestamp of the current transaction is returned to the application. This effectively returns the version of the record that was committed before the current transaction started, and guarantees that neither non-repeatable reads nor phantom reads can ever occur.

In `read_committed` mode, the Firebird database engine accesses the record version with the highest timestamp for which the corresponding transaction is marked as committed. This prevents the engine from reading the record versions which were modified in concurrent transactions that are not yet committed or were rolled back for whatever reason. However, such mode allows non-

repeatable reads as well as phantom reads if a concurrent transaction that modified records or inserted new ones has been committed.

The `read_committed` isolation mode requires another constant that specifies the behavior of the transaction when it sees a record version with a timestamp that belongs to a currently running transaction which is not yet committed.

Most applications require the `TpbItems.isc_tpb_rec_version` mode, or `READ COMMITTED RECORD VERSION`, which is shown in the code above. In this mode database engine fetches the latest committed version as described before.

The `TpbItems.isc_tpb_no_rec_version` constant tells the database engine to report a lock conflict when an uncommitted record version is seen while fetching data from the database, also known as `READ COMMITTED NO RECORD VERSION`. The outcome of the operation is then controlled by the lock resolution mode (see section Lock resolution mode).

The `TpbItems.isc_tpb_read_consistency` uses the `READ COMMITTED READ CONSISTENCY` mode introduced in Firebird 4.0.



**Firebird 4.0 and higher default to always use** `isc_tpb_read_consistency`

With default settings, Firebird 4.0 and higher will ignore `isc_tpb_rec_version` and `isc_tpb_no_rec_version`, and behave as if `isc_tpb_read_consistency` was specified. This is controlled through the `ReadConsistency` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fbconf/fbconf-firebird-cfg.html#fbconf-read-consistency] setting in `firebird.conf` or — per database — in `databases.conf`.

### 6.6.2. Read-only transactions

The read-only or read-write transaction mode is controlled by two constants:

- `TpbItems.isc_tpb_read` and
- `TpbItems.isc_tpb_write`

When the read-write mode (constant `isc_tpb_write`) is specified, the database engine stores the “timestamp” of the new transaction in the database even when no modification will be made in the transaction. The “timestamp” affects the garbage collection process, since the database engine cannot release records that were modified in transactions with higher “timestamps” even when these record versions are no longer needed (in other words, when there are already newer versions of the records). Thus, long-running read-write transaction inhibits the garbage collection even when no modifications are done in it.

Therefore, it is recommended to set the read-only mode for the transaction when it is used for read operations.



With the `READ COMMITTED CONSISTENCY` mode introduced in Firebird 4.0, even read-only transactions will inhibit garbage collection.

### 6.6.3. Lock resolution mode

Relational database systems that use pessimistic locking for concurrency control lock the records regardless of the operation type, read or write. When an application tries to read a record from the database, the database engine tries to obtain a “read lock” to that record. If the operation succeeds and the application later tries to update the record, the lock is upgraded to a “write lock”. And finally, if the resource is already locked for write, a concurrent transactions cannot lock it for reading, since the system cannot allow the transaction to make a decision based on data that might be rolled back later. This approach significantly decreases concurrency. However, databases systems that employ a record versioning mechanism do not have such restrictions because each transaction “sees” its own version of the record. The only possible conflict happens when two concurrent transactions try to obtain a “write lock” for the same database record.

Firebird belongs to the latter, and on `read_committed` and `concurrency` isolation levels it behaves appropriately — there are no lock conflicts between readers and writers, and only writers competing for the same resource raise a lock conflict. However, on the `consistency` isolation level Firebird emulates the behavior of systems with pessimistic locking — read operations will conflict with write operations. Even more, the locks are obtained for whole tables (see Table Reservation for details).

The following table summarizes the above for Firebird 2.0. It shows that read-committed or repeatable read transactions conflict only when they simultaneously update the same rows. In contrast, a `consistency` transaction conflicts with any transaction running in read-write mode, e.g. as soon as a `consistency` transaction gets write access to a table, other read-write transactions are not allowed to make changes in that table.

*Table 1. Lock conflicts within one table depending on the isolation level*

| | Read-committed, Concurrency read-write | Read-committed, Concurrency read-only | Consistency, read-write | Consistency, read-only |
|---|---|---|---|---|
| Read-committed, Concurrency read-write | some updates may conflict | | conflict | conflict |
| Read-committed, Concurrency read-only | | | | |
| Consistency read-write | conflict | | conflict | conflict |
| Consistency read-only | conflict | | conflict | |

## 6.7. Table Reservation

Table reservation allows you to specify the database tables and their corresponding access modes at the beginning of the transaction. When the transaction is started, the engine tries to obtain the requested locks for the specified tables and proceeds only when all of them were successfully obtained. Such behavior allows to create a deadlock-free execution history[3].

The table reservation is specified via a TPB and includes the table to lock, the lock mode (read or write) and lock type (shared, protected and exclusive).

*Example of using table reservation facility in Firebird*

```
FirebirdConnection connection = ...
TransactionParameterBuffer tpb =
    connection.createTransactionParameterBuffer(); ①

tpb.addArgument(TpbItems.isc_tpb_consistency); ②
tpb.addArgument(TpbItems.isc_tpb_write);
tpb.addArgument(TpbItems.isc_tpb_nowait);

tpb.addArgument(TpbItems.isc_tpb_lock_write,
    "TEST_LOCK");
tpb.addArgument(TpbItems.isc_tpb_protected);

connection.setTransactionParameters(tpb); ③

// next transaction will lock TEST_LOCK table for writing
// in protected mode
```

This shows an example of reserving the `TEST_LOCK` table for writing in a protected mode. The code does the following:

① Create a new instance of `TransactionParameterBuffer` class.

② Populate the TPB. The first three statements were described in "Transaction Parameter Buffer". The fourth call specifies that the application wants to obtain a lock on the table `TEST_LOCK` for writing. The fifth call specifies the type of the lock to obtain, in our case the protected lock.

③ Set the new TPB to be used for the next transaction.

The lock mode to the table specified in the TPB can be either

- `TpbItems.isc_tpb_lock_read` for read-only access to the table;
- or `TpbItems.isc_tpb_lock_write` for read-write access to the table.

The lock type can be either

- `TpbItems.isc_tpb_shared` for shared access to the table;
- or, `TpbItems.isc_tpb_protected` for protected access to the table;

The `TpbItems.isc_tpb_exclusive` mode was introduced in later versions of Firebird, however it behaves like `TpbItems.isc_tpb_protected` mode for all read-write transactions.

The lock conflict table depends on the isolation level of the transactions and has the following properties:

- `isc_tpb_lock_write` mode always conflicts with another `isc_tpb_lock_write` mode regardless of the lock type and transaction isolation mode;
- `isc_tpb_lock_write` always conflicts with another `isc_tpb_lock_read` mode if both transactions have `consistency` isolation, but has no conflict with shared-read locks if the other transaction

has either `concurrency` or `read_committed` isolation level;

- `isc_tpb_lock_read` mode never conflicts with `isc_tpb_lock_read` mode.

## 6.8. Transaction management statements

The Firebird syntax also defines a number of transaction management statements [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-transacs.html#fblangref50-transacs-statements]. In this section we discuss the `COMMIT [WORK]`, `ROLLBACK [WORK]` and `SET TRANSACTION` statements, which have a hard transaction boundary (either starting or ending a transaction).

In general, you should not execute these statements with Jaybird. The JDBC specification formally discourages use of statements or functionality which is accessible through the JDBC API:

> **Note:** When configuring a `Connection`, JDBC applications should use the appropriate `Connection` method such as `setAutoCommit` or `setTransactionIsolation`. Applications should not invoke SQL commands directly to change the connection's configuration when there is a JDBC method available.
>
> — java.sql.Connection, JDBC API documentation

In Jaybird 5 and older, the `COMMIT` and `ROLLBACK` statements can — sometimes — be executed, but they bring the connection in an inconsistent state, which will prevent further use of the connection. Execution of the `SET TRANSACTION` statement will always fail, as the statement is executed with an active transaction.

Since Jaybird 6, these statements are rejected by default, but support can be enabled with the connection property `allowTxStmts`.

Allowing execution of these statements can sometimes be helpful, for example for executing scripts, or simplifying access to more advanced features of transaction configuration like Table Reservation.

[1] Before InterBase was open-sourced, this allowed application developers to create multithreaded application without need to purchase additional user licenses.

[2] Other cases, e.g. closing the statement object or the connection object will still ensure that the result set object is closed. If you need result sets that can be "detached" from the statement object that created them, please look at a `javax.sql.RowSet` implementation.

[3] This approach follows the two-phase locking protocol, where all locks are acquired on the beginning of the transaction and are released only when transaction is finished.

# Chapter 7. Working with Services

In addition to normal database connections, Firebird features server-wide connections. These are used to perform various administrative tasks in Firebird, e.g. Backup and restore, Database maintenance , Database statistics . The set of API calls to perform such tasks are known under the name “Services API” or “Services Manager”. Additionally, client applications can use the Services API to get limited information about the server environment and configuration.

The execution of Services API calls can be viewed as a task triggered by the client application to be executed on the server. The parameters passed in the calls are similar to the ones that are passed to equivalent command-line tools. The processed parameter values are passed into the entry routines of the *gbak*, *gfix*, *gsec* or *gstat* utility. The output of the utility is transmitted over the network to the client application — instead of the standard out when run from the commandline.

Jaybird attempts to hide the complexity of the original API by providing a set of interfaces and their implementations to perform the administrative tasks regardless of the usage mode (i.e. remote server and embedded engine, wire protocol and access via native client library).

This chapter describes the Java API for the administrative tasks. All classes and interfaces described below are defined in the `org.firebirdsql.management` package. Each management class works as a standalone object and does not require an open connection to the server.

## 7.1. ServiceManager

The `ServiceManager` interface and the `FBServiceManager` class are defined as the common superclasses providing setters and getters for common properties as well as some common routines. The following properties can be specified:

| Name | Type | Description |
|---|---|---|
| `serverName`<br>`host` (***Removed in: Jaybird 6***) | `String` | Name or IP address of the host to which we make the Service API request. *Required.* |
| `portNumber`<br>`port` (***Removed in: Jaybird 6***) | `int` | Port to which we make the request, 3050 by default. |
|  | `serverName` and `portNumber` are replacements for `host` and `port`, which were removed in Jaybird 6. | |
| `database` | `String` | Path to the database. The meaning of the property depends on the service being invoked and will be described in each of sections below. |
| `user` | `String` | Name of the user on behalf of which all Service API calls will be executed. *Required.* |
| `password` | `String` | Password corresponding to the specified user. *Required.* |

| Name | Type | Description |
|---|---|---|
| `roleName` | `String` | Role name. *Optional.* |
| `expectedDb` | `String` | With Firebird 3.0 and higher, this is used to find the non-default security database to use when authenticating. Value is a database path or alias the user can connect to. *Optional.* |
| `authPlugins` | `String` | Comma-separated list of authentication plugins to use (ignored for Firebird 2.5 or earlier). Use `null` (the default) to use Jaybird defaults. |
| `processId` | `int` | Process id to report to the server.<br><br>Native connections will always use the actual process id and ignore this property.<br>***Since:*** *Jaybird 6* Since Jaybird 6, pure Java connections will default to the actual process id.<br><br>See also Process information. |
| `processName` | `String` | Process name to report to the server.<br><br>See also Process information. |
| `socketBufferSize` | `int` | Socket buffer size in bytes |
| `soTimeout` | `int` | Socket blocking read timeout in milliseconds (`0` uses OS default timeout) |
| `connectTimeout` | `int` | Socket connect timeout in milliseconds (`0` uses OS default timeout) |
| `wireCrypt` | `String` | Wire encryption level (`DISABLED`, `ENABLED`, `REQUIRED`, `DEFAULT`). |
| `wireCryptAsEnum` | `WireCrypt` | Alternative to `WireCrypt` to use `WireCrypt` enum. |
| `dbCryptConfig` | `String` | Database encryption config. See Database encryption support for details. |
| `wireCompression` | `boolean` | Enable wire compression (requires Firebird 3.0 or higher). Default is `false`. |
| `logger` | `java.io.OutputStream` | Stream into which the output of the remote service will be written to. *Optional.* |

The last parameter — `logger` — requires some explanation. The calls to all Services API routines are asynchronous. The client application can start the call, but there are no other means to find out whether execution of the service call is finished or not except reading the output of the service call — EOF in this case means that execution is finished.

The `FBServiceManager` converts the asynchronous calls into a synchronous call by constantly polling the service output stream. If the `logger` property is specified the received data is copied into the specified `OutputStream`, otherwise it is simply ignored and the EOF-marker is being watched.

This behavior can be changed by overriding the appropriate method in the `FBServiceManager` class and/or subclasses. The only requirement is to detach from the service manager when it is no longer needed.

## 7.2. Backup and restore

Jaybird supports server-side *gbak*, streaming *gbak*, and *nbackup* backup and restore.

### 7.2.1. Gbak backup and restore

The *gbak* backup and restore routines are defined in the `BackupManager` interface and are implemented in the `FBBackupManager` class. This class provides server-side *gbak* backups. For details on *gbak* itself, consult the Firebird Backup & Restore Utility [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gbak/firebird-gbak.html] documentation.

In addition to the setters and getters described in ServiceManager, the following properties and methods are used to specify the backup and restore paths and other configuration:

| Name | Type | Description |
|---|---|---|
| `database` | `String` | For backup, the path or alias of the database to back up. For restore, the path to the database into which the backup file will be restored. To restore a multi-file database, use the `addRestorePath(String, int)` method instead. |
| `backupPath` | `String` | For a backup, the path of the newly created backup file. For multi-file backup files, use the `addBackupPath(String, int)` method instead.<br>For a restore, the path to the single backup file. If a database should be restored from a multi-file backup, use the `addBackupPath(String)` method instead. |
| `restorePageBufferCount` | `int` | Page buffers size to configure in the restored database (restore only). If not set, the size recorded in the backup is used. |
| `restorePageSize` | `int` | Database page size (restore only). Valid values depend on the Firebird version, but should be one of the 1024, 2048, 4096, 8192, 16384 or 32768. If not set, the page size recorded in the backup is used. |
| `restoreReadOnly` | `boolean` | Set to `true` if the database should be restored in read-only mode. |

| Name | Type | Description |
|---|---|---|
| `restoreReplace` | `boolean` | Set to `true` if restore should replace the existing database with the one from backup.<br>  <br>It is easy to drop an existing database if the backup can't be restored, as the existing database is first deleted and only after that the restore process starts. To avoid such situation it is recommended to restore a database into some dummy file first and then use filesystem commands to replace the existing database with the newly created one. |
| `verbose` | `boolean` | Be verbose when writing to the log.<br><br>The service called on the server will produce lots of output that will be written to the output stream specified in logger property. |

In addition to the properties, the following methods are used to configure the paths to back up and database files when multi-file back up or restore operations are used.

| Method | Description |
|---|---|
| `addBackupPath(String)` | Add a path to a backup file from a multi-file backup. Should be used for restore operation only. |
| `addBackupPath(String, int)` | Add a path to the multi-file backup. The second parameter specifies the maximum size of the particular file in bytes. Should be used for backup operation only. |
| `addRestorePath(String, int)` | Add a path for the multi-file database. The second parameter specifies the maximum size of the database file in pages (in other words, the maximum size in bytes can be obtained by multiplying this value by `restorePageSize` parameter) |
| `clearBackupPaths()` | Clear all specified backup paths. |
| `clearRestorePaths()` | Clear all specified restore paths. This method also clears the path specified in the `database` property. |

All paths specified are paths specifications on the remote server. This has the following implications:

a. it is not possible to back up to the local or network drive unless it is mounted on the remote server;

b. it is not possible to restore from the local or network drive unless it is mounted on the remote server.

The `FBStreamingBackupManager` can be used to perform a backup and restore streaming the backup data to/from the client, see Gbak streaming backup and restore.

After configuration, use methods `backupDatabase()`, `backupMetadata()` and `restoreDatabase()` to perform the backup or restore tasks. These methods will block until the operation is finished. If the `logger` property was set, the output of the service will be written into the specified output stream, otherwise it will be ignored.[1]

*Example of backup and restore process*

```
// backup the database
var backupManager = new FBBackupManager();

backupManager.setServerName("localhost");
backupManager.setPortNumber(3050);
backupManager.setUser("SYSDBA");
backupManager.setPassword("masterkey");
backupManager.setLogger(System.out);
backupManager.setVerbose(true);

backupManager.setDatabase("C:/database/employee.fdb");
backupManager.setBackupPath("C:/database/employee.fbk");

backupManager.backupDatabase();
...
// and restore it
var restoreManager = new FBBackupManager();

restoreManager.setServerName("localhost");
restoreManager.setPortNumber(3050);
restoreManager.setUser("SYSDBA");
restoreManager.setPassword("masterkey");
restoreManager.setLogger(System.out);
restoreManager.setVerbose(true);

restoreManager.setRestoreReplace(true); // attention ①

restoreManager.setDatabase("C:/database/employee.fdb");
restoreManager.setBackupPath("C:/database/employee.fbk");

backupManager.restoreDatabase();
```

① Be careful with `restoreReplace`. It will unconditionally drop (delete) the database before restore. This can result in loss of your database if the restore fails, or if you pointed to the wrong database file.

The methods `backupDatabase(int)` and `restoreDatabase(int)` can be used to specify additional backup and restore options that cannot be specified via the properties of this class. The parameter value is a bitwise OR of the following constants:

| Constant | Description |
|---|---|
| BACKUP_CONVERT | Backup external files as tables.<br><br>By default, external tables are not backed up, only paths to the external data files are stored in the backup file. When this option is used, the backup will store the external table as if they were regular tables. On restore the tables are created as regular tables. |
| BACKUP_EXPAND | No data compression.<br><br>The *gbak* utility uses RLE compression for the strings in backup file. Using this option tells it to write strings in their full length, possibly fully consisting of empty characters, etc. |
| BACKUP_IGNORE_CHECKSUMS | Ignore checksums.<br><br>The backup utility can't back up a database with page checksum errors. Such database is considered corrupted and the completeness and correctness of the backup cannot be guaranteed. However, in some cases such errors can be ignored, e.g. when the index page is corrupted. In such cases the data in the database are OK and the error disappears when the database is restored and index is recreated.<br><br>Use this option only when checksum errors are detected and can't be corrected without full backup/restore cycle. Ensure that the restored database contains correct data afterwards. |
| BACKUP_IGNORE_LIMBO | Ignore in-limbo transactions.<br><br>The backup utility can't back up a database with in-limbo transactions. When such situation occurs, the backup has to wait for the decision about the outcome of the in-limbo transaction. After a wait timeout, an exception is thrown and the backup is aborted. This option allows to work around this situation — *gbak* looks for the most recent committed version of the record and writes it into the backup. |
| BACKUP_METADATA_ONLY | Backup metadata only.<br><br>When this option is specified, the backup utility creates a backup of only the metadata information (e.g. table and view structures, stored procedures, etc.), but no data are backed up. This allows restoring a clean database from the backup. |

| Constant | Description |
|---|---|
| BACKUP_NO_GARBAGE_COLLECT | Do not collect garbage during backup.<br><br>The backup process reads all records in the tables one by one. When cooperative garbage collection is enabled[2] the transaction that accesses the latest version of the record is also responsible for marking the previous versions as garbage. This process is time-consuming and can be switched off when creating a backup for immediate restore, reading only the latest record versions.<br><br>After backup, an operator can restore the database from the backup. In databases with many back-versions of the records, the backup-restore cycle can be faster than traditional garbage collection. |
| BACKUP_NON_TRANSPORTABLE | Use non-transportable backup format.<br><br>By default, *gbak* creates a so-called transportable backup where it does not make difference whether it is later restored on a big-endian or little-endian platform. By using this option, a non-transportable format will be used which only allows restoring the database on the same architecture. |
| BACKUP_OLD_DESCRIPTIONS | Save old style metadata descriptions.<br><br>Actually no real information exist for this option, by default it is switched off. |
| RESTORE_DEACTIVATE_INDEX | Deactivate indexes during restore.<br><br>By default, indexes are created at the beginning of the restore process, and they are updated with each record being restored from the backup file. For big tables, it is more efficient first to store data in the database and to update the index afterwards. When this option is specified, the indexes will be restored in the inactive state. The downside of this option is that the database administrator is required to activate indexes afterwards; it won't happen automatically. |
| RESTORE_NO_SHADOW | Do not restore shadow database.<br><br>If a shadow database is configured, an absolute path to the shadow is stored in the backup file. If such a backup is restored on a different system where the path does not exist (e.g. moving a database from Windows to Linux or otherwise), the restore will fail. Using this option allows to overcome such situations. |

| Constant | Description |
|---|---|
| RESTORE_NO_VALIDITY | Do not restore validity constraints.<br><br>This option is usually needed when the validity constraints (e.g. NOT NULL constraints) were added after the data were already in the database, but the database contains records that do not satisfy such constraints[3].<br><br>When this option is specified, the validity constraints won't be restored. This allows to recover the data and perform cleanup tasks. The application and/or database administrators are responsible for restoring the validity constrains afterwards. |
| RESTORE_ONE_AT_A_TIME | Commit after completing restore of each table.<br><br>By default, all data is restored in one transaction. If for some reason a complete restore is not possible, using this option will allow to restore at least some of the data. |
| RESTORE_USE_ALL_SPACE | Do not reserve space on each data page for the future record versions, useful for read-only databases. |

Example of using these options:

*Example of using extended options for restore*

```
var restoreManager = new FBBackupManager();

restoreManager.setServerName("localhost");
restoreManager.setPortNumber(3050);
restoreManager.setUser("SYSDBA");
restoreManager.setPassword("masterkey");
restoreManager.setLogger(System.out);
restoreManager.setVerbose(true);

restoreManager.setRestoreReplace(true); // attention ①

restoreManager.setDatabase("C:/database/employee.fdb");
restoreManager.setBackupPath("C:/database/employee.fbk");

// restore database with no indexes,
// validity constraints and shadow database
backupManager.restoreDatabase(
    BackupManager.RESTORE_DEACTIVATE_INDEX |
    BackupManager.RESTORE_NO_VALIDITY |
    BackupManager.RESTORE_NO_SHADOW |
    BackupManager.RESTORE_ONE_AT_A_TIME);
```

① Be careful with `restoreReplace`. It will unconditionally drop (delete) the database before restore. This can result in loss of your database if the restore fails, or if you pointed to the wrong database file.

### 7.2.2. Gbak streaming backup and restore

The `FBStreamingBackupManager` class is a sibling of `FBBackupManager`, providing streaming backup and restore. Streaming backup and restore allows client-server backup and restore, by streaming the backup from the server to the client — on backup, or from the client to the server — on restore.

The class implements interface `BackupManager` — just like `FBBackupManager`, but the methods to set backup files (`setBackupPath`, `addBackupPath`) are not supported and throw an `IllegalArgumentException`. Verbose backups are not supported.

In addition, the class defines the following methods:

| Method | Description |
|---|---|
| `setBackupOutputStream(OutputStream)` | `OutputStream` to write the backup. |
| `setRestoreInputStream(InputStream)` | `InputStream` to read the backup to restore. |
| `setBackupBufferSize(int)` | Size in bytes of the local buffer to use during backup, defaults to 30KB. |

### 7.2.3. Nbackup

The `NBackupManager` interface and its implementation `FBNbackupManager` provides *nbackup* backup and restore through the service API. This form of backup and restore is server-side, meaning that all paths are on the Firebird server. For information about *nbackup*, consult the Firebird's nbackup tool [https://firebirdsql.org/file/documentation/html/en/firebirddocs/nbackup/firebird-nbackup.html] documentation.

In addition to the setters and getters described in ServiceManager, the following methods are used to specify the backup and restore paths and properties:

| Name | Type | Description |
|---|---|---|
| `database` | `String` | For a backup, the path or alias of the database to back up.<br>For a restore, the path to the database into which the backup file will be restored. |
| `backupFile` | `String` | For a backup, the path of the newly created backup file on the server.<br>For a restore, the path to a single backup file. If a database should be restored from a multi-file backup, use the `addBackupFile(String)` method for additional files. |

| Name | Type | Description |
|---|---|---|
| `backupLevel` | `int` | The level of backup to perform. Setting a level of `0` performs a full backup. A level of `N` with `N > 0` will back up all data pages modified since the last level `N - 1` backup. Defaults to `0` if `backupGuid` has not been set. |
| `backupGuid` | `String` | Sets the GUID of a previous backup (requires Firebird 4.0 or higher). This will back up all data pages modified since the backup identified by the GUID. The GUID must be enclosed in braces ({ and }). |
| `noDbTriggers` | `boolean` | Disable database triggers for *nbackup* operations that connect to the database. |
| `inPlaceRestore` | `boolean` | Enables in-place restore (requires Firebird 4.0 or higher). This allows incremental restore, for example for a read-only replica, or a hot standby. |
| `preserveSequence` | `boolean` | Enables preserve sequence for restore or fixup (requires Firebird 4.0 or higher). This preserves the GUID and replication sequence of the original database. |
| `cleanHistory` | `boolean` | Enable the *clean history* option (requires Firebird 4.0.3 or higher). One of the properties `keepRows` *or* `keepDays` must also be set. When enabled, after performing a backup, old records from `RDB$BACKUP_HISTORY` will be removed. |
| `keepDays` | `int` | Number of days to keep backup history when `cleanHistory` is enabled. |
| `keepRows` | `int` | Number of rows (including the new backup!) to keep backup history when `cleanHistory` is enabled. |

In addition to the properties, the following methods are used to configure the paths to backup-files when multi-file restore operations are used.

| Method | Description |
|---|---|
| `addBackupFile(String)` | Add a path to a backup file from a multi-file backup. In practice, `setBackupFile` does the same as `addBackupFile`. For backup, only the first file set or added is used. For restore, the files must be in the correct order to form the chain of the level `0` backup up to the highest level or last GUID-based backup to restore. |
| `clearBackupFiles()` | Clears the list of backup files. |
| `backupDatabase()` | Perform backup. |
| `restoreDatabase()` | Perform restore. |

| Method | Description |
|---|---|
| `fixupDatabase()` | Perform the *nbackup* fixup operation. A fixup will switch a locked database back to "normal" state without merging the delta, so this is a potentially **destructive** action. The normal use-case of this option is to unlock a copy of a database file where the source database file was locked with `nbackup -L` or `ALTER DATABASE BEGIN BACKUP`. Enable `preserveSequence` to preserve the original database GUID and replication sequence. |

After configuration, use the `backupDatabase()`, `restoreDatabase()` and `fixupDatabase` methods to perform the backup or restore tasks. These methods will block until the operation is finished. No output is written to the `logger`.

*Example of nbackup backup and restore process*

```
var backupManager = new FBNBackupManager();

backupManager.setServerName("localhost");
backupManager.setPortNumber(3050);
backupManager.setUser("SYSDBA");
backupManager.setPassword("masterkey");

// backup level 1 increment against a previously performed level 0
backupManager.setDatabase("/path/to/database.fdb");
backupManager.setBackupFile("/path/to/backup_lvl_1.nbk");
backupManager.setBackupLevel(1);
backupManager.backupDatabase();

// restore level 0 and level 1 in new db restored.fdb
backupManager.clearBackupFiles();
backupManager.setDatabase("/path/to/restored.fdb");
backupManager.addBackupFile("/path/to/backup_lvl_0.nbk");
backupManager.addBackupFile("/path/to/backup_lvl_1.nbk");
backupManager.restoreDatabase();
```

## 7.3. User management



Starting with Firebird 3.0, user management through the Services API has been deprecated. You should use the SQL statements for user management [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-security-user.html] instead.

The user management routines are defined in the `UserManager` interface and are implemented in the `FBUserManager` class. Additionally, there is a `User` interface providing getters and setters for properties of a user account on the server and corresponding implementation in the `FBUser` class.[4]

The available properties of the FBUser class are:

| Name | Type | Description |
|---|---|---|
| userName | String | Unique name of the user on the Firebird server. Required. Maximum length is 31 byte. |
| password | String | Corresponding password. Getter return value only if the password had been set |
| firstName | String | First name of the user. Optional. |
| middleName | String | Middle name of the user. Optional. |
| lastName | String | Last name of the user. Optional. |
| userId | int | ID of the user on Unix. Optional. |
| groupId | int | ID of the group on Unix. Optional. |

The management class, FBUserManager has the following methods to manipulate the user accounts on the server:

| Method | Description |
|---|---|
| getUsers():Map | Method delivers a map containing usernames as keys and instances of FBUser class as values containing all users that are registered on the server. The instances of FBUser class do not contain passwords, the corresponding property is null. |
| add(User) | Create the user account on the server. |
| update(User) | Update the user account on the server. |
| delete(User) | Delete the user account on the server. |

An example of using the FBUserManager class:

*Example of FBUserManager class usage*

```
var userManager = new FBUserManager();

userManager.setServerName("localhost");
userManager.setPortNumber(3050);
userManager.setUser("SYSDBA");
userManager.setPassword("masterkey");

var user = new FBUser();
user.setUserName("TESTUSER123");
user.setPassword("test123");
user.setFirstName("John");
user.setMiddleName("W.");
user.setLastName("Doe");
```

```
userManager.add(user);
```

## 7.4. Database maintenance

Database maintenance is something that everybody would prefer to avoid, and, contrary to the backup/restore and user management procedures, there is little automation that can be done here. Usually the maintenance tasks are performed on the server by the database administrator, but some routines are needed to perform automated database upgrade or perform periodic checks for database validity.

This chapter describes the methods declared in the `MaintenanceManager` interface and its implementation, the `FBMaintenanceManager` class.

### 7.4.1. Database shutdown and restart

One of the most often used maintenance operations is database shutdown and/or bringing it back online. When a database is shut down, only the user that initiated the shutdown, either SYSDBA or the database owner, can connect to the database and perform other tasks, e.g. metadata modification or database validation and repair.

The database shutdown is performed with the `shutdownDatabase(int, int)` method. The first parameter is the shutdown mode, the second is the maximum allowed time for operation.

There are three shutdown modes:

| Shutdown mode | Description |
|---|---|
| `SHUTDOWN_ATTACH` | The shutdown process is initiated, and it is not possible to obtain a new connection to the database, but the currently open connections are fully functional.<br><br>When, after the maximum allowed time for operation, there are still open connections to the database, the shutdown process is aborted. |
| `SHUTDOWN_TRANSACTIONAL` | The shutdown process is started, and it is not possible to start new transactions or open new connections to the database. The transactions that were running at the time of shutdown initiation are fully functional.<br><br>When, after the maximum allowed time for operation, there are still running transactions, the shutdown process is aborted.<br><br>If no running transactions are found, the currently open connections are allowed to disconnect. |

| Shutdown mode | Description |
|---|---|
| `SHUTDOWN_FORCE` | The shutdown process is started and will be completed before or when the maximum allowed time for operation is reached. New connections and transactions are not prohibited during the wait.<br><br>After the timeout, any running transaction won't be able to commit. |

After database shutdown, the owner of the database or SYSDBA can connect to it and perform maintenance tasks, e.g. migration to the new data model[5], validation of the database, changing the database file configuration.

To bring the database back online use the `bringDatabaseOnline()` method.

### 7.4.2. Shadow configuration



Instead of shadows, consider using replication [https://firebirdsql.org/file/documentation/release_notes/html/en/4_0/rlsnotes40.html#rnfb40-replication], available since Firebird 4.0.

A database shadow is an in-sync copy of the database that is usually stored on a different hard disk, possibly on a remote computer[6], which can be used as a primary database if the main database server crashes. Shadows can be defined using `CREATE SHADOW` SQL command and are characterized by a *mode* parameter:

- in `AUTO` mode, the database continues operating even if the shadow becomes unavailable (disk or file system failure, remote node is not accessible, etc.)
- in `MANUAL` mode all database operations are halted until the problem is fixed. Usually this means that the DBA has to kill the unavailable shadow and define a new one.

The `MaintenanceManager` provides a `killUnavailableShadows()` method to kill the unavailable shadows. This is equivalent to the `gfix -kill` command.

Additionally, if the main database becomes unavailable, the DBA can decide to switch to the shadow database. In this case the shadow must be activated before use. To activate the shadow use the `activateShadowFile()` method. Please note, that in this case the `database` property of the `MaintenanceManager` must point to the shadow file which must be located on the local file system of the server to which the management class is connected.

### 7.4.3. Database validation and repair

The Firebird server does its best to keep the database file in a consistent state. This is achieved by an algorithm called *careful writes*, which guarantees that the server writes data on disk in such a manner that despite events like a server crash, the database file remains in a consistent state. Unfortunately, it is still possible that under certain conditions, e.g. a crash of the file system or hardware failure, the database file becomes corrupted. Firebird server can detect such cases including

**Orphan pages**

These are database pages that were allocated for subsequent write, but due to a crash were not used. Such pages have to be marked as unused, so the server can use them again.

**Corrupted pages**

These are database pages that were corrupted/damaged by operating system or hardware failures.

The `MaintenanceManager` class provides a `validateDatabase()` method to perform a simple health check of the database, releasing orphan pages if needed. This is the equivalent of `gfix -validate` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-db-validation]. It also reports presence of checksum errors. The output of the routine is written to the output stream configured in the `logger` property.

The `validateDatabase(int)` method can be used to customize the validation process:

| Validation mode | Description |
|---|---|
| `VALIDATE_READ_ONLY` | Perform read-only validation. In this case the database file won't be repaired, only the presence of database file errors will be reported.<br><br>Can be used for a periodical health check of the database. See also gfix: Read-only Validation [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-db-validate-noupdate]. |
| `VALIDATE_FULL` | Do a full check on record and pages structures, releasing unassigned record fragments.<br><br>See also gfix: Full Validation [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-db-validate-full]. |
| `VALIDATE_IGNORE_CHECKSUM` | Ignore checksums during repair operations.<br><br>The checksum error means that the database page was overwritten in a random order and the data stored on it are corrupted. When this option is specified, the validation process will succeed even if checksum errors are present.<br><br>See also gfix: Ignore Checksum Errors [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-db-validate-ignore]. |

To repair a corrupted database, use the `markCorruptRecords()` method which marks corrupted records as unavailable. This method is equivalent to `gfix -mend` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-db-mend]. After this operation, a database can be backed up and restored to a different place.



The presence of checksum errors and subsequent use of `markCorruptedRecords()` method will mark all corrupted data as unused space. You have to perform a

careful check after a backup and restore cycle to assess the damage.

### 7.4.4. Limbo transactions

Limbo transactions — or in-limbo transactions — are transactions that were prepared for commit but were not (yet) committed or rolled back; or in other words, an incomplete two-phase commit. This can happen, for example, with a database used by JTA-enabled applications from Java[7]. A limbo transactions affects the normal database operation, since the records modified in that transactions are not available, as Firebird does not know whether the new version will be committed or rolled back and blocks access to them. Also, limbo transactions prevent garbage collection, since the garbage collector does not know whether it can discard the record versions of the limbo transaction.

Jaybird contains functionality to allow the JTA-enabled transaction coordinator to recover the limbo transactions and either commit them or perform a rollback. For cases where this is not possible, `MaintenanceManager` provides the following methods to perform this in interactive mode:

| Method | Description |
|---|---|
| `listLimboTransactions()` | Method lists IDs of all in-limbo transactions to the output stream specified in logger property.<br><br>Equivalent of gfix -list [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-transactions-list]<br><br>The application has to either parse the output to commit or rollback the transactions in some automated fashion, or it should present the output to the user and let him/her make a decision. Alternatively, use one of the following two methods. |
| `limboTransactionsAsList()` | Returns a `List<Long>` of the IDs of all in-limbo transactions |
| `getLimboTransactions()` | Returns an array of `long` with the IDs of all in-limbo transactions |
| `commitTransaction(long)` | Commit the transaction with the specified ID.<br><br>Equivalent of gfix -commit *N* [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-transactions-commit-rollback]. |
| `rollbackTransaction(long)` | Rollback the transaction with the specified ID.<br><br>Equivalent of gfix -rollback *N* [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-transactions-commit-rollback]. |

### 7.4.5. Sweeping the database

The in-limbo transactions are not the only kind of transactions that prevent garbage collection. Another type are transactions are those that were finished by “rollback” and the changes made in such transactions were not automatically undone by the internal savepoint mechanism, e.g. when there were a lot of changes made in the transaction (e.g. 10,000 records and more), or for

transactions started with “NO AUTO UNDO”/`isc_tpb_no_auto_undo`. Such transactions are marked as "rollback" transactions on the Transaction Inventory Page. These transactions prevent advancing the so-called Oldest Interesting Transaction (OIT), the ID of the oldest transaction which created record versions that are relevant to any of the currently running transactions. On each access to a record, Firebird has to check all record versions between the current transaction and the OIT, which leads to performance degradation on large databases. To solve the issue, Firebird periodically starts a database sweeping process, that traverses all database records, removes changes made by rolled back transactions and moves forward the OIT.[8]

The sweep process is controlled by a threshold parameter, a difference between the Next Transaction and OIT, by default it is set to 20,000. While this value is OK for the average database, a DBA can decide to increase or decrease the number to fit the database usage scenario. Alternatively, a DBA can trigger the sweep process manually, regardless of the current difference between Next Transaction and OIT, for example during a maintenance window or hours with low load on the server.

The `MaintenanceManager` provides following methods to help with database sweeping:

| Method | Description |
|---|---|
| `setSweepThreshold(int)` | Set the threshold between Next Transaction and OIT that will trigger the automatic sweep process. Default value is 20,000.<br><br>Equivalent of `gfix -housekeeping N` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-housekeeping-sweep]. |
| `sweepDatabase()` | Perform the sweep regardless of the current difference between Next Transaction and OIT.<br><br>Equivalent of `gfix -sweep` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-housekeeping-manual-sweep]. |

### 7.4.6. Other database properties

There are a few other properties of the database that can be set via `MaintenanceManager`:

| Method | Description |
|---|---|
| `setDatabaseAccessMode(int)` | Change the access mode of the database. Possible values are:<br><br>• `ACCESS_MODE_READ_ONLY` — make database read-only;<br>• `ACCESS_MODE_READ_WRITE` — allow writes into the database.<br><br>Only read-only databases can be placed on read-only media. Read-write databases will need write access even if only accessed with read-only transactions.<br><br>Equivalent of `gfix -mode` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-dbmode]. |

| Method | Description |
| --- | --- |
| `setDatabaseDialect(int)` | Change the database SQL dialect. Allowed values are 1 (deprecated) and 3.<br><br>Equivalent of `gfix -sql_dialect` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-dialect]. |
| `setDefaultCacheBuffer(int)` | Configure page buffers (page cache size) of the database.<br><br>This setting applies to this specific database, overriding the system-wide default.<br><br>Equivalent of `gfix -buffers` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-buffers]. |
| `setForcedWrites(boolean)` | Change the forced writes setting for the database.<br><br>When forced writes are switched off, the database engine does not enforce flushing pending changes to disk, so they might be kept in the OS cache. If the same page is changed again later, the write may happen in memory, which in many cases increases performance. However, in case of OS or hardware crashes, the database might get corrupted as some or all of the changes might not have been written to disk yet.<br><br>Equivalent of `gfix -write` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-sync]. |
| `setPageFill(int)` | Set the page fill factor.<br><br>By default, Firebird leaves some free space on each database page for future record versions.<br><br>It is possible to tell Firebird not to reserve this space. This makes sense for read-only databases, since more data fits the page, which reduces database size and may increase performance. For read-write database, this is likely to decrease performance.<br><br>Allowed values are:<br><br>• `PAGE_FILL_FULL` — do not reserve additional space for future versions;<br>• `PAGE_FILL_RESERVE` — reserve free space for future record versions.<br><br>Equivalent of `gfix -use` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gfix/firebird-gfix.html#gfix-pagespace]. |

## 7.5. Table statistics of a connection (experimental)

The class `org.firebirdsql.management.FBTableStatisticsManager` can be used to retrieve the table statistics of a connection.

Create an instance with `FBTableStatisticsManager#of(java.sql.Connection)` — the connection must unwrap to a `FirebirdConnection` — and retrieve a snapshot of the statistics with `FBTableStatisticsManager#getTableStatistics()`.

The method `getTableStatistics()` returns a `Map<String, TableStatistics>`, where the keys are table names, and the values statistics of tables.

A `TableStatistics` object has the following properties:

- `tableName`
- `readSeqCount`
- `readIdxCount`
- `insertCount`
- `updateCount`
- `deleteCount`
- `backoutCount`
- `purgeCount`
- `expungeCount`



This is an experimental feature. Its API may change in point releases, or it may be removed or replaced entirely in a future major release.

## 7.6. Database statistics

The `StatisticsManager` interface and corresponding implementation in the `FBStatisticsManager` class can be used to obtain statistical information for the database, like page size, values of OIT and Next transactions, database dialect, database page allocation and its distribution.

The following methods provide the functionality equivalent to the *gstat* [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html] command line tool. The output of the commands are written to the output stream specified in the `logger` property. It is the responsibility of the application to correctly parse the text output if needed.

| Method | Description |
|---|---|
| `getDatabaseStatistics()` | Get complete statistics about the database.<br><br>Equivalent of `gstat -all` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-all]. |

| Method | Description |
| --- | --- |
| `getDatabaseStatistics(int)` | Get the statistical information for the specified options.<br><br>Possible values are (bit mask, can be combined):<br><br>• `DATA_TABLE_STATISTICS` (`gstat -data` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-data])<br>• `SYSTEM_TABLE_STATISTICS` (`gstat -system` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-system])<br>• `INDEX_STATISTICS` (`gstat -index` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-index])<br>• `RECORD_VERSION_STATISTICS` `gstat -record` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-record]) |
| `getHeaderPage()` | Get information from the header page (e.g. page size, OIT, OAT and Next transaction values, etc.)<br><br>Equivalent of `gstat -header` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-header]. |
| `getTableStatistics(String[])` | Get statistic information for the listed tables.<br><br>The statistics will be limited to the listed tables. In Jaybird 6 and earlier, at least one table is required.<br><br>***Since:*** *Jaybird 7* Method accepts varargs (`String...`). In Jaybird 7 and later, specifying no tables will list all tables.<br><br>Equivalent of `gstat -table` [https://firebirdsql.org/file/documentation/html/en/firebirddocs/gstat/firebird-gstat.html#gstat-cmdline-table]. |
| `getTableStatistics(`<br>`Collection<String>)` | ***Since:*** *Jaybird 7* Same as previous, but as a collection of table names. |
| `getTableStatistics(`<br>`Collection<String>,`<br>`Collection<String>)` | ***Since:*** *Jaybird 7* Get statistic information for the listed tables in the listed schemas. Accepts a collection of schema names and a collection of table names.<br><br>The statistics will be limited to the listed schemas, and the listed tables. If both collections are empty, all tables in all schemas are reported. On Firebird 5.0 and older, the collections of schemas will be ignored. |

| Method | Description |
| --- | --- |
| `getDatabaseTransactionInfo()` | Get transaction information of a database<br><br>The returned `DatabaseTransactionInfo` object has the following properties:<br><br>• `oldestTransaction`<br>• `oldestActiveTransaction`<br>• `oldestSnapshotTransaction`<br>• `nextTransaction`<br>• `activeTransactionCount` |
| `static getDatabaseTransactionInfo(Connection)` | Same as previous, but using an existing connection (must unwrap to `FirebirdConnection`) |

[1] The output of the service is always transferred over the network regardless whether the logger property is set or not. In addition to providing a possibility to the user to track the service progress, it acts also as a signal of operation completion — in this case the Java code will receive an EOF marker.

[2] Cooperative garbage collection can be switched off in SuperServer architecture with configuration option "`GCPolicy`". It can't be switched off in ClassicServer and SuperClassic architectures.

[3] Firebird 2.5 and older allow you to define validity constraints despite the table(s) containing data that do not satisfy them. Only new records will be validated, and it is responsibility of the database administrator to ensure the validity of existing ones.

[4] The class implementation is a simple bean publishing the properties via getters and setters. You can replace it with any other implementation of the `User` interface.

[5] Until Firebird 2.0 adding a foreign key constraint required exclusive access to the database.

[6] Currently possible only on Unix platforms by using NFS shares.

[7] Another reason for limbo transactions are multi-database transactions which can be initiated via the native Firebird API. However, since Jaybird does not provide methods to initiate them, we do not consider them in this manual.

[8] For more information please read the following article by Ann Harrison, "Firebird for the Database Expert: Episode 4 - OAT, OIT, & Sweep" [https://web.archive.org/web/20231004130607/https://www.ibphoenix.com/resources/documents/design/doc_21]

# Chapter 8. Working with Events

Events are a Firebird feature that provides asynchronous notification to connected applications about events triggered by the database or other applications. Instead of requiring applications to reread the database tables to check for changes, events make it possible to avoid that: triggers in the database can post an event in case of a change. And even more, the event can be so specific that an application would need to reread only a limited set of records, possibly only one.

This chapter describes the event mechanism in Firebird and the common usage scenarios.

## 8.1. Database events

An *event* is a message generated in PSQL code using `POST_EVENT` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-psql-coding.html#fblangref50-psql-postevent] that is delivered to subscribed applications. An event is characterized only by its name, so two different events must use different names. Applications subscribing for events specify the event names of interest, no wildcards are allowed; an applications either provides a callback function that will be invoked asynchronously for the event, or it polls for the posted events periodically.

Events are delivered to the application only on (after) commit of the transaction that generated the event. Firebird does not provide any guarantees about the time of event delivery, it depends on the load of the Firebird engine, application load, network delays between application and the database system. The database engine will continue operating even if no application subscribes to events or when the subscribed application crashed in the meantime.

It's possible that multiple transactions have been committed before the events are delivered to subscribed applications. In that case, the callback function is invoked only once, and only the event name and the count of the events are passed as parameters. The same applies to periodical polling, the application will receive event names and counts of the events since the last poll.

Internally, Firebird can be thought to store the subscription information in a table where columns contain event names, rows correspond to the subscribed applications and the cells contain the count of the particular event for a particular application. When an event is posted in a trigger or stored procedure, Firebird checks the subscription information and increases the event count for the subscribed applications. Another thread checks the table periodically and notifies subscribed applications about all new events relevant to the particular application. Such mechanism allows Firebird to keep the event notification table very small[1] and to reduce the number of messages sent to the application.

It is not possible to pass parameters with the event, e.g. an ID of the modified records. It is also not possible to encode such information in the event names, as wildcards are not supported. For such cases, applications should maintain a change tracking table where the IDs of the modified records are stored and the event mechanism is used to tell the application that new records were added to the table.

## 8.2. Posting events

Events are posted from PSQL code (trigger, stored procedure, execute block, function) using the `POST_EVENT` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-psql-coding.html#fblangref50-psql-postevent] statement. It is possible to create a stored procedure with the sole purpose of posting events (e.g. to notify events from an application):

*Example of posting events from PSQL code*

```
CREATE PROCEDURE sp_post_event(event_name VARCHAR(72))
AS BEGIN
  POST_EVENT :event_name;
END
```

The `EXECUTE BLOCK` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-dml-execblock.html#fblangref50-dml-execblock] statement can be used to execute PSQL statements within DSQL code:

*Using EXECUTE BLOCK to post events*

```
try (Statement stmt = connection.createStatement()) {
  stmt.execute(
      "EXECUTE BLOCK AS BEGIN POST_EVENT 'some_evt'; END");
}
```

## 8.3. Subscribing to events

The design of the classes and interfaces in the `org.firebirdsql.event` package is similar to the Services API support; there is a central manager-class that establishes a database connection and provides service methods to work with the events, a callback interface that applications must implement to use the asynchronous event notification, and an interface representing a database event with two properties, event name and occurrence count.

Applications have to configure the following properties before starting use of the implementation of the `EventManager` interface:

| Name | Type | Description |
|---|---|---|
| `serverName`<br>`host`<br>(***Removed in:*** *Jaybird 6*) | `String` | Name or the IP address of the host to which we subscribe for events. *Required.* |
| `portNumber`<br>`port`<br>(***Removed in:*** *Jaybird 6*) | `int` | Port to which we connect to, 3050 by default. |

| Name | Type | Description |
|---|---|---|
| `databaseName`<br>`database`<br>(**Removed in:** Jaybird 6) | `String` | Path to the database. The path is specified for the remote host but must be absolute. *Required.* |
|  | `serverName`, `portNumber` and `databaseName` are replacements for `host`, `port` and `database`, which were removed in Jaybird 6. | |
| `user` | `String` | Name of the user on behalf of which we connect to the database. *Required.* |
| `password` | `String` | Password corresponding to the specified user. *Required.* |
| `roleName` | `String` | Role name. *Optional.* |
| `expectedDb` | `String` | With Firebird 3.0 and higher, this is used to find the non-default security database to use when authenticating. Value is a database path or alias the user can connect to. *Optional.* |
| `authPlugins` | `String` | Comma-separated list of authentication plugins to use (ignored for Firebird 2.5 or earlier). Use `null` (the default) to use Jaybird defaults. |
| `processId` | `int` | Process id to report to the server. |
| `processName` | `String` | Process name to report to the server. |
| `socketBufferSize` | `int` | Socket buffer size in bytes |
| `soTimeout` | `int` | Socket blocking read timeout in milliseconds (`0` is OS default timeout) |
| `connectTimeout` | `int` | Socket connect timeout in milliseconds (`0` is OS default timeout) |
| `wireCrypt` | `String` | Wire encryption level (`DISABLED`, `ENABLED`, `REQUIRED`, `DEFAULT`). |
| `wireCryptAsEnum` | `WireCrypt` | Alternative to `WireCrypt` to use `WireCrypt` enum. |
| `dbCryptConfig` | `String` | Database encryption config. See Database encryption support for details. |
| `wireCompression` | `boolean` | Enable wire compression (requires Firebird 3.0 or higher). Default is `false`.<br><br>This property only affects the primary connection, not the event channel (secondary connection). |

After configuring these properties, the application has to invoke the `connect()` method to establish a physical connection to the database. At this point the `EventManager` is ready to receive event notifications.

Now the application developer has two choices: use asynchronous event notification or use

methods that will block until an event is delivered or a timeout occurs.

### 8.3.1. Asynchronous event notification

The asynchronous event notification uses a separate daemon thread to wait for the event notifications and to deliver the events to the registered listeners. The listeners are added using the `addEventListener(String, EventListener)` method, where the first parameter contains the name of the event to register on and the second parameter, an instance of `EventListener` interface that will be notified about occurrences of this event. It is allowed to use the same instance of `EventListener` interface to listen on different events. The code below shows an example of using asynchronous event notification.

*Example of registering an event listener for asynchronous event notification*

```
var eventManager = new FBEventManager();
eventManager.setServerName("localhost");
eventManager.setUser("SYSDBA");
eventManager.setPassword("masterkey");
eventManager.setDatabaseName("c:/database/employee.fdb");

eventManager.connect();

eventManager.addEventListener("test_event",
    event ->
        System.out.printf("Event [%s] occured %d time(s)%n",
            event.getEventName(), event.getEventCount()));
```

### 8.3.2. Using blocking methods

Alternatively, an application can use the synchronous methods, one that blocks until the named event is received — the `waitForEvent(String)` method, or one that will block until the named event is received or timeout specified in the second parameter occurs — the `waitForEvent(String, int)` method. The following shows an example of using the blocking methods.

*Example of blocking waiting for event with a specified timeout*

```
var eventManager = new FBEventManager();

eventManager.setServerName("localhost");
eventManager.setUser("SYSDBA");
eventManager.setPassword("masterkey");
eventManager.setDatabaseName("c:/database/employee.fdb");

eventManager.connect();

int eventCount =
    eventManager.waitForEvent("test_event", 10 * 1000);

System.out.println(
```

```
    "Received " + eventCount + " event(s) during 10 sec.");
```

[1] For example, the effective size for 100 applications subscribed for 100 different events is about 40k in memory.

# Reference Manual

# Chapter 9. Connection reference

## 9.1. Authentication plugins

Firebird 3.0 introduced authentication plugins together with a new authentication model. By default, Firebird 3.0 uses the authentication plugin `Srp` (*Secure remote password*). It also includes plugins `Legacy_Auth` that supports the pre-Firebird-3 authentication mechanism, and `Srp256`. Firebird 4.0 introduced the plugins `Srp224`, `Srp384` and `Srp512`.

The `Srp` plugin uses SHA-1, the other Srp-variants use SHA-224, SHA-256, SHA-384 and SHA-512 respectively.[1]



Support for these plugins depends on support of these hash algorithms in the JVM. For example, SHA-224 was not supported in Oracle Java 7 by default and required an additional JCE library.

### 9.1.1. Default authentication plugins

#### Jaybird 6

The default plugins applied by Jaybird 6 are — in order — `Srp256` and `Srp`. This applies only for the pure Java protocol and only when connecting to Firebird 3.0 or higher. The native implementation will use its own default or the value configured through its `firebird.conf`.

When connecting to Firebird 3.0 or higher, the pure Java protocol in Jaybird will not try the `Legacy_Auth` plugin by default as it is an unsafe authentication mechanism. We strongly suggest to use SRP users only, but if you really need to use legacy authentication, you can specify connection property `authPlugins=Legacy_Auth`, see Configure authentication plugins for details.

When connecting to Firebird 3.0 versions earlier than 3.0.4, or if `Srp256` has been removed from the `AuthServer` setting in Firebird, this might result in slightly slower authentication because more roundtrips to the server are needed. After an attempt to use `Srp256` fails, authentication continues with `Srp`.

To avoid this, consider explicitly configuring the authentication plugins to use, see Configure authentication plugins for details.

Firebird 2.5 and earlier — if enabled — will always use legacy authentication.

#### Jaybird 5

The default plugins applied by Jaybird 5 are — in order — `Srp256` and `Srp`.

Firebird 2.5 and earlier will always use legacy authentication.

See section on Jaybird 6 for further details.

### 9.1.2. Configure authentication plugins

The connection property `authPlugins` (alias `auth_plugin_list`) specifies the authentication plugins to try when connecting. The value of this property is a comma-separated list with the plugin names.



The `authPlugins` values can be separated by comma, space, tab, or semicolon. We recommend using comma as the separator. The semicolon should not be used in a JDBC URL as there the semicolon is a separator between connection properties.

Unknown or unsupported plugins will be logged and skipped. When no known plugins are specified, Jaybird will throw an exception with:

- For pure Java

  *Cannot authenticate. No known authentication plugins, requested plugins: [<plugin-names>] [SQLState:28000, ISC error code:337248287]*

- For native

  *Error occurred during login, please check server firebird.log for details [SQLState:08006, ISC error code:335545106]*

The `authPlugins` property only affects connecting to Firebird 3.0 or later. It will be ignored when connecting to Firebird 2.5 or earlier. The setting will also be ignored for native connections when using a fbclient library of version 2.5 or earlier.

Examples:

- JDBC URL to connect using `Srp256` only:

  ```
  jdbc:firebirdsql://localhost/employee?authPlugins=Srp256
  ```

- JDBC URL to connect using `Legacy_Auth` only (this is unsafe!)

  ```
  jdbc:firebirdsql://localhost/employee?authPlugins=Legacy_Auth
  ```

- JDBC URL to try `Legacy_Auth` before `Srp512` (this order is unsafe!)

  ```
  jdbc:firebirdsql://localhost/employee?authPlugins=Legacy_Auth,Srp512
  ```

The property is also supported by the data sources, service managers and event manager.

### 9.1.3. External authentication plugin support (experimental)

If you develop your own Firebird authentication plugin (or use a third-party authentication plugin), it is possible — for pure Java only — to add your own authentication plugin by implementing the interfaces

- `org.firebirdsql.gds.ng.wire.auth.AuthenticationPluginSpi`
- `org.firebirdsql.gds.ng.wire.auth.AuthenticationPlugin`

The SPI implementation needs to be listed in `META-INF/services/org.firebirdsql.gds.ng.wire.auth.AuthenticationPluginSpi` in your JAR.

For a modular JAR, using Jaybird 6 or higher, the plugin will also need to be declared in `module-info.java`:

```
module org.example.auth {
  requires org.firebirdsql.jaybird;

  provides org.firebirdsql.gds.ng.wire.auth.AuthenticationPluginSpi
    with org.example.MyAuthPlugin;
}
```

This support is experimental and comes with a number of caveats:

- We haven't tested this extensively (except for loading Jaybird's own plugins internally)
- The authentication plugin (and provider) interfaces should be considered unstable; they may change with point-releases (though we will try to avoid that)
- For now, it is probably necessary for the JAR containing the authentication plugin to be loaded by the same class loader — or at least, from the same classpath — as Jaybird itself

If you implement a custom authentication plugin and run into problems, contact us on the firebird-java Google Group [https://groups.google.com/g/firebird-java].

If you use a native connection, check the Firebird documentation how to add third-party authentication plugins to fbclient.

## 9.2. Wire encryption support

Firebird 3.0 and higher have support for encrypting the data sent over the network. This *wire encryption* is configured using the connection property `wireCrypt`, with the following (case-insensitive) values:

`DEFAULT`
: default (value used when `wireCrypt` is not specified; you'd normally not specify `DEFAULT` explicitly)

`ENABLED`
: enable, but not require, wire encryption

`REQUIRED`
: require wire encryption (only if Firebird version is 3.0 or higher)

`DISABLED`
: disable wire encryption

The default value acts as `ENABLED` for pure Java connections, for native connections this wil use the fbclient default (either `Enabled` or the configured value of `WireCrypt` from a `firebird.conf` read by the native library).

Connection property `wireCrypt=REQUIRED` will **not** reject unencrypted connections when connecting to Firebird 2.5 or lower. This behavior matches the Firebird 3.0 client library behavior. The value will also be ignored when using native connections with a Firebird 2.5 client library.

Using `wireCrypt=DISABLED` when Firebird 3.0 or higher uses setting `WireCrypt = Required` (or vice versa) will yield error *"Incompatible wire encryption levels requested on client and server"* (error: *isc_wirecrypt_incompatible / 335545064*).

The same error is raised when connecting to Firebird 3.0 and higher with a legacy authentication user with connection property `wireCrypt=REQUIRED`, as the legacy authentication does not provide an encryption key to establish an encrypted connection.

Alternative wire encryption plugins are currently not supported, although we made some preparations to support this. If you want to develop such a plugin, contact us on the firebird-java Google Group [https://groups.google.com/g/firebird-java], so we can work out the details of adding plugin support.



The implementation comes with a number of caveats:

- we cannot guarantee that the session key cannot be obtained by someone with access to your application or the machine hosting your application (although that in itself would already imply a severe security breach)
- the ARC4 encryption — the default provided by Firebird — is considered to be a weak (maybe even broken) cipher these days
- the encryption cipher uses ARCFOUR with a 160 bits key, this means that the unlimited Cryptographic Jurisdiction Policy needs to be used (or at minimum a custom policy that allows ARCFOUR with 160 bits keys). See also FAQ entry Encryption key did not meet algorithm requirements of Symmetric/Arc4 (337248282) [https://firebirdsql.org/docs/drivers/java/faq.html#encryption-key-did-not-meet-algorithm-requirements-of-symmetricarc4-337248282]

On Java 11 or higher, Jaybird also supports the `ChaCha` wire encryption.
***Since:*** *Jaybird 6* Since Jaybird 6, the `ChaCha64` wire encryption is supported **if** the `chacha64-plugin` artifact and its Bouncy Castle dependency are on the classpath.

## 9.3. Wire compression

Jaybird supports zlib wire compression in the pure Java wire protocol. Compression can be enabled using boolean connection property `wireCompression`.

The connection property only has effect for the pure Java wire protocol connections on Firebird 3.0

and higher, if the server has the zlib library. Native connections will follow the `WireCompression` configuration in the `firebird.conf` read by the client library, if the zlib library is on the search path.

Compression is currently disabled by default. This may change in future versions of Jaybird to be enabled by default.

The `wireCompression` property is also available on data sources and the management classes in `org.firebirdsql.management`.

## 9.4. Database encryption support

Jaybird 3.0.4 added support for Firebird 3.0 database encryption callbacks in the pure Java implementation of the version 13 protocol.

The current implementation is simple and only supports replying with a static value from a connection property. Be aware that a static value response for database encryption is not very secure as it can easily lead to replay attacks or unintended key exposure.

Future versions of Jaybird may introduce plugin support for database encryption plugins that require a more complex callback.

The static response value of the encryption callback can be set through the `dbCryptConfig` connection property. Data sources and `ServiceManager` implementations have an equivalent property with the same name. This property can be set as follows:

- Absent or empty value — empty response to callback (depending on the database encryption plugin this may just work or yield an error later).
- Strings prefixed with `base64:` — rest of the string is decoded as *base64* to bytes. The `=` padding characters are optional, but when present they must be valid (that is: if you use padding, you must use the right number of padding characters for the length).
- Strings prefixed with `base64url:` — rest of the string is decoded as *base64url* to bytes (using "Base 64 URL and Filename safe" alphabet). As with `base64:`, the `=` padding characters are optional
- Plain string value — string is encoded to bytes using UTF-8, and these bytes are used as the response.

Because of the limitation of connection URL parsing, avoid plain string values with `&` or `;`. Likewise, avoid `:` so that we can support other prefixes similar to `base64:` and `base64url:` in the future. If you need these characters, consider using a base64 encoded value instead.

Jaybird expects keys and values in the JDBC URL to be URL encoded. We recommend using *base64url*, and **not** *base64*, as *base64* encoded values can contain `+`, which must be escaped as `%2B` in the JDBC URL, otherwise it is decoded to a space and decoding will fail.

For service operations, as implemented in the `org.firebirdsql.management` package, Firebird requires the `KeyHolderPlugin` configuration to be globally defined in `firebird.conf`. Database-specific configuration in `databases.conf` will be ignored for service operations. Be aware that some service operations on encrypted databases are not supported by Firebird 3.0 (e.g. `gstat` equivalents

other than `gstat -h` or `gstat -e`).



Other warnings and limitations

- Database encryption callback support is only available in the pure Java implementation. Support for native and embedded connections may be added in a future version.
- The database encryption callback does not require an encrypted connection, so the key can be exchanged unencrypted if wire protocol encryption has been disabled client-side or server-side, or if legacy authentication is used. Consider setting connection property `wireCrypt=REQUIRED` to force encryption (caveat: see the next point).
- Firebird may ask for the database encryption key before the connection has been encrypted (for example if the encrypted database itself is used as the security database).
- We cannot guarantee that the `dbCryptConfig` value cannot be obtained by someone with access to your application or the machine hosting your application (although that in itself would already imply a severe security breach).

## 9.5. Default holdable result sets

This connection property enables a connection to create holdable result sets by default. This property can be used as a workaround for applications that expect a result set to remain open after commit, or have expectations regarding result sets in auto-commit mode that do not conform to the JDBC specification.

Specifically, such applications open a result set and, while traversing it, execute other statements using the same connection. According to the JDBC specification the result set has to be closed if another statement is executed using the same connection in auto-commit mode. With the default result set holdability, close on commit, doing this yields a `SQLException` with message *"The result set is closed"*.

The property is accessible as:

- connection property `defaultResultSetHoldable` with no value, empty value or `true` (aliases: `defaultHoldable` and `result_set_holdable`);
- `FirebirdConnectionProperties` interface methods `isDefaultResultSetHoldable()` and `setDefaultResultSetHoldable(boolean)`



The price for using this feature is that each holdable result set will be fully cached in memory. The memory occupied by this result set will be released when the result is closed, or the statement that produced the result set is closed or re-executed.

## 9.6. Firebird auto commit mode (experimental)



**Do not use this unless you really know what you're doing.**

This functionality is experimental, and will remain so unless Firebird changes how its auto-commit mode works.

Incorrect use of this functionality can result in excessive growth of the database due to increases in back-version chains of records, which can also cause performance degradation. Additionally, when used with an isolation level other than READ COMMITTED, the connection will only see changes committed at the time the initial transaction was started; the auto-commit barrier will not make new committed changes visible to the current transaction.

This option is enabled by specifying the connection property `useFirebirdAutocommit=true`.

With this option, a connection in auto-commit mode will configure the transaction to use `isc_tpb_autocommit`. This means that Firebird server will internally commit the transaction after each statement completion using "*commit retain*". Jaybird itself will not commit until connection close (or disabling auto-commit). The exception is if the statement was of type `isc_info_sql_stmt_ddl`, in that case Jaybird will commit on statement success and rollback on statement failure (just like it does for all statements in normal auto-commit mode). The reason is that Firebird for some DDL commands only executes at a real commit boundary and relying on the Firebird auto-commit is insufficient.

On statement completion (as specified in JDBC), result sets will still close unless they are holdable over commit. The result set is only closed client-side, which means that the cursor remains open server-side to prevent roundtrips. This may lead to additional resource usage server-side unless explicitly closed in the code. Note that any open blobs will be closed client- and server-side (until this is improved with jaybird#442 [https://github.com/FirebirdSQL/jaybird/issues/442]).

A connection can be interrogated using `FirebirdConnection.isUseFirebirdAutocommit()` if it uses `isc_tpb_autocommit`.

If you manually add `isc_tpb_autocommit` to the transaction parameter buffer, and you enable this option, the `isc_tpb_autocommit` will be removed from the TPB on `setAutoCommit(false)`.

Artificial testing with repeated inserts (using a prepared statement) against a Firebird server on localhost shows that this leads to a reduction of execution time of +/- 7%.

Support for this option is experimental, and should only be enabled if you

1. know what you're doing, and
2. really need this feature.

Internally `isc_tpb_autocommit` uses `commit_retaining`, which means that using this feature may increase the transaction gap with associated sweep and garbage collection impact.

# 9.7. Process information

Introduced in Firebird 2.1, the `MON$ATTACHMENTS` monitoring table contains information about database attachments, including information about the client process.

The columns `MON$REMOTE_PID` and `MON$REMOTE_PROCESS` report the process id and process name of the connected process.

Jaybird 5 and earlier do not provide this information by default, except for the process ID of native connections, but they can be specified with system or connection properties (see below). This has two main reasons: until recently Java did not have a portable way of retrieving the process id, and in most cases the process name is just "java" (or similar), which is not very useful.

***Since:*** *Jaybird 6* Starting with Jaybird 6, pure Java connections will by default report the actual process ID, just like native connections. When a `SecurityManager` is installed, the entire call-chain needs to have the `RuntimePermission("manageProcess")` to obtain the process ID in pure Java connections. If this permission check fails, Jaybird will silently ignore it and not set the `isc_dpb_process_id` based on the actual process ID.

***Since:*** *Jaybird 6* The process name is returned from `Connection.getClientInfo("ApplicationName")`, if the client info property `ApplicationName` has not been set explicitly with `Connection.setClientInfo`, and/or if there is no property `ApplicationName` in the `USER_SESSION` context.



Do not use the process name and process id information for security decisions. Treat it as informational only, as clients can report fake information.

Native connections will always report the actual process ID, if a fbclient of Firebird 2.1 or higher is used. This cannot be overridden using the options below.

Since Firebird 3.0, the `MON$ATTACHMENTS` table also includes the column `MON$CLIENT_VERSION`. Jaybird will report its full version (e.g. `Jaybird jaybird-6.0.7`).

It is possible to specify the process name and process id in two ways:

## 9.7.1. System properties for process information

It is possible to specify the process information through Java system properties:

**`org.firebirdsql.jdbc.pid`**
: Process id

  This property is ignored on native connections, which will always report the actual process id.

**`org.firebirdsql.jdbc.processName`**
: Process name

In Jaybird 5 and earlier, this is the preferred method because you only need to specify it once. For Jaybird 6 and later, in case of the process id, the preferred method is to not set any option and use the actual process ID.

These properties are read every time a connection is created, so they can be changed during the lifetime of a program. Changes to these system properties will only affect connections created after the change.

### 9.7.2. Connection properties for process information

It is also possible to specify the process information through connection properties:

`processId`
: Process id (alias: `process_id`)

  This property is ignored on native connections, which will always report the actual process id.

`processName`
: Process name (alias: `process_name`, `ApplicationName`, `applicationName`)

The connection properties take precedence over the system properties.

These properties are also exposed on data sources.

## 9.8. Data type bind support

The `SET BIND` statement and `isc_dpb_set_bind` DPB item allow you to define data type conversion rules for compatibility or ease of processing data.

In Jaybird `isc_dpb_set_bind` is exposed as connection property `dataTypeBind` (alias `set_bind`). The value of this connection property is a semicolon-separated list of data type bind definitions.

A data type bind definition is of the form `<from-type> TO <to-type>`. A definition is the same as the second half of a `SET BIND` statement after the `OF`. See the Firebird 5.0 documentation of `SET BIND` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-management.html#fblangref50-management-setbind] for more information. Invalid values or impossible mappings will result in an error on connect.

When using the `dataTypeBind` connection property in a JDBC URL, the semicolons of the list need to be encoded as `%3B`, as unescaped semicolons in the JDBC URL are an alternative to `&` as the separator between properties.

For example:

*JDBC url with dataTypeBind and encoded semicolon*

```
String jdbcUrl = "jdbc:firebirdsql://localhost/database?charSet=utf-8"
    + "&dataTypeBind=decfloat to varchar%3Btimestamp with time zone to legacy"
```

When the property is set through a `Properties` object or a `DataSource` configuration, you should not encode the semicolon as this will result in errors.

For example:

*Properties object with dataTypeBind*

```
var props = new Properties();
props.setProperty("dataTypeBind",
    "decfloat to varchar;timestamp with time zone to legacy"
```

Values set through this connection property will be the session default configuration, which means that they are retained (or reverted to) when executing `ALTER SESSION RESET` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-management-session-reset.html#fblangref50-management-session-reset-alter].

## 9.9. Enabling unsupported protocol versions

***Since:*** *Jaybird 6*

Since Jaybird 6, the pure Java protocol implementation will no longer attempt to connect with protocol versions of Firebird versions which are not supported. Since the minimum supported Firebird version in Jaybird 6 is Firebird 3.0, this means that protocol versions 10, 11 and 12 are no longer tried by default. Protocol versions higher than 19 (Firebird 5.0.3) will also not be attempted (assuming a protocol implementation with that version number exists on the classpath).

As a workaround, the connection property `enableProtocol` can enable unsupported protocols, assuming a suitable protocol implementation is available on the classpath.

This connection property can have the following values:

- A comma-separated list of additional protocol versions to try (e.g. `"11,12"`). The listed versions are tried in addition to the supported protocol versions. Non-integer values or unknown protocol versions are silently ignored.

  It is possible to use the “masked” protocol version (e.g. `"32780"` for protocol version 12). However, we recommend using the unmasked version (e.g. `"12"`).

- `"*"` — enable all available protocol versions
- `null` or empty string (`""`) — default behaviour, only use supported protocols

A different default value of `enableProtocol` can be set using the system property `org.firebirdsql.jdbc.defaultEnableProtocol`. This system property is checked each time a connection configuration is created, so it can be changed at runtime. If you use a Jaybird `DataSource` implementation, it uses the value at the time the `DataSource` is created; if you use `DriverManager` — this can include third-party data sources, it uses the value at the time the connection is created.



Given these protocol versions and their Firebird version are not supported, there is no guarantee that the driver will function correctly when an unsupported protocol is enabled this way. Especially things like database metadata could use features that are not supported by older Firebird versions. We recommend upgrading your Firebird version, or downgrading to a Jaybird version which still supports your Firebird version.

> For maximum compatibility, it is recommended to either use "*", or to make sure to include the maximum protocol version of your Firebird version.
>
> Unsupported protocol versions may be removed in future releases of Jaybird.

*Table 2. Firebird versions and protocol versions*

| Firebird version | Maximum protocol |
|---|---|
| 1.0 — 2.0 | 10 |
| 2.1 | 11 |
| 2.5 | 12 |
| 3.0 | 15[2][3] |
| 4.0 | 16[4] |
| 5.0 | 19[5] |

## 9.10. Opt-in feature for package information in DatabaseMetaData

***Since:*** *Jaybird 6*

Firebird supports packages, which are a way to bundle stored procedures and functions. The JDBC API does not provide a “standard” way of accessing information about packages, or the routines defined in packages. Instead of adding additional Jaybird-specific metadata methods, we’ve added an “opt-in” feature that provides access through normal metadata methods, using the “catalog” to report packages.

This feature can be enabled by setting the connection property `useCatalogAsPackage` to `true`. When this connection property is enabled, the `DatabaseMetaData` of that connection will have the following changes in behaviour:

- `getCatalogs()` lists packages, with package names in `TABLE_CAT`.
- `getFunctions`, `getFunctionColumns`, `getProcedures`, and `getProcedureColumns` include information on procedures or functions in packages
  - Columns `FUNCTION_CAT`/`PROCEDURE_CAT` will report:
    - For packaged procedures and functions — the package name
    - For normal (non-package) procedures and functions — an empty string instead of `null` (because of the following rule)
  - If parameter `catalog` is `""` (empty string), only normal stored procedures or stored functions are reported.
  - If parameter `catalog` is `null`, both packaged and normal stored procedures or stored functions are reported.
  - For other values of parameter `catalog`, these metadata methods will only return procedures, functions, or their columns of the specified package (exact match, case-sensitive; not a `LIKE`

pattern)

- For normal (non-package) procedures and functions, the `SPECIFIC_NAME` column will be the unquoted function or procedure name (same as when `useCatalogAsPackage` is not enabled), and for packaged procedures and functions, it will be `quoted-package-name + '.' + quoted-routine-name` (e.g. `"SOME_PACKAGE"."SOME_FUNCTION"`)

- `getCatalogSeparator()` returns `"."` (string with period).
- `getCatalogTerm()` returns `"PACKAGE"`.
- `isCatalogAtStart()` returns `true`.
- `getMaxCatalogNameLength()` returns 31 or 63 depending on the max identifier length of the database.
- `supportsCatalogsInDataManipulation()` returns `true` (i.e. access selectable stored procedures and functions from packages).
- `supportsCatalogsInProcedureCalls()` returns `true`.
- The other `supportsCatalogsIntype()` methods continue to return `false`.
- Other metadata methods with a `catalog` parameter continue to ignore it, just like they do when `useCatalogAsPackage` is disabled.

The `useCatalogAsPackage` connection property does not result in any other behaviour.

Keep in mind that this is non-standard behaviour, and standard JDBC tools or libraries may not work correctly when this property is enabled. This feature may be discontinued and removed in the future if Jaybird needs to implement "real" catalogs (e.g. because Firebird started supporting catalogs).

See also jdp-2023-08: Use Catalog as Package [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2023-09-use-catalog-as-package.adoc]

# 9.11. Client info properties

Client info properties allow you to set properties on a connection for informational purposes, or as the JDBC API documentation says "*used for accounting, diagnostics and debugging purposes only*"[6].

## 9.11.1. Support in Jaybird 5 and earlier

Support for client info properties was introduced in Jaybird 2.2, storing properties in the `USER_SESSION` context of `RDB$GET/SET_CONTEXT` [https://firebirdsql.org/file/documentation/chunk/en/refdocs/fblangref50/fblangref50-functions.html#fblangref50-functions-workcontext]. Support in Jaybird 5 and earlier is quite limited, allowing you to:

- Set properties individually or collectively using a `Properties` object (`Connection#setClientInfo(String,String)`, `Connection#setClientInfo(Properties)`)
- Clear properties individually (setting them to `null`) (`Connection#setClientInfo(String,String)`)
- Get individual properties (`Connection#getClientInfo(String)`)

Properties are always stored in USER_SESSION. Properties are retrieved from USER_TRANSACTION, falling back to USER_SESSION if the property was not present in USER_TRANSACTION.

Absent is support to:

- Get properties collectively (Connection#getClientInfo() always returns an empty Properties object)
- Collectively clear properties by not including them in the Properties object passed to Connection#setClientInfo(Properties)
- retrieving supported client property names from DatabaseMetaData#getClientInfoProperties() (always returns an empty result set)

### 9.11.2. Support in Jaybird 6 and later

***Since:*** *Jaybird 6*

In Jaybird 6, the client info properties support was rewritten, improving and extending support of this feature.

The name of a client info property is classified in two types:

1. without context; *get*/*set* in USER_SESSION context of RDB$GET/SET_CONTEXT
2. with context, when the name ends in @USER_SESSION, @USER_TRANSACTION or @SYSTEM; *get*/*set* *without* that suffix in the specified context. Properties with multiple contexts (e.g. property@SYSTEM@USER_SESSION) are not allowed to prevent ambiguity. Everything else is a property without context.

A property named <name>@USER_SESSION is handled identical to <name>. The "without context" name is preferred and recommended for USER_SESSION properties, and Connection.getClientInfo() will use the "without context" name as the key for USER_SESSION properties.

A property named <name>@USER_TRANSACTION is *set* and *get* as <name> in context USER_TRANSACTION. When a connection is in auto-commit mode, attempts to *set* or *get* USER_TRANSACTION properties are ignored (nothing is stored, and null is returned without accessing the database).

A property named <name>@SYSTEM is retrieved as <name> in the context SYSTEM. Properties in the SYSTEM context are read-only, so they cannot be *set*. For Connection.setClientInfo(String, String), attempts to *set* a SYSTEM property result in a SQLClientInfoException, for Connection.setClientInfo(Properties), SYSTEM properties are silently ignored.

Attempts to retrieve non-existent properties, even from SYSTEM will return null and will not result in an exception.

A connection registers properties known to that specific connection. By default, only the JDBC-specified properties ApplicationName, ClientUser and ClientHostName (see also JDBC-specified properties below) are known. A successful *get* or *set* of a property will register that property as a known property for the current connection only.

The method Connection.getClientInfo() will retrieve only known properties (the default properties

and those registered for the current connection). It will not attempt to query or identify other variables registered in any context.

The method `Connection.setClientInfo(Properties)` clears (sets to `NULL`) known properties in `USER_SESSION` and — if not in auto-commit — `USER_TRANSACTION` if they are not included in the `Properties` object.

`DatabaseMetaData.getClientInfoProperties()` reports the JDBC-specified properties only; it will not report the additional properties registered for the current connection.

See also jdp-2023-11: New design client info properties [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2023-11-new-design-client-info-properties.adoc]

#### JDBC-specified properties

The JDBC-specified properties are:

> `ApplicationName`
>
> The name of the application currently utilizing the connection
>
> `ClientUser`
>
> The name of the user that the application using the connection is performing work for. This may not be the same as the user name that was used in establishing the connection.
>
> `ClientHostname`
>
> The hostname of the computer the application using the connection is running on.

All JDBC-specified properties are *get* and *set* in `USER_SESSION`. On *get* of `ApplicationName`, if this property is not currently set in `USER_SESSION`, it falls back to *get* the value from `CLIENT_PROCESS@SYSTEM`. The `CLIENT_PROCESS@SYSTEM` value can be specified using the connection property `processName` or system property `org.firebirdsql.jdbc.processName`. The `ClientUser` and `ClientHostname` properties are considered application-specific and have no default value or fallback.

### 9.11.3. Connection pools and client info properties

***Since:*** *Jaybird 6*

If you use a connection pool populated from `org.firebirdsql.ds.FBConnectionPoolDataSource` or `org.firebirdsql.ds.FBXADataSource`, the known properties are reset each time the connection is handed out by the pool. This assumes the pool uses `getConnection()` from `PooledConnection` /`XAConnection` each time it hands out the connection. This reset only removes the property name from the list of known properties of the connection, it does not clear the property values from `USER_SESSION`. If properties contain *sensitive* values, you will need to explicitly clear them.

This reset is not applied when using a connection pool which is populated by different means (e.g.

from `DriverManager` or from a normal `javax.sql.DataSource` like `org.firebirdsql.ds.FBSimpleDataSource`). If your connection pool implementation supports custom code to reset a connection, and you need known properties to reset when a connection is reused, make it call `FirebirdConnection.resetKnownClientInfoProperties()`, or call it yourself:

```
if (connection.isWrapperFor(FirebirdConnection.class)) {
    connection.unwrap(FirebirdConnection.class)
            .resetKnownClientInfoProperties();
}
```

In the future, Jaybird may be changed to also perform this reset in `Connection.beginRequest()` and/or `Connection.endRequest()`.

## 9.12. Create database if it does not exist

***Since:*** *Jaybird 6*

The Boolean connection property `createDatabaseIfNotExist` instructs Jaybird to attempt to create the database if it does not exist yet. This property can be used in the JDBC URL, in the `Properties` object passed to `DriverManager`, and can be set on data sources.

Jaybird also allows you to create databases using `org.firebirdsql.management.FBManager`, but this is not always accessible, for example in tools or libraries that only expose or use the JDBC API. This property provides an alternative to create databases in those cases.

Jaybird also allows you to set or override connection properties specifically for creating the database by suffixing the property name with `@create` (case-sensitive). This can be used to set configuration properties that are only relevant for creating the database, or — for example — to use a different user or role for creating the database than used for normal connections.

As an example:

```
String jdbcUrl = "jdbc:firebird://localhost/exampledb" +
    "?createDatabaseIfNotExist=true&" +
    "user@create=sysdba&password@create=masterkey";
try (Connection connection = DriverManager.getConnection(
    jdbcUrl, "normaluser", "password")) {
  // ...
}
```

If the database already exists, the connection will be made with user `normaluser`, but if the database does not exist, the database will be created with user `sysdba`.



The errors Firebird returns do not make it possible to determine definitively if the database does not exist, or if there is another reason that the connection failed. Jaybird applies some simple rules to exclude some classes of errors, and not attempt to create a database in those cases.

> If you find cases where you think Jaybird should not (or on the contrary should) attempt to create a database, please report them on https://github.com/FirebirdSQL/jaybird/issues.

For more information, see also jdp-2024-02: Create database through JDBC URL [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2024-02-create-database-through-jdbc-url.adoc].

## 9.13. Reporting of `SQLWarnings`

The JDBC Specification requires that a JDBC driver reports a `SQLWarning` when it deviates from certain JDBC-specified behaviour (for example, changing a requested result set type), or when the server reports a warning.

These warnings are reported on the `Connection`, `Statement`, or `ResultSet`[7], and can be retrieved with `getWarnings()`.

***Since:*** *Jaybird 6* The connection property `reportSQLWarnings` can be used to disable the reporting of `SQLWarnings`.

This connection property supports the following values (case-insensitive):

`ALL` (default) report all `SQLWarnings`

`NONE` Do not report any `SQLWarnings`

Invalid values on the connection property will be rejected.

The default value can be overridden with the system property `org.firebirdsql.jdbc.defaultReportSQLWarnings`. It supports the same values as connection property `reportSQLWarnings`. Invalid values of the system property are ignored.

## 9.14. Custom socket factory

***Since:*** *Jaybird 6*

A custom socket factory can be specified to customize the creation of the `java.net.Socket` instance of a pure Java database or service connection.

The connection property `socketFactory` accepts the class name of an implementation of `javax.net.SocketFactory`. This socket factory is created anew for each connection. If `socketFactory` is not specified, Jaybird will use `SocketFactory.getDefault()` as its factory.

The `SocketFactory` implementation must adhere to the following rules:

- The class must have a public constructor accepting a `java.util.Properties` object, or a public no-arg constructor.
- The implementation of `SocketFactory#createSocket()` must return an unconnected socket; the other `createSocket` methods are not called by Jaybird.

If you don't want to implement the other `createSocket` methods, we recommend throwing `java.lang.UnsupportedOperationException` with a clear message from those methods.

It is possible to pass custom connection properties to the socket factory if it has a public single-arg constructor accepting a `Properties` object. Jaybird will instantiate the socket factory with a `Properties` object containing *only* the connection properties with the suffix `@socketFactory` and non-`null` values; non-string values are converted to string. In the future, we may also — selectively — pass other connection properties, but for now we only expose those properties that are explicitly set for the socket factory.

For example, say we have some custom socket factory called `org.example.CustomProxySocketFactory` with a `CustomProxySocketFactory(Properties)` constructor:

```
var props = new Properties()
props.setProperty("user", "sysdba");
props.setProperty("password", "masterkey");
props.setProperty("socketFactory", "org.example.CustomProxySocketFactory");
props.setProperty("proxyHost@socketFactory", "localhost");
props.setProperty("proxyPort@socketFactory", "1234");
props.setProperty("proxyUser@socketFactory", "proxy-user");
props.setProperty("proxyPassword@socketFactory", "proxy-password");

try (var connection = DriverManager.getConnection(
        "jdbc:firebird://remoteserver.example.org/db", props)) {
    // use connection
}
```

This will create the specified socket factory, passing a `Properties` object containing **only** the four custom properties ending in `@socketFactory`. The other properties — here `user`, `password` and `socketFactory` — are **not** passed to the socket factory.

See also jdp-2024-09: Custom socket factory for pure Java connections [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2024-09-custom-socket-factory-for-pure-java-connections.adoc]

## 9.15. Schema support

***Since:*** *Jaybird 7*
***Since:*** *Firebird 6.0*

Firebird 6.0 introduced schema support, and schemas are supported in Jaybird 7 and higher.

This sections describes aspects of schema support that we think are relevant, or deviate from JDBC defined behaviour.

### 9.15.1. Search path

In Firebird 6.0 and higher, the search path is a list of schemas that Firebird searches for objects that are not fully-qualified — for example a select statement referencing a table without a schema name.

Jaybird has a connection property `searchPath` (aliases `search_path`, `isc_dpb_search_path`) to configure the initial search path of a connection. If the `SYSTEM` schema is not included, Firebird will automatically add it to the end of the search path. The configured value is also what is reverted to when executing `ALTER SESSION RESET`.

The value of this connection property is a comma-separated list of schema names. For a case-sensitive or otherwise non-regular schema name, the identifier **must** be quoted.

The `FirebirdConnection` interface defines the methods `setSearchPath(String)` and `setSearchPathList(List<String>)` to change the search path of the current connection. It also defines methods `String getSearchPath()` and `List<String> getSearchPathList()` to obtain the search path of the connection.

### 9.15.2. Current schema and `Connection.setSchema/getSchema`

The first *valid* (existing) schema on the search path is the *current schema*. The current schema is used to create objects if they are not fully-qualified.

JDBC defines the method `setSchema` to change the current schema, and `getSchema` to retrieve the current schema.

The implementation in Jaybird will do one of the following:

1. Prepend the schema to the existing search path.

   This is done if `setSchema` was not called before, or if Jaybird detected that the search path was changed in some other fashion.

2. Replace the first value of the current search path with a new value.

   This is done if the current search path is unchanged since the previous call to `setSchema`, otherwise it's prepended.

The `setSchema` method will not check if the specified schema exists, nor throw an exception. In other words, the current schema — and the value reported by `getSchema` — might not change, or change to a different value in case 2 (replacing the first schema), if the specified schema does not exist.

If a schema on the search path that previously did not exist is created, the value reported by `getSchema` may change even if the search path was not changed.

### 9.15.3. Non-standard statement behaviour

JDBC requires that a statement object, once created, does not change the current schema it uses. That is, changes made with `Connection.setSchema` should not apply to statements created before that call. Jaybird cannot fully meet this requirement as the search path is a connection-level property, and Firebird will use the search path as it is at prepare time.

- For `Statement`, Jaybird will prepare the statement text on execute, so the search path at that time is used and resolution can change between executes on the same `Statement` object.
- For `PreparedStatement` (not `CallableStatement`), Jaybird will prepare the statement text at

`Connection.prepareStatement` time, so it can guarantee the required behaviour.

- For `CallableStatement`, Jaybird will attempt to identify the procedure at `Connection.prepareCall` time, and rewrite the statement text to use a fully-qualified reference. If it can identify the procedure, Jaybird will meet the required behaviour.

  However, actual server-side statement preparation is delayed and will be repeated during the lifetime of the `CallableStatement`. So, if Jaybird was not able to identify the procedure during `prepareCall`, the actual schema and procedure resolution can change between executes.



If you need stable resolution where Jaybird doesn't guarantee it, you can fully qualify the objects (e.g. tables) in your statements:

```
select
  MY_TABLE.*,
  MY_SCHEMA.MY_FUNCTION(MY_COLUMN),
  MY_SCHEMA.MY_PACKAGE.MY_FUNCTION(MY_COLUMN)
from MY_SCHEMA.MY_TABLE
```

```
{call MY_SCHEMA.MY_PROCEDURE(?, ?)}
-- or
{call MY_SCHEMA.MY_PACKAGE.MY_PROCEDURE(?, ?)}
```

[1] Internally `SrpNNN` continues to use SHA-1, only the client-proof applies the SHA-NNN hash. See also firebird#6051 [https://github.com/FirebirdSQL/firebird/issues/6051]).

[2] since Firebird 3.0.2, version 13 for 3.0.0 and 3.0.1

[3] Jaybird has no protocol 14 implementation

[4] Jaybird has no protocol 17 implementation

[5] since Firebird 5.0.3, version 18 for 5.0.0 - 5.0.2

[6] from `java.sql.Connection#setClientInfo(String,String)` [https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/Connection.html#setClientInfo(java.lang.String,java.lang.String)]

[7] In the current implementation, Jaybird never has warnings on `ResultSet`

# Chapter 10. Statement reference

## 10.1. Async fetch

***Since:*** *Jaybird 6*

For pure Java connections, non-holdable forward-only result sets perform asynchronous fetches by default. Asynchronous fetches are implemented for protocol implementation version 11 and higher (i.e. Firebird 2.1 or higher), but are formally only supported for protocol version 13 and higher (i.e. Firebird 3.0 or higher).

In normal usage of a result set, the first fetch will be a normal synchronous fetch. If certain conditions are met, subsequent fetches will be performed asynchronously.

The conditions for using asynchronous fetch are considered an implementation detail and may change in point releases. In Jaybird 6.0.0, the conditions to enable asynchronous fetching are:

1. The result set has type `FORWARD_ONLY`.
2. The result set does not have a cursor name set.
3. Any synchronous fetch on the statement cursor retrieved at least 15 rows. Be aware that the server may return fewer rows than the configured fetch size (default is 400), depending on Firebird version, row size and other factors.

When the asynchronous fetch is triggered is also an implementation detail and may change in point releases. In Jaybird 6.0.0, the conditions to perform an asynchronous fetch are:

- Size of the row buffer is equal to the “low-water mark”.

  This “low-water mark” is calculated as ~33% of the maximum number of rows returned by previous fetches, with a minimum of 10 rows.

  As a consequence of this condition, reducing the fetch size equal to or less than the “low-water mark” may cause asynchronous fetches to stop being triggered, as the buffer size may remain smaller than this “low-water mark” (this effect may be reduced if there is other activity on the connection after the asynchronous fetch was performed). In that case, a synchronous fetch is triggered when the buffer is empty.

- Fetch size at time of async fetch is larger than 1. This may be the configured fetch size, or the fetch size derived taking into account the configured maximum row count.

Native connections do not support this type of asynchronous fetching, but `fbclient` itself provides a different form of internal asynchronous fetching for native connections.

Asynchronous fetching — for pure Java — can be disabled with the Boolean connection property `asyncFetch`, by setting it to `false`. The system property `org.firebirdsql.jdbc.defaultAsyncFetch` can be used to disable it globally; this system property is checked dynamically when the connection configuration is created. These properties are primarily intended for troubleshooting or workarounds if it turns out this feature has problems; they may be removed in a future Jaybird

version once this feature has proven itself.

## 10.2. Generated keys retrieval

Jaybird provides support for the `getGeneratedKeys()` JDBC feature for `Statement` and `PreparedStatement`. This feature can be used to retrieve the generated ids (and other columns) from DML statements.

This feature is available for `Connection.prepareStatement`, and `Statement.execute`, `Statement.executeUpdate` and `Statement.executeLargeUpdate`.

There are four distinct use-cases:

1. Methods accepting an `int` parameter with values of `Statement.NO_GENERATED_KEYS` and `Statement.RETURN_GENERATED_KEYS`, see Basic generated keys retrieval,
2. Methods accepting an `int[]` parameter with column indexes, see Generated keys by column index.
3. Methods accepting a `String[]` parameter with column names, see Generated keys by column name.
4. Providing a query already containing a `RETURNING` clause to any of these methods.

   In this case all the previous cases are ignored and the query is executed as is. It is possible to retrieve the result set using `getGeneratedKeys()`.

This functionality is available for `INSERT`, `UPDATE`, `UPDATE OR INSERT`, `DELETE`, and `MERGE`.

Generated keys retrieval modifies the statement to add a `RETURNING`-clause. Firebird 4.0 and earlier only support `RETURNING` for DML operations that affect a single row. Attempting to use generated keys retrieval with a statement that affects multiple rows will yield the error *"multiple rows in singleton select"*.

***Since:*** *Firebird 5.0* Support for multi-row `RETURNING` is introduced in Firebird 5.0. Support for multi-row `RETURNING` is available since Jaybird 5.

The examples in this section use the following (Firebird 3.0) table:

*Example person table*

```
create table PERSON (
  ID integer generated by default as identity constraint pk_employee primary key,
  FIRSTNAME varchar(20),
  LASTNAME varchar(20),
  BIRTHDATE date,
  "age" integer generated always as (datediff(year, birthdate, current_date))
)
```

### 10.2.1. Basic generated keys retrieval

This form of generated keys execution involves the following methods:

- `Connection.prepareStatement(String sql, int autoGeneratedKeys)`
- `Statement.execute(String sql, int autoGeneratedKeys)`
- `Statement.executeUpdate(String sql, int autoGeneratedKeys)`
- `Statement.executeLargeUpdate(String sql, int autoGeneratedKeys)`

When `NO_GENERATED_KEYS` is passed, the query will be executed as a normal query.

When `RETURN_GENERATED_KEYS` is passed, the driver will return *all* columns of the table as generated keys. The columns are ordered by ordinal position (as reported in the JDBC metadata of the table). It is advisable to retrieve the values from the `getGeneratedKeys()` result set by column name.

We opted to include all columns as it is next to impossible to decide which columns are populated by a trigger or otherwise. Only returning the primary key will be too restrictive (consider computed columns, default values, etc).

Passing `NO_GENERATED_KEYS` hardcoded should normally not be done. It would be better to use the equivalent `prepareStatement` or `executeXXX` method that only accepts a `String`. Use of the value `NO_GENERATED_KEYS` only makes sense in code that dynamically decides between `NO_GENERATED_KEYS` and `RETURN_GENERATED_KEYS`.

#### Example using RETURN_GENERATED_KEYS

The following will insert a person using a `Statement` and retrieve the generated id using `Statement.RETURN_GENERATED_KEYS`:

*Statement generated keys retrieval*

```
Connection connection = ...;
try (Statement statement = connection.createStatement()) {
  statement.executeUpdate(
      "insert into person(firstname, lastname, birthdate) "
          + "values ('Mark', 'Rotteveel', date'1979-01-12')",
      Statement.RETURN_GENERATED_KEYS);                ①

  try (ResultSet keys = statement.getGeneratedKeys()) { ②
    if (keys.next()) {                                 ③
      int generatedId = keys.getInt("id");             ④
      int age = keys.getInt("age");                    ⑤
      String firstName = keys.getString("firstname");  ⑥

      System.out.printf("Inserted: %s, Id: %d, Age: %d%n",
          firstName, generatedId, age);
    }
  }
}
```

① Use of `Statement.RETURN_GENERATED_KEYS` instructs Jaybird to parse the statement and add a `RETURNING` clause with all columns of the `PERSON` table

② Get the generated keys result set from the statement

③ Just like a normal result set, it is positioned before the first row, so you need to call `next()`

④ The generated value of the `ID` column

⑤ The calculated value of the `AGE` column

⑥ The generated keys result set also contains the normal columns like `FIRSTNAME`

The equivalent using `PreparedStatement` is:

*Prepared statement generated keys retrieval*

```
try (PreparedStatement statement = connection.prepareStatement(
        "insert into person(firstname, lastname, birthdate) values (?, ?, ?)",
        Statement.RETURN_GENERATED_KEYS)) {            ①
  statement.setString(1, "Mark");
  statement.setString(2, "Rotteveel");
  statement.setObject(3, LocalDate.of(1979, 1, 12));

  statement.executeUpdate();
  try (ResultSet keys = statement.getGeneratedKeys()) { ②
    if (keys.next()) {                                 ③
      int generatedId = keys.getInt("id");             ④
      int age = keys.getInt("age");                    ⑤
      String firstName = keys.getString("firstname");

      System.out.printf("Inserted: %s, Id: %d, Age: %d%n",
          firstName, generatedId, age);
    }
  }
}
```

① Besides use of parameters, the only real difference is that use of `Statement.RETURN_GENERATED_KEYS` moved from execution to prepare. This makes sense if you consider that once prepared, the statement can be reused.

### 10.2.2. Generated keys by column index

This form of generated keys execution involves the following methods:

- `Connection.prepareStatement(String sql, int[] columnIndexes)`
- `Statement.execute(String sql, int[] columnIndexes)`
- `Statement.executeUpdate(String sql, int[] columnIndexes)`
- `Statement.executeLargeUpdate(String sql, int[] columnIndexes)`

The values in the `int[]` parameter are the ordinal positions of the columns as specified in the (JDBC) metadata of the table.



Passing a null or empty array will throw an exception with message "*Generated keys array columnIndexes was empty or null. A non-empty array is required.*"

Passing invalid ordinal positions will throw an exception with message "*Generated keys column position <position> does not exist for table <tablename>. Check DatabaseMetaData.getColumns (column ORDINAL_POSITION) for valid values.*"

### Example using column indexes

Retrieval by column index uses the ordinal position as reported in `DatabaseMetaData.getColumns`, column `ORDINAL_POSITION`. In practice this is the value of `RDB$RELATION_FIELDS.RDB$FIELD_POSITION + 1` of that column.

In our example, the columns are

1. `ID`
2. `FIRSTNAME`
3. `LASTNAME`
4. `BIRTHDATE`
5. `age`

*Prepared statement generated keys retrieval by index*

```
try (PreparedStatement statement = connection.prepareStatement(
         "insert into person(firstname, lastname, birthdate) values (?, ?, ?)",
         new int[] { 1, 5 })) {                          ①
  statement.setString(1, "Mark");
  statement.setString(2, "Rotteveel");
  statement.setObject(3, LocalDate.of(1979, 1, 12));

  statement.executeUpdate();
  try (ResultSet keys = statement.getGeneratedKeys()) {
    if (keys.next()) {
      int generatedId = keys.getInt("id");               ②
      int age = keys.getInt(2);                          ③

      System.out.printf("Id: %d, Age: %d%n",
          generatedId, age);
    }
  }
}
```

① Instead of `Statement.RETURN_GENERATED_KEYS`, the column indices are passed in an array, in this case `1` for `ID` and `5` for `age`.

② Retrieval of the first column, `ID`, by name

③ Retrieval of the second column, `age`, by result set position. Notice that the index used for retrieval does not match the column position in the table (5) passed in the prepare. As this is the

second column in the result set, it is retrieved from the result set by 2.

### 10.2.3. Generated keys by column name

This form of generated keys execution involves the following methods:

- `Connection.prepareStatement(String sql, String[] columnNames)`
- `Statement.execute(String sql, String[] columnNames)`
- `Statement.executeUpdate(String sql, String[] columnNames)`
- `Statement.executeLargeUpdate(String sql, String[] columnNames)`

The values in the `String[]` are the column names to be returned. The column names provided are processed as is and are not checked for validity or the need of quoting. Providing non-existent or incorrectly (un)quoted columns will result in an exception when the statement is processed by Firebird (be aware: the JDBC specification is not entirely clear if this is valid behavior, so this might change in the future). This method is the fastest as it does not retrieve metadata from the server.



Passing a null or empty array will throw an exception with message *"Generated keys array columnNames was empty or null. A non-empty array is required."*

#### Example using column names

*Prepared statement generated keys retrieval by name*

```
try (PreparedStatement statement = connection.prepareStatement(
         "insert into person(firstname, lastname, birthdate) values (?, ?, ?)",
         new String[] { "id", "\"age\"" })) {           ①
  statement.setString(1, "Mark");
  statement.setString(2, "Rotteveel");
  statement.setObject(3, LocalDate.of(1979, 1, 12));

  statement.executeUpdate();
  try (ResultSet keys = statement.getGeneratedKeys()) {
    if (keys.next()) {
      int generatedId = keys.getInt("id");
      int age = keys.getInt("age");

      System.out.printf("Id: %d, Age: %d%n",
          generatedId, age);
    }
  }
}
```

① The column names are passed as is, this means that correct quoting is required for case-sensitive columns (and other names that require quoting).



The requirement to pass column names correctly quoted is not specified in the JDBC standard. It may change in future Jaybird versions to conform with column

> names as returned from `DatabaseMetaData.getColumn`. That is, unquoted exactly as stored in `RDB$RELATION_FIELDS.RDB$FIELD_NAME`. Quoting the column names would then be done by Jaybird.
>
> When this changes, a connection property for backwards compatibility will be provided.

## 10.2.4. Configuring generated keys support

The connection property `generatedKeysEnabled` (alias `generated_keys_enabled`) allows the behaviour of generated keys support to be configured. This property is also available on data sources.

This property supports the following values (case-insensitive):

- `default`: default behaviour to enable generated keys for statement types with `RETURNING` clause in the connected Firebird version. Absence of this property, `null` or empty string implies `default`.
- `disabled`: disable support. Attempts to use generated keys methods other than using `Statement.NO_GENERATED_KEYS` will throw a `SQLFeatureNotSupportedException`.
- `ignored`: ignore generated keys support. Attempts to use generated keys methods will not attempt to detect generated keys support and execute as if the statement generates no keys. The `Statement.getGeneratedKeys()` method will always return an empty result set. This behaviour is equivalent to using the non-generated keys methods.
- A comma-separated list of statement types to enable.

For `disabled` and `ignored`, `DatabaseMetaData.supportsGetGeneratedKeys` will report `false`.

Because of the behaviour specified in the next section, typos in property values will behave as `ignored` (e.g. using `generatedKeysEnabled=disable` instead of `disabled` will behave as `ignored`).

### Selectively enable statement types

This last option allows you to selectively enable support for generated keys. For example, `generatedKeysEnabled=insert` will only enable it for `insert` while ignoring it for all other statement types. Statement types that are not enabled will behave as if they generate no keys and will execute normally. For these statement types, `Statement.getGeneratedKeys()` will return an empty result set.

Possible statement type values (case-insensitive) are:

- `insert`
- `update`
- `delete`
- `update_or_insert`
- `merge`

Invalid values will be ignored. If none of he specified statement types are supported by Firebird, it will behave as `ignored`.[1]

Some examples:

- `jdbc:firebird://localhost/testdb?generatedKeysEnabled=insert` will only enable insert support
- `jdbc:firebird://localhost/testdb?generatedKeysEnabled=merge` will only enable merge support. But only on Firebird 3.0 and higher, for Firebird 2.5 this will behave as `ignored` given the lack of `RETURNING` support for merge.
- `jdbc:firebird://localhost/testdb?generatedKeysEnabled=insert,update` will only enable insert and update support

This feature can be used to circumvent issues with frameworks or tools that always use generated keys methods for prepare or execution. For example with `UPDATE` statements that touch multiple records and — given the Firebird 4.0 and earlier limitations for `RETURNING` — produce the error "*multiple rows in singleton select*".

## 10.3. Inline blobs

***Since:*** *Jaybird 5.0.8/6.0.2*
***Since:*** *Firebird 5.0.3*

Inline blobs are a performance optimization for reading blobs as part of the query.

When querying a blob column, the row data received from the server contains only the blob id, not the data of the blob. Without inline blobs, the client will need to retrieve the blob data separately, requiring additional round trips between client and server for each blob. With inline blobs, small blobs — default less than 65535 bytes — are sent together with the row data, and the client can then serve the blob from the data it has locally. Inline blobs can also increase the effectiveness of wire compression — if enabled, assuming the blob data compresses well.

In short, inline blobs increase throughput when querying small blobs, especially on high latency network connections.

This is not entirely free:

- Fetching rows itself can be slower as more data is sent for a fetch.
- Caching the inline blobs consumes memory.

  By default, a maximum of 10 MiB per connection is used for this cache.

If the maximum blob cache size is reached, received inline blobs will be discarded. For pure Java connections, an inline blob is removed from the cache on first use, or when the transaction associated with the blob ends. The native client implementation may have different cache eviction rules.

As pure Java connections remove the inline blob from the cache on first use, subsequent attempts to read the same blob — by getting a different instance of `java.sql.Blob` or through multiple calls to the `ResultSet.getXXX` methods — will use a server-side blob. This can also happen if multiple columns or rows, even in different result sets on the same connection, point to the same blob id in the same transaction.

If you execute queries returning blobs, while those blobs are never actually opened, you may fill up the cache and later received inline blobs are then discarded. Especially in long-running transactions, this may reduce the effectiveness of this feature.

Inline blobs are introduced in Firebird 5.0.3 (protocol 19). For pure Java connections, you need Jaybird 5.0.8 or higher or Jaybird 6.0.2 or higher. For native connections, you need *fbclient* 5.0.3 or higher, irrespective of the Jaybird version; to configure inline blobs for native connections, you will also need Jaybird 5.0.8 or higher or Jaybird 6.0.2 or higher.

### 10.3.1. Configuring inline blobs

There are two connection properties affecting inline blobs:

`maxInlineBlobSize` **(aliases:** `max_inline_blob_size`, `isc_dpb_max_inline_blob_size`**)**

Maximum size in bytes of the blob (default: `65535`).
A value of `0` will disable sending of inline blobs.

The maximum allowed value is decided by the Firebird server, and is currently `65535`; this may change in the future

If a blob is smaller than the specified size, the server will send it inline. The size includes segment lengths, so the actual maximum blob data received is `N * 2` bytes smaller, where *N* is the number of segments of the actual blob, or at least one segment for stream blobs.

The default can be changed with system property `org.firebirdsql.jdbc.defaultMaxInlineBlobSize`.

`maxBlobCacheSize` **(aliases:** `max_blob_cache_size`, `isc_dpb_max_blob_cache_size`**)**

Maximum size in bytes — per connection — of the blob cache (default: `10485760` or 10 MiB).
A value of `0` will disable the cache, but does not disable sending of inline blobs. Set `maxInlineBlobSize` to `0` to disable sending of inline blobs.

For pure Java, only the data size is counted towards the cache size. For native, the segment lengths also count towards the cache size.

The default can be changed with system property `org.firebirdsql.jdbc.defaultMaxBlobCacheSize`.

Without explicitly setting the connection or system property, native connections will use the default of *fbclient* (which are currently the same as those of Jaybird).

## 10.4. Connection property ignoreProcedureType

On Firebird 2.1 and higher, Jaybird will use the procedure type information from the database metadata to decide how to execute `CallableStatement`. When a procedure is selectable, Jaybird will automatically transform a call-escape or `EXECUTE PROCEDURE` statement to a `SELECT`.

In some cases this automatic transformation to use a `SELECT` leads to problems. You can explicitly set `FirebirdCallableStatement.setSelectableProcedure(false)` to fix most of these issues, but this is not always an option. For example spring-data-jpa's `@Procedure` will not work correctly with selectable

procedures, but you can't call `setSelectableProcedure`.

To disable this automatic usage of procedure type information, set connection property `ignoreProcedureType=true`. When necessary you can use `FirebirdCallableStatement.setSelectableProcedure(true)` to execute a procedure using `SELECT`.

Be aware though, when `EXECUTE PROCEDURE` is used with a selectable procedure, it is executed only up to the first `SUSPEND`, and the rest of the stored procedure is not executed.

For Firebird 2.0 and lower this property has no effect, as there the procedure type information is not available.

## 10.5. Maximum field size (`maxFieldSize`)

***Removed in:*** *Jaybird 7*

In Jaybird 6 and older, the `maxFieldSize` property of `Statement` is effectively ignored — the value is only stored, and otherwise ignored.

***Since:*** *Jaybird 7*

In Jaybird 7 and higher, the `maxFieldSize` property of `Statement` is used to truncate values returned from a result set.

The `maxFieldSize` value is defined in bytes, and is applied when the value is greater than zero. Truncation occurs for `CHAR`, `VARCHAR`, `BINARY`, `VARBINARY`, `BLOB SUB_TYPE TEXT` (JDBC `LONGVARCHAR`), and `BLOB SUB_TYPE BINARY` and other Firebird built-in (positive) blob subtypes (JDBC `LONGVARBINARY`), but not custom (negative) subtypes (Jaybird considers those JDBC `BLOB`, and JDBC specifies that type should not be limited by `maxFieldSize`).

In case of `CHAR`, `VARCHAR`, `BINARY`, and `VARBINARY`, the truncation is performed when fetching a row (in the GDS-ng layer) *if* it is not an `RDB$DB_KEY` column. This is a client-side truncation, and does not reduce the number of bytes transferred, only the number of bytes retained in memory and returned from the result set.

In the case of built-in (non-negative) blob subtypes, the truncation is performed in the result set field implementation in the JDBC layer. This is a client-side truncation, but it may reduce the number of bytes transferred *if* the blob wasn't already transferred as an inline blob.

For these blob subtypes, the truncation will not occur for methods returning a `Blob`, `Clob`, `InputStream` or `Reader` *if* the result set is not cached locally (cached result sets are used for holdable result sets and emulated scrollable result sets). In the case of `InputStream` and `Reader`, this doesn't conform to the JDBC requirements, but avoided complications in the implementation. We may address this in the future.

Caveats:

- Given the truncation happens at a set number of bytes, values in a multibyte character set (UTF8) might end in the Unicode replacement character (U+FFFD) due to truncation before the end of the encoded codepoint.

- Jaybird will accept any non-negative value for `maxFieldSize`. The JDBC apidoc recommends using values greater than `256`.
- A too small `maxFieldSize` may result in "wrong" values or missing precision being returned without error for numeric and Boolean getters; for datetime getters it may result in missing precision without errors, or parse errors.
- Setting the max field size after execute may not be immediately applied to the current result set for `CHAR`, `VARCHAR`, `BINARY`, and `VARBINARY`:
  - For a locally cached result set: never, as the rows were truncated to the `maxFieldSize` on execute.
  - Otherwise, already fetched rows are truncated to the `maxFieldSize` on their fetch, and only rows returned by a subsequent fetch will apply the new limit.
- For `LONGVARCHAR`/`LONGVARBINARY`, the value will be truncated on access, with the following caveats:
  - For cached blobs, the limit is applied on retrieval, and applies to all methods (as the cached value is truncated).
  - The value of a cached blob (holdable result set or emulated scrollable result set) will be truncated to the initial `maxFieldSize`. Subsequent use of a larger `maxFieldSize` will continue to return the shorter value.
- The detection of `RDB$DB_KEY` columns includes "normal" `BINARY`/`CHAR CHARACTER SET OCTETS` columns called `DB_KEY`.

For further details, see jdp-2025-07: Statement Max Field Size [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2025-07-statement-max-field-size.adoc].

## 10.6. Connection property scrollableCursor

***Since:*** *Firebird 5.0*

On Firebird 5.0 and higher, Jaybird supports server-side scrollable cursors.

This feature is only available on pure Java connections, and only when explicitly enabled with connection property `scrollableCursor`. Support is only available in pure Java, as the bindings for native and embedded use the legacy fbclient API, and scrollable cursors are not exposed in the legacy fbclient API.

The connection property `scrollableCursor` has the following values (case-insensitive):

`EMULATED`
: (default) Use client-side cached result set; this is the same behaviour as previous Jaybird versions used

`SERVER`
: Use server-side scrollable cursors, if possible



A future version of Jaybird may make `SERVER` the default.

If a connection does not support scrollable cursors, or if holdable cursors are requested, the behaviour will silently fall back to *emulated* scrollable cursors.

Compared to emulated scrollable cursor, server-side scrollable cursors exhibit slightly different behaviour in Jaybird 5:

- New rows are inserted at the end of the cursor, where in *emulated* they are inserted before the current row
- Deleted rows are visible with an all-null marker row, where in *emulated*, the deleted row is removed from the result set
- Result sets now report `true` for `rowUpdated()`, `rowDeleted()` and `rowInserted()` for rows updated, deleted or inserted through the result set.

  This is not yet reflected in `updatesAreDetected()`, `deletesAreDetected()` and `insertsAreDetected()` of `DatabaseMetaData`. This will be corrected when we retrofit the new behaviour for *emulated* as well.

In Jaybird 6 and higher, *emulated* has been changed to have the same behaviour as *server*.

See also jdp-2021-04 [https://github.com/FirebirdSQL/jaybird/tree/master/devdoc/jdp/jdp-2021-04-real-scrollable-cursor-support.md.md] and jdp-2024-05 [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2024-05-behavior-of-updatable-result-sets.adoc].

## 10.7. Connection property useServerBatch

Connection property `useServerBatch` is a Boolean property to configure use of server-side batch updates. The default value is `true`, to use server-side batch updates when possible. Set to `false` to disable server-side batch updates and always use emulated batch updates.

See Server-side Batch Updates for more information.

## 10.8. Connection property serverBatchBufferSize

Connection property `serverBatchBufferSize` configures the server-side batch buffer size in bytes.

*Possible values*

`< 0` use server-side default (16MB as of Firebird 4.0)

`0` (default) use server-side maximum (256MB as of Firebird 4.0)

`> 0` batch buffer size capped at server-side maximum

Too small buffer sizes will use 128KB or the size necessary for two rows (whichever is bigger). Too large buffer sizes will use the server-side maximum (256MB as of Firebird 4.0).

See Server-side Batch Updates for more information.

# 10.9. Connection property allowTxStmts

***Since:*** *Jaybird 6*

Enables support for the transaction management statements `COMMIT [WORK]`, `ROLLBACK [WORK]` (without retain or savepoint) and `SET TRANSACTION`.

When disabled (`false`, the default), Jaybird will reject attempts to execute or prepare these statements with one of the following errors:

**337248313** *"Execution of COMMIT statement is not allowed, use Connection.commit(), or set connection property allowTxStmts to true"*

**337248314** *"Execution of ROLLBACK statement is not allowed, use Connection.rollback(), or set connection property allowTxStmts to true"*

**337248315** *"Execution of SET TRANSACTION statement is not allowed, use Connection.setTransactionIsolation(int) or FirebirdConnection.setTransactionParameters(..), or set connection property allowTxStmts to true"*

When enabled (`true`), Jaybird will support these statements under the following conditions:

- Execution with the `execute`, `executeUpdate` or `executeLargeUpdate` methods of `Statement` or `PreparedStatement`.
- For `PreparedStatement`, the statement is prepared with one of the `prepareStatement` methods of `Connection`.
- When auto-commit is disabled, executing `COMMIT` or `ROLLBACK` will commit or rollback the current transaction, or do nothing if there is no current transaction. The implementation doesn't execute the actual statements, but instead calls `Connection.commit()` or `Connection.rollback()`.
- For the `SET TRANSACTION` statement, if there is no active transaction. The implementation executes the statement using "execute immediate", which means it is not executed through the statement handle.

These statements will throw an `SQLException` under the following conditions:

- Execution when auto-commit mode is enabled.
- Execution when the connection is participating in a distributed transaction.
- For the `SET TRANSACTION` statement, execution while there is an active transaction.
- Preparing with the `prepareCall` methods of `Connection`.
- Execution with the `executeQuery` methods of `Statement` or `PreparedStatement`.
- Adding to a batch (using the `addBatch` methods of `Statement` or `PreparedStatement`).

The `allowTxStmts` connection property has no effect on the other transaction management statements, i.e. `COMMIT [WORK] RETAIN`, `ROLLBACK [WORK] RETAIN`, `ROLLBACK [WORK] TO [SAVEPOINT] name`, `SAVEPOINT name` and `RELEASE SAVEPOINT name [ONLY]`. These statements can always be executed. However, for the savepoint related methods it is advisable to use the relevant JDBC API methods

instead.

See also Transaction management statements, and jdp-2024-01: Explicit support for transaction statements [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2024-01-explicit-support-for-transaction-statements.adoc].

## 10.10. Connection property extendedMetadata

***Since:*** *Jaybird 5.0.5*

By default, Jaybird queries extended metadata information. This can be disabled by setting the connection property `extendedMetadata` to `false`.

This setting only affects the following methods:

`ResultSetMetaData.getPrecision(int)`

When enabled, the actual precision of `NUMERIC` and `DECIMAL` columns is queried. When disabled, the precision will be estimated.

`ResultSetMetaData.isAutoIncrement(int)`

When enabled (***Since:*** *Jaybird 6*), the identity state of `INTEGER`, `BIGINT` and `SMALLINT` columns is queried on Firebird 3.0 and higher. When disabled, the result is always `false`.

Invocation of either of these methods will query the extended metadata needed for *both* methods.

Setting `extendedMetadata` to `false` may improve performance of these methods in exchange for an estimated precision, and not being able to determine the auto-increment state of a column.

## 10.11. Schema support

See Non-standard statement behaviour.

[1] This is not the case for the unsupported Firebird 1.0 and 1.5 versions. There this will behave similar to `disabled`, and you will need to explicitly specify `ignored` instead to get this behaviour.

# Chapter 11. General

## 11.1. Logging

Jaybird logs a variety of information during its work.

For logging, jaybird uses the following log levels:

| Jaybird log level | Description |
|---|---|
| *trace* | low-level debug information |
| *debug* | debug information |
| *info* | informational messages |
| *warn* | warnings |
| *error* | errors |
| *fatal* | severe/fatal errors (though in general, level *error* will be used instead of *fatal*)<br><br>***Removed in:*** *Jaybird 6* |

### 11.1.1. Java Platform Logging API

***Since:*** *Jaybird 6*

Since Jaybird 6, the Java Platform Logging API (JEP 264 [https://openjdk.org/jeps/264]) is used.

The Java Platform Logging API by default logs to `java.util.logging`, but it is possible to plugin different logging platforms. For example, Log4j provides the `log4j-jpl` dependency which will replace the default binding to `java.util.logging` with one to Log4j.

### 11.1.2. java.util.logging

***Removed in:*** *Jaybird 6*

Jaybird uses `java.util.logging` by default.

Jaybird applies the following mapping for its log levels:

| Jaybird log level | *jul* log level |
|---|---|
| `Logger.trace` | `Level.FINER` |
| `Logger.debug` | `Level.FINE` |
| `Logger.info` | `Level.INFO` |
| `Logger.warn` | `Level.WARNING` |
| `Logger.error` | `Level.SEVERE` |
| `Logger.fatal` | `Level.SEVERE` |

### 11.1.3. Disable logging

***Since:*** *Jaybird 6* To disable logging, configure your logging library. For example, when using `java.util.logging`, use a properties file (specified with `-Djava.util.logging.config.file=<file>`) including:

```
org.firebirdsql.level = OFF
```

Consult the documentation of `java.util.logging` or your actual logging library for other options to disable logging.

***Removed in:*** *Jaybird 6* To disable logging, specify system property `org.firebirdsql.jdbc.disableLogging` with value `true`.

### 11.1.4. Console logging

***Since:*** *Jaybird 6* To write logging to the console, configure your logging library. For example, when using `java.util.logging`, use a properties file (specified with `-Djava.util.logging.config.file=<file>`) including:

```
handlers = java.util.logging.ConsoleHandler
```



Contrary to the solution available under Jaybird 5 and earlier, `java.util.logging` does not provide a (standard) way to log to `System.out` (its `ConsoleHandler` logs to `System.err`), this would require a custom handler implementation.

***Removed in:*** *Jaybird 6* Jaybird can write its logging to the `System.out` for *info* and lower and `System.err` for *warn* and above. Levels *debug* and *trace* are disabled in the implementation. To enable logging to the console, you can set system property `org.firebirdsql.jdbc.forceConsoleLogger` to `true`.

### 11.1.5. Custom logging implementation

#### Custom logging in Jaybird 6 and higher

***Since:*** *Jaybird 6* If you need a custom logger implementation under Jaybird 6 or higher, you will need to provide an implementation of `java.lang.System.Logger` and `java.lang.System.LoggerFinder` and a service loader definition.

Make sure you really need this: if you want to use a different logging library, it may already have the necessary implementation (for example, Log4j has the `log4j-jpl` dependency).

#### Custom logging in Jaybird 5 and earlier

***Removed in:*** *Jaybird 6*

You can provide your own logging implementation if you don't want to use `java.util.logging` or

console logging.

To provide a custom logging implementation, you need to implement interface `org.firebirdsql.logging.Logger`. This implementation must be public and must have a public constructor with a single String argument for the logger name. Set system property `org.firebirdsql.jdbc.loggerImplementation` with the fully-qualified name of your implementation.

The `org.firebirdsql.logging.Logger` interface should be considered volatile and might change in minor releases (but not point/bugfix releases).

For example:

```
package org.example.jaybird.logging;

public class CustomLogger implements org.firebirdsql.logging.Logger {
    public CustomLogger(String name) {
        // create the logger
    }
    // implementation of org.firebirdsql.logging.Logger interface
}
```

On the Java command line, specify:

```
-Dorg.firebirdsql.jdbc.loggerImplementation=org.example.jaybird.logging.CustomLogger
```

# Chapter 12. Datatype reference



This section documents non-standard datatypes supported by Jaybird, behaviour deviating from standard JDBC expectations, or recently introduced features. It does not provide full documentation for all supported datatypes.

## 12.1. Binary types BINARY/VARBINARY

The JDBC (and SQL standard) types `BINARY` and `VARBINARY` are called `CHAR(n) CHARACTER SET OCTETS` and `VARCHAR(n) CHARACTER SET OCTETS` in Firebird.

In Firebird, the SQL type names `BINARY` and `VARBINARY/BINARY VARYING` are effectively aliases for `(VAR)CHAR(n) CHARACTER SET OCTETS`.

In Java, `BINARY` and `VARBINARY` are usually handled with byte arrays and `InputStream/OutputStream`.

### 12.1.1. Always BINARY/VARBINARY

Jaybird always handles `(VAR)CHAR(n) CHARACTER SET OCTETS` as JDBC types as `BINARY/VARBINARY`.

Jaybird will report the JDBC `BINARY/VARBINARY` type information in all metadata (`DatabaseMetaData`, `ResultSetMetaData`, `ParameterMetaData`) for columns and parameters of type `(VAR)CHAR(n) CHARACTER SET OCTETS`, and `getObject(int/String)` will always return `byte[]`.

The getters (on result set/callable statement), setters (prepared/callable statement), and update methods (result set) for columns of this type are restricted to:

- `get/set/updateNull`
- `get/set/updateBytes`
- `get/set/updateBinaryStream`
- `get/set/updateAsciiStream`
- `get/set/updateString` (using the default encoding or connection encoding)
- `get/set/updateCharacterStream` (using the default encoding or connection encoding)
- `get/set/updateObject` (with `String`, `byte[]`, `InputStream`, `Reader`)
- `get/setObject` with a `java.sql.RowId`
- `get/setRowId`

Other getters/setters/updaters or object types supported for 'normal' `(VAR)CHAR` fields are not available.

## 12.2. Type BOOLEAN

The SQL datatype `BOOLEAN` is for Boolean (truth) values.

### 12.2.1. Support for BOOLEAN

On parameters of type BOOLEAN, Jaybird supports most of the other Java types, using the following mapping:

| Type | setXXX | getXXX |
|---|---|---|
| String | "true"/"Y"/"T"/"1" sets true, everything else sets false | true is "true", false is "false" |
| integer types | 0 sets false, everything else sets true | true is 1, false is 0 |
| float | Exact 0.0f sets false, everything else sets true [1] | true is 1.0f, false is 0.0f |
| double | Exact 0.0 sets false, everything else sets true [1] | true is 1.0, false is 0.0 |
| BigDecimal | 0 (ZERO) (using compareTo) sets false, everything else sets true | true is 1 (ONE), false is 0 (ZERO) |

We recommend to avoid the float, double and BigDecimal options.

### 12.2.2. Workarounds for booleans

Firebird 2.5 and earlier do not support BOOLEAN, but support for booleans can be simulated

The following options are recommended:

- Use SMALLINT with values 0 and 1
- Use CHAR(1) (or VARCHAR(1)) with values 'Y' and 'N'
- Use CHAR or VARCHAR with length 5 or longer with values 'true' and 'false'

We recommend creating a domain for 'simulated' booleans with a check constraint to restrict the possible values. If you do this, name the domain something like D_BOOLEAN, and avoid the name BOOLEAN to prevent problems when upgrading to Firebird 3.0.

For non-boolean types, Jaybird supports the following conversions with setBoolean and getBoolean:

| Type | getBoolean | setBoolean |
|---|---|---|
| (VAR)CHAR(<5) | 'Y'/'T'/'1'/'true' is true (case-insensitive), everything else is false | true sets 'Y', false sets 'N' |
| (VAR)CHAR(>=5) | 'Y'/'T'/'1'/'true' is true (case-insensitive), everything else is false | true sets 'true', false sets 'false' |
| BLOB SUB_TYPE TEXT | 'Y'/'T'/'1'/'true' is true (case-insensitive), everything else is false | true sets 'true', false sets 'false' |
| SMALLINT/INTEGER/BIGINT | 1 is true, everything else is false | true sets 1, false sets 0 |
| DECIMAL/NUMERIC | Truncated integral value 1 is true, everything else is false | true sets 1, false sets 0 |

| Type | getBoolean | setBoolean |
|---|---|---|
| REAL/FLOAT | Exact 1.0f is true, everything else is false [2] | true sets 1.0f, false sets 0.0f |
| DOUBLE PRECISION | Exact 1.0 is true, everything else is false [2] | true sets 1.0, false sets 0.0 |
| DECFLOAT | Exact 1E0 is true, everything else is false (including 1.0E0!) [3] | true sets 1E0, false sets 0E0 |

We recommend to avoid the DECIMAL, NUMERIC, REAL/FLOAT, DOUBLE PRECISION or DECFLOAT options.

# 12.3. Date/time types

## 12.3.1. Time zones

Firebird 4.0 introduced time zone types, with types TIME WITH TIME ZONE and TIMESTAMP WITH TIME ZONE. See the Firebird 4.0 release notes [https://firebirdsql.org/file/documentation/release_notes/html/en/4_0/rlsnotes40.html] and doc/sql.extensions/README.time_zone.md in the Firebird installation for details on these types.

Two connection properties, dataTypeBind and sessionTimeZone, are available to provide limited support for these types. See Defining time zone data type bind and Connection property sessionTimeZone for details.

The time zone types are supported under Java 8 and higher, using the Java 8 (or higher) version of Jaybird. Time zone types are not supported under Java 7, and you will need to enable legacy time zone bind to use these types. With legacy time zone bind, Firebird will convert to the equivalent TIME and TIMESTAMP (WITHOUT TIME ZONE) types using the session time zone. Time zone binds can be configured with connection property dataTypeBind, for more information see Defining time zone data type bind.

### Scope of time zone support

JDBC 4.2 introduced support for time zones, and maps these types to java.time.OffsetTime and java.time.OffsetDateTime. JDBC does not define explicit setters for these types. Use setObject(index, value), updateObject(index, value), getObject(index/name) or getObject(index/name, classType).

Firebird 4.0 supports both offset and named time zones. Given the definition in JDBC, Jaybird defaults to offset time zones. On retrieval of a value with a named zone, Jaybird will make a best effort to convert to the equivalent offset using Java's time zone information. If no mapping is available the time will be returned at UTC (offset zero).

It is also possible to get and set java.time.ZonedDateTime, which preserves the named zone information.

Jaybird supports the following Java types on fields of time zone types (those marked with * are not defined in JDBC)

TIME WITH TIME ZONE:

- `java.time.OffsetTime` (default for `getObject`)
  - On get, if the value is a named zone, it will derive the offset using the base date 2020-01-01 (in 4.0.0 it used the current date). The offset can be different from the offset of the `OffsetDateTime` for the same value.
- `java.time.OffsetDateTime`
  - On get the current date is added
    - For a named zone, the time in the zone is derived at 2020-01-01 and then rebased to the current date. As a result, the offset can be different from an `OffsetTime`.
  - On set the date information is removed
- `java.time.ZonedDateTime` (*)
  - On get the time in the zone is derived at 2020-01-01 and then rebased to the current date.
  - On set, the time is rebased to 2020-01-01 and then the date information is removed.
- `java.lang.String`
  - On get applies `OffsetTime.toString()` (e.g. `13:25:13.1+01:00`)
  - On set tries the default parse format of either `OffsetTime` or `OffsetDateTime` (e.g. `13:25:13.1+01:00` or `2019-03-10T13:25:13+01:00`) and then sets as that type
- `java.sql.Time` (*)
  - On get obtains `java.time.OffsetDateTime`, converts this to epoch milliseconds and uses `new java.sql.Time(millis)`
  - On set applies `toLocalTime()`, combines this with `LocalDate.now()` and then derives the offset time for the default JVM time zone
- `java.sql.Timestamp` (*)
  - On get obtains `java.time.OffsetDateTime`, converts this to epoch milliseconds and uses `new java.sql.Timestamp(millis)`
  - On set applies `toLocalDateTime()` and derives the offset time for the default JVM time zone

`TIMESTAMP WITH TIME ZONE`:

- `java.time.OffsetDateTime` (default for `getObject`)
- `java.time.OffsetTime` (*)
  - On get, the date information is removed
  - On set, the current date is added
- `java.time.ZonedDateTime` (*)
- `java.lang.String`
  - On get applies `OffsetDateTime.toString()` (e.g. `2019-03-10T13:25:13.1+01:00`)
  - On set tries the default parse format of either `OffsetTime` or `OffsetDateTime` (e.g. `13:25:13.1+01:00` or `2019-03-10T13:25:13+01:00`) and then sets as that type
- `java.sql.Time` (*)

    - On get obtains `java.time.OffsetDateTime`, converts this to epoch milliseconds and uses `new java.sql.Time(millis)`
    - On set applies `toLocalTime()`, combines this with `LocalDate.now()` and then derives the offset date time for the default JVM time zone
- `java.sql.Timestamp` (*)
    - On get obtains `java.time.OffsetDateTime`, converts this to epoch milliseconds and uses `new java.sql.Timestamp(millis)`
    - On set applies `toLocalDateTime()` and derives the offset date time for the default JVM time zone
- `java.sql.Date` (*)
    - On get obtains `java.time.OffsetDateTime`, converts this to epoch milliseconds and uses `new java.sql.Date(millis)`
    - On set applies `toLocalDate()` at start of day and derives the offset date time for the default JVM time zone

In addition, Firebird 4.0 has 'bind-only' data types `EXTENDED TIME/TIMESTAMP WITH TIME ZONE`. These data types can be set through the data type bind configuration and include an extra offset in its data so clients without access to ICU or other time zone data can use the offset as determined by Firebird.

Jaybird provides minimal support for these types by handling them the same as the normal `WITH TIME ZONE` types. That means the extra offset information is ignored and Jaybird will always use the Java time zone information to calculate the offset of a named zone, and if a zone is unknown in Java, Jaybird will fallback to UTC even when the actual offset is available in the 'extended' time zone type.

See also:

jdp-2020-01: Extended Time Zone Types Support [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2020-01-extended-time-zone-types-support.md]

jdp-2020-06: OffsetTime derivation for named zone [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2020-06-offsettime-derivation-for-named-zone.md]

jdp-2020-09: Add ZonedDateTime support [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2020-09-add-zoneddatetime-support.md]

#### Support for legacy JDBC date/time types

For the `WITH TIME ZONE` types, JDBC does not define support for the legacy JDBC types (`java.sql.Time`, `java.sql.Timestamp` and `java.sql.Date`). To ease the transition and potential compatibility with tools and libraries, Jaybird does provide support. However, we strongly recommend to avoid using these types.

Compared to the `WITHOUT TIME ZONE` types, there may be small discrepancies in values as Jaybird uses 1970-01-01 for `WITHOUT TIME ZONE`, while for `WITH TIME ZONE` it uses the current date. If this is problematic, then either apply the necessary conversions yourself, enable legacy time zone bind, or define or cast your columns to `TIME` or `TIMESTAMP`.

#### No support for other java.time types

The types `java.time.LocalTime`, `java.time.LocalDateTime` and `java.time.LocalDate` are not supported for the time zone types. Supporting these types would be ambiguous. If you need to use these, then either apply the necessary conversions yourself, enable legacy time zone bind, or define or cast your columns as `TIME` or `TIMESTAMP`.

Jaybird also does not support non-standard extensions like `java.time.Instant`. If there is interest, we may add them in the future.

### Defining time zone data type bind

Jaybird 3.0.9 added the connection property `dataTypeBind` to defining data type mappings. This can be used to configure conversion of `WITH TIME ZONE` types to a different datatype. When configured, Firebird will present columns or parameters of `TIME(STAMP) WITH TIME ZONE` as the specified type, allowing clients without support for `WITH TIME ZONE` to read or set values.

This property needs to be explicitly set if you are using Jaybird 4 on Java 7 or Jaybird 3 (on any Java version), and need to handle the `WITH TIME ZONE` types. It can also be used for tools or applications that expect `java.sql.Time/Timestamp` types and cannot use the `java.time.OffsetTime/OffsetDateTime` types returned for the `WITH TIME ZONE` types.

To map `TIME WITH TIME ZONE` and `TIMESTAMP WITH TIME ZONE` to the legacy without time zone types, you can use:

*Map with time zone types to without time zone types*

```
Properties props = new Properties();
props.setProperty("dataTypeBind",
    "time with time zone to legacy;timestamp with time zone to legacy");
```

Instead of `legacy`, you can also explicitly specify `time` and `timestamp` respectively.

Firebird will convert a `WITH TIME ZONE` type to the equivalent `WITHOUT TIME ZONE` type using the session time zone to derive the value.

Result set columns and parameters on prepared statements will behave as the equivalent `WITHOUT TIME ZONE` types. This conversion is not applied to the database metadata which will always report `WITH TIME ZONE` information.

The `TIME ZONE TO EXTENDED` binds (including type-specific variants) is only supported under Java 8 and higher using the Java 8 or higher version of Jaybird. As mentioned earlier, the support for 'extended' time zone types will behave identical to the normal time zone types.

See also Data type bind support.

### Connection property sessionTimeZone

The connection property `sessionTimeZone` (alias `session_time_zone`) does two things:

1. specifies the Firebird 4.0 session time zone,

2. specifies the time zone to use when converting values of without time zone types to the legacy JDBC datetime types on all Firebird versions.

See Firebird session time zone for information on the effects of `sessionTimeZone` on the server-side.

Valid values are time zone names known by Firebird, we recommend to use the long names (e.g. `Europe/Amsterdam`) and not the ambiguous short IDs (e.g. `CET`). Although not required, we recommend to use time zone names that are known by Firebird and Java (see Session time zone for conversion for caveats).

In Jaybird 3, `sessionTimeZone` will only configure the server-side session time zone. Client-side, Jaybird will continue to use the JVM default time zone for parsing the without time zone values to the `java.sql.Time/Timestamp/Date` types. Setting `sessionTimeZone` to the JVM default time zone will yield the best (i.e. correct) values, but not setting it (and thus using the server default) will retain behaviour that is backwards compatible with behaviour of previous versions of Jaybird. In Jaybird 4, this property also configures client-side parsing of values to these legacy types.



On Jaybird 3, we recommend not setting this property, or setting it to the default JVM time zone. If you set it to a different time zone, then we recommend that you do not use the legacy `java.sql.Time/Timestamp/Date` types, but instead use `java.time.LocalTime/LocalDateTime/LocalDate`.

The remainder of this section only applies to Jaybird 4 and higher.

By default, Jaybird 4 and higher will use the JVM default time zone as reported by `java.util.TimeZone.getDefault().getID()` as the session time zone. Using the JVM default time zone as the default is the best option in the light of JDBC requirements with regard to `java.sql.Time` and `java.sql.Timestamp` using the JVM default time zone.

To use the default server time zone and the Jaybird 3 and earlier behaviour to use the JVM default time zone, set the connection property to `server`. This will result in the conversion behaviour of Jaybird 3 and earlier. Be aware that this is inconsistent if Firebird and Java are in different time zones.

#### Firebird session time zone

The session time zone is used for conversion between `WITH TIME ZONE` values and `WITHOUT TIME ZONE` values (i.e. using cast or with legacy time zone bind), and for the value of `LOCALTIME`, `LOCALTIMESTAMP`, `CURRENT_TIME` and `CURRENT_TIMESTAMP`, and other uses of the session time zone as documented in the Firebird 4.0 documentation.

The value of `sessionTimeZone` must be supported by the Firebird server. It is possible that time zone identifiers used by Java are not supported by Firebird. If Firebird does not know the session time zone, error (`Invalid time zone region: <zone name>`) is reported on connect.

Jaybird will apply the JVM default time zone as the default session time zone. The use of the JVM default time zone as the default session time zone will result in subtly different behaviour compared to previous versions of Jaybird and — even with Jaybird 4 — Firebird 3.0 or earlier, as current time values like `LOCALTIMESTAMP` (etc) will now reflect the time in the JVM time zone, and not the server time zone rebased on the JVM default time zone.

As an example, with a Firebird in Europe/London and a Java application in Europe/Amsterdam with Firebird time 12:00, in Jaybird 3, the Java application will report this time as 12:00, in Jaybird 4 with Firebird 4.0, this will now report 13:00, as that is the time in Amsterdam if it is 12:00 in London (ignoring potential DST start/end differences).

Other examples include values generated in triggers and default value clauses.

#### Session time zone for conversion

For `WITHOUT TIME ZONE` types, the session time zone will be used to derive the `java.sql.Time`, `java.sql.Timestamp` and `java.sql.Date` values. This is also done for Firebird 3.0 and earlier.

If Java does not know the session time zone, no error is reported, but when retrieving `java.sql.Time`, `java.sql.Timestamp` or `java.sql.Date` a warning is logged and conversion will happen in GMT, which might yield unexpected values.

We strongly suggest that you use `java.time.LocalTime`, `java.time.LocalDateTime` and `java.time.LocalDate` types instead of these legacy datetime types.

For `WITH TIME ZONE` types, the session time zone has no effect on the conversion to the legacy JDBC date/time types: the offset date/time is converted to epoch milliseconds and used to construct these legacy types directly.

Executing `SET TIME ZONE <zone name>` statements after connect will change the session time zone on the server, but Jaybird will continue to use the session time zone set in the connection property for these conversions.

### Time zone support for CONVERT

Although not defined in JDBC (or ODBC), Jaybird has added a non-standard extension to the `CONVERT` JDBC escape to allow conversion to the time zone types.

In addition to the standard-defined types, it also supports the type names `TIME_WITH_TIME_ZONE`, `TIME_WITH_TIMEZONE`, `TIMESTAMP_WITH_TIME_ZONE` and `TIMESTAMP_WITH_TIMEZONE` (and the same with the `SQL_` prefix).

### Caveats for time zone types

- Time zone fields do not support `java.time.LocalDate`, `java.time.LocalTime`, `java.time.LocalDateTime`.
- Firebird 4.0 redefines `CURRENT_TIME` and `CURRENT_TIMESTAMP` to return a `WITH TIME ZONE` type. Use `LOCALTIME` and `LOCALTIMESTAMP` (introduced in Firebird 2.5.9 and Firebird 3.0.4) if you want to ensure a `WITHOUT TIME ZONE` type is used.
- The database metadata will always return JDBC 4.2 compatible information on time zone types, even on Java 7, and even when legacy time zone bind is set. For Java 7 compatibility, the JDBC 4.2 `java.sql.Types` constants `TIME_WITH_TIMEZONE` and `TIMESTAMP_WITH_TIMEZONE` are also defined in `org.firebirdsql.jdbc.JaybirdTypeCodes`.
- The default `sessionTimeZone` is set to the JVM default time zone, this may result in different application behavior for `DATE`, `TIME` and `TIMESTAMP`, including values generated in triggers and

default value clauses. To prevent this, either switch those types to a `WITH TIME ZONE` type, or set the `sessionTimeZone` to `server` or to the actual time zone of the Firebird server.

- As `CURRENT_TIME` uses the session time zone, which usually is a named zone, use in combination with `java.time.OffsetTime` can yield confusing results. For example, if the current date and time is '2020-07-01T14:51:00 Europe/Amsterdam', then retrieving `CURRENT_TIME` as an `OffsetTime` will return the value '14:51:00+01:00', and not '14:51:00+02:00'.
  It is recommended to avoid `CURRENT_TIME` and use `CURRENT_TIMESTAMP` instead.
- Overall, using `TIME WITH TIME ZONE` with named zones is rather fragile and prone to interpretation errors. This is a result of how this is implemented in Firebird: values are stored at UTC with their offset or named zones, where derivation of the time in the named zone needs to use 2020-01-01 as the date for the time zone rules to apply.
  We recommend avoiding `TIME WITH TIME ZONE` where possible.

## 12.4. Decimal floating point type DECFLOAT

Firebird 4.0 introduced the SQL:2016 `DECFLOAT` datatype, a decimal floating point with a precision of 16 or 34 digits (backed by an IEEE-754 Decimal64 or Decimal128). See the Firebird 4.0 documentation for details on this datatype.

Jaybird supports the `DECFLOAT` datatype mapping to `java.math.BigDecimal`. For more information, see DECFLOAT support.

### 12.4.1. DECFLOAT support

The 'default' object type for `DECFLOAT` is a `java.math.BigDecimal`, but conversion from and to the following datatypes is supported:

- `java.math.BigDecimal` (see note 1)
- `byte` (valid range -128 to 127(!); see notes 2, 3)
- `short` (valid range -32768 to 32767; see note 3)
- `int` (valid range $-2^{31}$ to $2^{31}-1$; see note 3)
- `long` (valid range $-2^{63}$ to $2^{63}-1$; see notes 3, 4)
- `float` (valid range -1 * Float.MAX_VALUE to Float.MAX_VALUE; see notes 5, 6, 7, 8, 9)
- `double` (valid range -1 * Double.MAX_VALUE to Double.MAX_VALUE; see notes 6, 7, 8, 9)
- `boolean` (see notes 10, 11)
- `java.lang.String` (see notes 12, 13, 14)
- `java.math.BigInteger` (see notes 15, 16)
- `org.firebirdsql.extern.decimal.Decimal32/64/128` (see notes 17, 18)

The `DECFLOAT` type was not standardized in JDBC until the JDBC 4.5 specification (Java 26).

***Removed in:*** *Jaybird 5.0.12/6.0.5* Up to Jaybird 5.0.11/6.0.4, Jaybird defines a custom type code with value `-6001`.
***Since:*** *Jaybird 5.0.12/6.0.5* Starting with Jaybird 5.0.12/6.0.5, Jaybird changed this type code to `2015`,

as defined by JDBC 4.5.

This value is available through constant `org.firebirdsql.jdbc.JaybirdTypeCodes.DECFLOAT`, and — for JDBC 4.2 and higher — `org.firebirdsql.jdbc.JaybirdType.DECFLOAT`, which is an enum implementing `java.sql.SQLType`. On Java 26 and higher, we recommend using `java.sql.Types.DECFLOAT` or `java.sql.JDBCType.DECFLOAT` instead.

The `DECFLOAT` constants in `JaybirdTypeCodes` and `JaybirdType` may be deprecated for eventual removal once Jaybird no longer supports Java versions before Java 26.

To avoid additional dependencies, Jaybird uses a local ("shaded") copy of the FirebirdSQL/decimal-java [https://github.com/FirebirdSQL/decimal-java] library, with a custom package `org.firebirdsql.extern.decimal`.

### Precision and range

The `DECFLOAT` datatype supports values with a precision of 16 or 34 decimal digits, and an exponent [4] between -398 and 369 (`DECFLOAT(16)`), or between -6176 and 6111 (`DECFLOAT(34)`), so the minimum and maximum values are:

| Type | Min/max value | Smallest (non-zero) value |
|---|---|---|
| `DECFLOAT(16)` | +/-9.9..9E+384 (16 digits) | +/-1E-398 (1 digit) |
| `DECFLOAT(34)` | +/-9.9..9E+6144 (34 digits) | +/-1E-6176 (1 digit) |

When converting values from Java types to `DECFLOAT` and retrieving `DECFLOAT` values as `Decimal32` or `Decimal64`, the following rules are applied:

- Zero values can have a non-zero exponent, and if the exponent is out of range, the exponent value is 'clamped' to the minimum or maximum exponent supported. This behavior is subject to change, and future releases may 'round' to exact `0` (or `0E0`)
- Values with a precision larger than the target precision are rounded to the target precision using `RoundingMode.HALF_EVEN`
- If the magnitude (or exponent) is too low, then the following steps are applied:
  1. Precision is reduced applying `RoundingMode.HALF_EVEN`, increasing the exponent by the reduction of precision.

     An example: a `DECFLOAT(16)` stores values as an integral coefficient of 16 digits and an exponent between `-398` and `+369`. The value `1.234567890123456E-394` or `1234567890123456E-409` is coefficient `1234567890123456` and exponent `-409`. The coefficient is 16 digits, but the exponent is too low by 11.

     If we sacrifice least-significant digits, we can increase the exponent, this is achieved by dividing the coefficient by $10^{11}$ (and rounding) and increasing the exponent by 11. We get coefficient = round(1234567890123456 / $10^{11}$) = 12346 and exponent = -409 + 11 = -398.

     The resulting value is now `12346E-398` or `1.2346E-394`, or in other words, we sacrificed precision to make the value fit.

2. If after the previous step, the magnitude is still too low, we have what is called an underflow, and the value is truncated to 0 with the minimum exponent and preserving sign, e.g. for `DECFLOAT(16)`, the value will become `+0E+398` or `-0E-398` (see note 19). Technically, this is just a special case of the previous step.

- If the magnitude (or exponent) is too high, then the following steps are applied:

  1. If the precision is less than maximum precision, and the difference between maximum precision and actual precision is larger than or equal to the difference between the actual exponent and the maximum exponent, then the precision is increased by adding zeroes as least-significant digits and decreasing the exponent by the number of zeroes added.

     An example: a `DECFLOAT(16)` stores values as an integral coefficient of 16 digits and an exponent between `-398` and `+369`. The value `1E+384` is coefficient 1 with exponent 384. This is too large for the maximum exponent, however, we have a value with a single digit, leaving us with 15 'unused' most-significant digits.

     If we multiply the coefficient by $10^{15}$ and subtract 15 from the exponent we get: coefficient = 1 * $10^{15}$ = 1000000000000000 and exponent = 384 - 15 = 369. And these values for coefficient and exponent are in range of the storage requirements.

     The resulting value is now `1000000000000000E+369` or `1.000000000000000E+384`, or in other words, we 'increased' precision by adding zeroes as least-significant digits to make the value fit.

  2. Otherwise, we have what is called an overflow, and an `SQLException` is thrown as the value is out of range.

If you need other rounding and overflow behavior, make sure you round the values appropriately before you set them.

### Configuring decfloat traps and rounding

To configure the server-side(!) error and rounding behaviour of the `DECFLOAT` data types, you can configure use the following connection properties:

- `decfloatRound` (alias: `decfloat_round`)

  Possible values: `ceiling`, `up`, `half_up` (default), `half_even`, `half_down`, `down`, `floor`, `reround`

- `decfloatTraps` (alias: `decfloat_traps`)

  Comma-separated list with options: `Division_by_zero` (default), `Inexact`, `Invalid_operation` (default), `Overflow` (default), `Underflow`

Configuring these options does not change driver behaviour, only server-side behaviour.

### Notes

1. `java.math.BigDecimal` is capable of representing numbers with larger precisions than `DECFLOAT`, and numbers that are out of range (too large or too small). When performing calculations in Java, use `MathContext.DECIMAL64` (for `DECFLOAT(16)`) or `MathContext.DECIMAL128` (for `DECFLOAT(34)`)

to achieve similar results in calculations as in Firebird. Be aware there might still be differences in rounding, and the result of calculations may be out of range.

1. Firebird 4.0 currently allows storing NaN and Infinity values, retrieval of these values will result in a `SQLException`, with a `DecimalInconvertibleException` cause with details on the special. The support for these special values is currently under discussion and may be removed in future Firebird versions.

2. `byte` in Java is signed, and historically Jaybird has preserved sign when storing byte values, and it considers values outside -128 and +127 out of range.
3. All integral values are — if within range — first converted to `long` using `BigDecimal.longValue()`, which discards any fractional parts (rounding by truncation).
4. When storing a `long` in `DECFLOAT(16)`, rounding will be applied using `RoundingMode.HALF_EVEN` for values larger than `9999999999999999L` or smaller than `-9999999999999999L`.
5. `float` values are first converted to (or from) double, this may lead to small rounding differences
6. `float` and `double` can be fully stored in `DECFLOAT(16)` and `DECLOAT(34)`, with minor rounding differences.
7. When reading `DECFLOAT` values as `double` or `float`, rounding will be applied as binary floating point types are inexact, and have a smaller precision.
8. If the magnitude of the `DECFLOAT` value is too great to be represented in `float` or `double`, +Infinity or -Infinity may be returned (see `BigDecimal.doubleValue()`). This behavior is subject to change, future releases may throw a `SQLException` instead, see also related note 9.
9. Storing and retrieving values NaN, +Infinity and -Infinity are currently supported, but this may change as this doesn't seem to be allowed by the SQL:2016 standard.

   It is possible that Jaybird or Firebird will disallow storing and retrieving NaN and Infinity values in future releases, causing Jaybird to throw an `SQLException` instead. We strongly suggest not to rely on this support for special values.

   1. Firebird `DECFLOAT` currently discerns four different NaNs (+/-NaN and +/-signaling-NaN). These are all mapped to `Double.NaN` (or `Float.NaN`), Java NaN values are mapped to +NaN in Firebird.

10. Setting `boolean` values will set `0` (or `0E+0`) for `false` and `1` (or `1E+0`) for `true`.
11. Retrieving as `boolean` will return `true` for `1` (exactly `1E+0`) and `false` for **all other values**. Be aware that this means that `1.0E+0` (or `10E-1`) etc will be **`false`**.

    This behavior may change in the future and only allow `0` for `false` and exactly `1` for `true` and throw an `SQLException` for all other values, or maybe `true` for everything other than `0`. In general, we advise to not use numerical types for boolean values, and especially not to retrieve the result of a calculation as a boolean value. Instead, use a real `BOOLEAN`.

12. Setting values as `String` is supported following the format rules of `new BigDecimal(String)`, with extra support for special values `+NaN`, `-NaN`, `+sNaN`, `-sNaN`, `+Infinity` and `-Infinity` (case-insensitive). Other non-numerical strings throw an `SQLException` with a `NumberFormatException` as cause. Out of range values are handled as described in Precision and range.
13. Getting values as `String` will be equivalent to `BigDecimal.toString()`, with extra support for the

special values mentioned in the previous note.

14. As mentioned in earlier notes, support for the special values is under discussion, and may change in future versions of Firebird and/or Jaybird.
15. Getting as `BigInteger` will behave as `BigDecimal.toBigInteger()`, which discards the fractional part (rounding by truncation), and may add (`-1 * scale - precision`) least-significant zeroes if the scale exceeds precision. Be aware that use of `BigInteger` for large values may result in significant memory consumption.
16. Setting as `BigInteger` will lose precision for values with more digits than the target type. It applies the rules described in Precision and range.
17. Values can also be set and retrieved as types `Decimal32`, `Decimal64` and `Decimal128` from the `org.firebirdsql.extern.decimal` package. Where `Decimal64` exactly matches the `DECFLOAT(16)` protocol format, and `Decimal128` the `DECFLOAT(34)` protocol format. Be aware that this is an implementation detail that might change in future Jaybird versions (both in terms of support for these types, and in terms of the interface (API) of these types).
18. Setting a `Decimal128` on a `DECFLOAT(16)`, or a `Decimal32` on a `DECFLOAT(16)` or `DECFLOAT(34)`, or retrieving a `Decimal32` from a `DECFLOAT(16)` or `DECFLOAT(34)`, or a `Decimal64` from a `DECFLOAT(34)` will apply the rules described in Precision and range.
19. Zero values can have a sign (e.g. `-0` vs `0` (`+0`)), this can only be set or retrieved using `String` or the `DecimalXX` types, or the result of rounding. This behaviour is subject to change, and future releases may 'round' to `0` (a.k.a. `+0`).

# 12.5. Exact numeric types DECIMAL/NUMERIC

The JDBC types `DECIMAL` and `NUMERIC` are supported by Firebird and Jaybird and map to `java.math.BigDecimal`.



Behaviour in Firebird of `NUMERIC` is closer to the SQL standard `DECIMAL` behaviour than the SQL standard `NUMERIC` behaviour. The precision specified is the minimum precision, not the exact precision.

## 12.5.1. Precision and range

In Firebird 3.0 and earlier, the maximum precision and scale of `DECIMAL` and `NUMERIC` is 18.[5]

In Firebird 4.0 and higher, the maximum precision and scale of `DECIMAL` and `NUMERIC` is 38. Any `NUMERIC` or `DECIMAL` with a precision between 19 and 38 will allow storage up to a precision of 38.[6]

In the implementation in Firebird, this extended precision is backed by an `INT128`.

Values set on a field or parameter will be rounded to the target scale of the field using `RoundingMode.HALF_EVEN`. Values exceeding a precision of 38 after rounding will be rejected with a `TypeConversionException`.

# 12.6. Type INT128

Firebird 4.0 introduced the non-standard type `INT128`, a signed 128-bit number.

## 12.6.1. Support for INT128

As JDBC does not define an `INT128` type, Jaybird maps the Firebird type `INT128` to the JDBC type `NUMERIC` with precision of 38. This way, most tools should be able to handle the full range of values without issue.

For details on numeric, see Exact numeric types DECIMAL/NUMERIC.

The introduction of this type is specified by jdp-2020-08 Int128 support [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2020-08-int128-support.md].

[1] This behaviour may change in a future version to the equivalent of `setLong((long) value)`

[2] This behaviour may change in a future version to the equivalent of `getLong(..) == 1L`

[3] This behaviour may change in a future version to use `compareTo` or the equivalent of `getLong(..) == 1L` instead

[4] The `DECFLOAT` decimal format stores values as sign, integral number with 16 or 34 digits, and an exponent. This is similar to `java.math.BigDecimal`, but instead of an exponent, that uses the concept `scale`, where `scale = -1 * exponent`.

[5] In practice, values with precision 19 are possible up to the maximum value of the `BIGINT` backing the value.

[6] In practice, values with precision 39 are possible up to the maximum value of the `INT128` backing the value.

# Appendices

# Appendix A: Extended connection properties

Jaybird has a number of connection properties that can be used to configure a connection.

This appendix provides a list of most connection properties and a short explanation to each of them. The properties listed below are usable as JDBC connection properties. Some properties have one or more aliases. The first name is the primary and preferred property name; it's usually also the name of the property in the data source classes.

The properties marked as *boolean property* can be included in the JDBC URL with values `true`, `false`, but also without a value, or with an empty value (which will both signify `true`). For readability, we suggest that you only specify these properties explicitly when you want to enable or disable them, and if you do, to use explicit values like `true` or `false`.

The default of *boolean* properties varies, but unless explicitly mentioned, the default is `false`.

A subset of these properties is also exposed in the `javax.sql.DataSource` implementations in Jaybird. When using data sources, unexposed properties can be set using `setNonStandardProperty`.

## A.1. Authentication and security properties

| Connection property (+ aliases) | Explanation |
|---|---|
| `user`<br>`userName`, `user_name`, `isc_dpb_user_name` | Name of the user for the connection. |
| `password`<br>`isc_dpb_password` | Password corresponding to the specified user. |
| `roleName`<br>`sqlRole`, `sql_role_name`, `isc_dpb_sql_role_name` | Name of the SQL role for the specified connection. |
| `authPlugins`<br>`auth_plugin_list` | Jaybird specific property. The comma-separated list of authentication plugins to try. See Authentication plugins for more information.<br>Default: `Srp256,Srp` |
| `wireCrypt` | Jaybird specific property. Allowed values: `DEFAULT`, `ENABLED`, `REQUIRED`, `DISABLED` (case-insensitive). Configures Firebird 3.0 and higher wire encryption behaviour. See Wire encryption support for more information.<br>Default: `DEFAULT`. |
| `dbCryptConfig` | Jaybird specific property. Configures Firebird 3.0 and higher database encryption support. See Database encryption support for more information. |
| `processId`<br>`process_id`, `isc_dpb_process_id` | Specifies the process id reported to Firebird. See Process information for more information. |

| Connection property (+ aliases) | Explanation |
|---|---|
| `processName`<br>`process_name`, `isc_dpb_process_name`, `ApplicationName` (***Since:*** *Jaybird 6*), `applicationName` (***Since:*** *Jaybird 6*) | Specifies the process name reported to Firebird. See Process information for more information. |
| `legacyAuthCharset`<br>`legacyAuthCharSet` | Jaybird specific property (***Since:*** *Jaybird 5.0.13/6.0.6*). The Java character set to use when hashing the password for Firebird 2.5 and older, or for the `Legacy_Auth` plugin.<br>Default: `UTF-8`. |

## A.2. Other properties

| Connection property (+ aliases) | Explanation |
|---|---|
| `asyncFetch` | Jaybird specific property (***Since:*** *Jaybird 6*). Boolean property. Enables/disables async fetching for pure Java connections. Default is `true`. See Async fetch for more information. |
| `encoding`<br>`lc_ctype`, `isc_dpb_lc_ctype` | Character encoding for the connection using the Firebird character set name. This property tells the database server the encoding in which it expects character content. For a list of the available encodings see Available Encodings.<br>In general, only specify `charSet` *or* `encoding`, not both.<br>Default: `NONE` (unless `charSet` is specified). |
| `charSet`<br>`localEncoding`, `charset` | Jaybird specific property. Character set for the connection using Java character set name. Similar to the previous property, but instead of Firebird-specific name accepts a Java character set name.<br>In general, only specify `charSet` *or* `encoding`, not both. |
| `searchPath`<br>`search_path`, `isc_dpb_search_path` | Schemas to search for unqualified objects (***Since:*** *Jaybird 7*, ***Since:*** *Firebird 6.0*)<br>Comma-separated list of schema names. For a case-sensitive or otherwise non-regular schema name, the identifier **must** be quoted. See Search path for more information. |
| `sqlDialect`<br>`dialect`, `sql_dialect`, `isc_dpb_sql_dialect` | SQL dialect, can be 1, 2 or 3.<br>Default: `3` |
| `defaultHoldable`<br>`defaultResultSetHoldable` | Jaybird specific property. Boolean property. Set result sets to be holdable by default. A workaround for applications with incorrect assumptions on result sets in auto-commit mode. See Default holdable result sets for more information. |

| Connection property (+ aliases) | Explanation |
|---|---|
| `useFirebirdAutocommit` | Jaybird specific property. Boolean property. Enable experimental feature to use Firebird auto-commit for JDBC auto-commit mode. **Do not use this unless you really know what you're doing.** See Firebird auto commit mode (experimental) for more information. |
| `generatedKeysEnabled` | Jaybird specific property. Configure generated keys support behaviour. See Configuring generated keys support for more information. |
| `isolation`<br>`defaultIsolation` | Jaybird specific property. Specify the default transaction isolation level. Accepted values are: `TRANSACTION_NONE`, `TRANSACTION_READ_UNCOMMITTED`, `TRANSACTION_READ_COMMITTED`, `TRANSACTION_REPEATABLE_READ`, `TRANSACTION_SERIALIZABLE` (case-sensitive). Although `TRANSACTION_NONE` and `TRANSACTION_READ_UNCOMMITTED` are allowed values, these behave the same as `TRANSACTION_READ_COMMITTED`.<br>Default: `TRANSACTION_READ_COMMITTED` |
| `socketBufferSize`<br>`socket_buffer_size` | Jaybird specific property. Size in bytes of the socket buffer. Only has effect on Type 4 (pure Java) connections. Can be used for performance tuning, or when the default socket buffer provided by the JVM or OS is not correct (e.g. too small). |
| `blobBufferSize`<br>`blob_buffer_size` | Jaybird specific property. Size in bytes of the buffer used to transfer BLOB content. Specifying a size greater than 65535 may result in additional server roundtrips per read, but may also waste memory for larger buffers than used in some cases. Values lower than 512 will use 512 (***Since:*** *Jaybird 6*).<br>Default: `16384` |
| `maxInlineBlobSize`<br>`max_inline_blob_size`, `isc_dpb_max_inline_blob_size` | Maximum size in bytes of inline blobs (***Since:*** *Jaybird 5.0.8/6.0.2* ***Since:*** *Firebird 5.0.3*). A value of `0` disables transmission of inline blobs. See Inline blobs for more information.<br>Default: `65535`, maximum value: `65535`[1] |
| `maxBlobCacheSize`<br>`max_blob_cache_size`, `isc_dpb_max_blob_cache_size` | Maximum size in bytes — per connection — of the inline blob cache (***Since:*** *Jaybird 5.0.8/6.0.2* ***Since:*** *Firebird 5.0.3*). A value of `0` disables the cache. Disabling the cache does not disable transmission of inline blobs: set `maxInlineBlobSize` to `0` to disable transmission. See Inline blobs for more information.<br>Default: `10485760` (10 MiB) |
| `soTimeout` | Jaybird specific property. Socket blocking timeout in milliseconds. Only has effect on Type 4 (pure Java) connections. |

| Connection property (+ aliases) | Explanation |
|---|---|
| `connectTimeout`<br>`connect_timeout`, `isc_dpb_connect_timeout` | Connect timeout in seconds. For Type 4 (pure Java) connections, the connect timeout will detect unreachable hosts. For Type 2 (native) connections, the connect timeout works as the DPB item `isc_dpb_connect_timeout` which only works after connecting to the server for the `op_accept` phase of the protocol. This means that for native connections the connect timeout will not detect unreachable hosts within the timeout. |
| `wireCompression` | Jaybird specific property. Boolean property. Enables zlib wire compression. See Wire compression for more information. |
| `columnLabelForName` | Jaybird specific property. Boolean property. Backwards-compatible behaviour of `getColumnName()` (return the column label instead of the column name). Don't set the property or set it to `false` for JDBC-compliant behaviour (recommended). |
| `useStreamBlobs`<br>`use_stream_blobs` | Jaybird specific property. Boolean property. Blobs are created as stream (`true`) or segmented (`false`) blobs.<br>Default: `true` |
| `ignoreProcedureType` | Jaybird specific property. Boolean property. Value `true` disables usage of procedure type metadata to decide to use `SELECT` for selectable procedure instead of `EXECUTE PROCEDURE`. See Connection property ignoreProcedureType for more information. |
| `scrollableCursor` | Jaybird specific property (***Since:*** *Firebird 5.0*). Allowed values: `EMULATED`, `SERVER` (case-insensitive) Configures use of server-side scrollable cursors (Firebird 5.0 and higher, pure Java connections only). See Connection property scrollableCursor for more information.<br>Default: `EMULATED` |
| `useServerBatch` | Jaybird specific property. Boolean property. Value `true` enables server-side batch if available (Firebird 4.0 and higher, pure Java connection only). Set to `false` to always use emulated batch updates. See Connection property useServerBatch for more information.<br>Default value: `true` |
| `serverBatchBufferSize` | Jaybird specific property. Size in bytes of the server-side batch buffer. Value `< 0` — server-side default, `0` — server-side maximum, `> 0` — size (capped at server-side maximum) See Connection property serverBatchBufferSize for more information.<br>Default value: `0` |

| Connection property (+ aliases) | Explanation |
|---|---|
| `dataTypeBind`<br>`set_bind` | Defines data type bind from one type to a different type. Multiple bind definitions are separated by semicolons. When used in a JDBC url, the semicolon must be escaped using `%3B`. See Data type bind support for more information. |
| `sessionTimeZone` | Configures the session time zone. This property also configures the time zone used for legacy datetime conversion on all Firebird versions. See Connection property sessionTimeZone for more information.<br>Default: the JVM default time zone |
| `decfloatRound`<br>`decfloat_round` | Defines server-side rounding behaviour for `DECFLOAT` data types Possible values: `ceiling`, `up`, `half_up` (default), `half_even`, `half_down`, `down`, `floor`, `reround` See Configuring decfloat traps and rounding for more information. |
| `decfloatTraps`<br>`decfloat_traps` | Defines server-side error behaviour for `DECFLOAT` data types Comma-separated list with options: `Division_by_zero` (default), `Inexact`, `Invalid_operation` (default), `Overflow` (default), `Underflow` See Configuring decfloat traps and rounding for more information. |
| `timestampUsesLocalTimezone` | Jaybird specific property (***Removed in:*** *Jaybird 6*). Boolean property. Changes how `getTime`/`getTimestamp` methods accepting a `java.util.Calendar` apply the calendar offset in calculations. |
| `num_buffers`<br>`isc_dpb_num_buffers` | Number of database pages that will be cached. Overrides server or database default for this specific connection if the server mode is Classic or SuperClassic. Use with care to avoid using an excessive amount of memory. |
| `set_db_readonly`<br>`isc_dpb_set_db_readonly` | Boolean property. Set the database into read-only state. |
| `set_db_sql_dialect`<br>`isc_dpb_set_db_sql_dialect` | Set the SQL dialect of the database. Use with care to avoid database issues. |
| `set_db_charset`<br>`isc_dpb_set_db_charset` | Set the default character set of the database. |
| `enableProtocol` | Jaybird specific property (***Since:*** *Jaybird 6*). Enable unsupported protocol versions, if available. Comma-separated list of protocol versions to enable in addition to the supported protocol versions. The value "*" will enable all available protocol versions. See Enabling unsupported protocol versions for more information. |
| `parallelWorkers`<br>`parallel_workers`,<br>`isc_dpb_parallel_workers` | The number of parallel workers to use (***Since:*** *Jaybird 5.0.2* ***Since:*** *Firebird 5.0*) |

| Connection property (+ aliases) | Explanation |
|---|---|
| `nativeLibraryPath` | Jaybird specific property (***Since:*** *Jaybird 6*).<br><br>Specifies a directory to try and load the client library for native (`fbclient`) or embedded (`fbembed` or `fbclient`) connections. If a file name is specified instead of a directory name, the parent directory will be used.<br><br>  <br>This property only works for the **first** native or embedded connection (to a database or service) created inside a JVM. Any subsequent connection uses the library that was loaded by the first connection. |
| `useCatalogAsPackage` | Jaybird specific property (***Since:*** *Jaybird 6*). Boolean property. Enables support for reporting database metadata information on packages, and functions and stored procedures in packages. See Opt-in feature for package information in DatabaseMetaData for more information. |
| `allowTxStmts` | Jaybird specific property (***Since:*** *Jaybird 6*). Boolean property. Enables support for executing `COMMIT [WORK]`, `ROLLBACK [WORK]` and `SET TRANSACTION`. Default is `false`. See Connection property allowTxStmts for more information. |
| `createDatabaseIfNotExist` | Jaybird specific property (***Since:*** *Jaybird 6*). Boolean property. Attempts to create a database if it does not exist. Default is `false`. See Create database if it does not exist for more information. |
| `escapeProcessing` | JDBC standard property (***Since:*** *Jaybird 7*). Boolean property. Sets the default escape processing behaviour of `Statement` objects of a connection. Default is `true`. |
| `extendedMetadata` | Jaybird specific property (***Since:*** *Jaybird 5.0.5*). Boolean property. Enables support for retrieving extended metadata. Default is `true`. See Connection property extendedMetadata for more information. |
| `reportSQLWarnings` | Jaybird specific property (***Since:*** *Jaybird 6*). Possible values (case-insensitive): `ALL` (default), `NONE`. Can be used to disable reporting of `SQLWarnings`. See Reporting of `SQLWarnings` for more information. |
| `socketFactory` | Jaybird specific property (***Since:*** *Jaybird 6*). Sets a custom socket factory for pure Java connections. See Custom socket factory for more information. |

In addition, Jaybird allows using arbitrary Database Parameters Block entries as connection properties (provided they are defined in Jaybird's `DpbItems` and `SpbItems`). The current Firebird API has over 100 DPB parameters, however only few of them are interesting for regular users. If a DPB

item called `isc_dpb_XXX` exists, then Jaybird allows these to be specified as `isc_dpb_XXX` and `XXX`. By default properties are mapped as string DPB items. If a DPB item requires another type, it will need to be explicitly defined in Jaybird to work correctly.

## A.3. Transaction isolation levels

It is possible to redefine the transaction isolation levels through connection properties.

| Connection property | Explanation |
|---|---|
| `TRANSACTION_READ_COMMITTED` | Specify the definition of transaction isolation level `READ_COMMITTED`.<br>Default: `isc_tpb_read_committed,isc_tpb_rec_version,isc_tpb_write,isc_tpb_wait` |
| `TRANSACTION_REPEATABLE_READ` | Specify the definition of transaction isolation level `REPEATABLE_READ`.<br>Default: `isc_tpb_concurrency,isc_tpb_write,isc_tpb_wait` |
| `TRANSACTION_SERIALIZABLE` | Specify the definition of transaction isolation level `TRANSACTION_SERIALIZABLE`.<br>Default: `isc_tpb_consistency,isc_tpb_write,isc_tpb_wait` |

For data sources, this feature is exposed using a definition properties file and the `setTpbMapping` property. See Transaction Isolation Levels for more information.

[1] The maximum is decided server-side, and may change in future Firebird versions.

# Appendix B: System properties

Jaybird provides a number of system properties to control global behaviour of Jaybird.

## B.1. Logging

***Removed in:*** *Jaybird 6*

To configure logging, the following system properties are available. See Logging for details.

`org.firebirdsql.jdbc.forceConsoleLogger`
: Set to true to force logging to console (`System.out` for info, `System.err` for warn, error and fatal) instead of default `java.util.logging`

`org.firebirdsql.jdbc.disableLogging`
: Set to true to disable logging

`org.firebirdsql.jdbc.loggerImplementation`
: Fully-qualified name of `org.firebirdsql.logging.Logger` implementation to use for logging

These properties need to be set before Jaybird is loaded and used.



These properties are not available in Jaybird 6 or later.

## B.2. Process information

For Firebird 2.1 and higher, Jaybird can provide Firebird with process information. This information can be specified in connection properties, or globally using the following system properties.

`org.firebirdsql.jdbc.processName`
: Process name to send to Firebird

`org.firebirdsql.jdbc.pid`
: PID to send to Firebird (must be a valid integer)

  Ignored for native and embedded connections

The property values are read for each connect, so the value can be changed at any time.

See also Process information.

## B.3. Character set defaults

The following system properties control character set behaviour for connections.

`org.firebirdsql.jdbc.defaultConnectionEncoding`

Firebird character set name to use as connection character set when no explicit connection character set is configured (defaults to `NONE` when not set)

`org.firebirdsql.jdbc.requireConnectionEncoding`

Set to true to disallow connections without an explicit connection character set. This property will have no effect if `org.firebirdsql.jdbc.defaultConnectionEncoding` has been set.

The property values are read for each connect, so the value can be changed at any time.

## B.4. Default `asyncFetch` value

***Since:*** *Jaybird 6*

`org.firebirdsql.jdbc.defaultAsyncFetch`

Configures the default value for the `asyncFetch` connection property. See Async fetch for more information.

## B.5. Default `enableProtocol` value

***Since:*** *Jaybird 6*

`org.firebirdsql.jdbc.defaultEnableProtocol`

Configures the default value for the `enableProtocol` connection property. See Enabling unsupported protocol versions for more information.

## B.6. Default inline blob property values

***Since:*** *Jaybird 5.0.8/6.0.2*
***Since:*** *Firebird 5.0.3*

`org.firebirdsql.jdbc.defaultMaxInlineBlobSize`

Configures the default value for the `maxInlineBlobSize` connection property.

`org.firebirdsql.jdbc.defaultMaxBlobCacheSize`

Configures the default value for the `maxBlobCacheSize` connection property.

See also Inline blobs.

## B.7. Default `reportSQLWarnings` value

***Since:*** *Jaybird 6*

`org.firebirdsql.jdbc.defaultReportSQLWarnings`

Configures the default value for the `reportSQLWarnings` connection property. See Reporting of `SQLWarnings` for more information.

## B.8. Wire protocol buffer sizes

***Since:*** *Jaybird 6*

The following system properties control the sizes of a number of buffers used by Jaybird's wire protocol implementation.

`org.firebirdsql.wire.deflateBufferSize`

Buffer size in bytes for deflate (compression). Has a minimum size of 512 and a default value of 8192.

`org.firebirdsql.wire.inflateBufferSize`

Buffer size in bytes for inflate (decompression). Has a minimum size of 512 and a default value of 8192.

`org.firebirdsql.wire.decryptBufferSize`

Buffer size in bytes for decryption. Has a minimum size of 512 and a default value 8192.



As the decryption input buffer determines the size of the decryption output buffer, the actual allocation is up to twice the specified value.

`org.firebirdsql.wire.inputBufferSize`

Buffer size in bytes for reading data from the socket. Has a minimum size of 1024 and a default value of 16384.



This property should not be confused with the socket buffer size (configurable with the `socketBufferSize` *connection property*). Setting this value higher than `socketBufferSize` is unlikely to have any benefits.

`org.firebirdsql.wire.outputBufferSize`

Buffer size in bytes for writing data. Has a minimum size of 1024 and a default value of 32767.



This property should not be confused with the socket buffer size (configurable with the `socketBufferSize` *connection property*). Contrary to the input buffer size, setting it higher than `socketBufferSize` might have performance benefits.

These properties need to be set before Jaybird is loaded and used.



The minimum sizes and default values should be considered an implementation detail and might change in a future major version.

The "deflate" and "inflate" buffers are only used when wire compression is enabled. The "decrypt" buffer is only applied when wire encryption is used (the default with Firebird 3.0 and higher).

The order of buffers for input is as follows:

```
Jaybird ⇐ [inflate ⇐] [decrypt ⇐] input ⇐ socket
```

The order of buffers for output is as follows:

```
Jaybird ⇒ output [⇒ deflate] ⇒ socket
```

## B.9. Close connection on fatal error

***Since:*** *Jaybird 5.0.13/6.0.6*

The following system properties configure if Jaybird forcibly closes connections on fatal errors (default) or not.

`org.firebirdsql.jdbc.connection.forceCloseOnFatal`

Boolean property. Configures force close behaviour of "normal" JDBC connections (i.e. created with `org.firebirdsql.jaybird.xca.FBStandAloneConnectionManager`); defaults to `true`.

`org.firebirdsql.ds.xa.connection.forceCloseOnFatal`

Boolean property. Configures force close behaviour of (XA) connections created by `org.firebirdsql.ds.FBXADataSource`; defaults to `true`.

These system properties are checked *dynamically*, so they can be changed at run time.

These properties should only be used for debugging or workarounds. Correct behaviour of a connection after a fatal error is ignored is not guaranteed.

See also jdp-2026-04: Disable closing of broken connections [https://github.com/FirebirdSQL/jaybird/blob/master/devdoc/jdp/jdp-2026-04-disable-closing-of-broken-connections.adoc].

## B.10. Other properties

The following system properties control other global behaviour of Jaybird.

`org.firebirdsql.jna.syncWrapNativeLibrary`

Set to true to add a synchronization proxy around the native client library.

This can be used to address thread-safety issues with older client libraries (Firebird 2.1 and older, as far as we know).

`org.firebirdsql.datatypeCoderCacheSize`

Integer value for the number of encoding specific data type coders cached (default and minimum is 1). Setting to a higher value may improve performance, most common use case is connection character set `NONE` with a database that uses more than one character set for its columns. Jaybird will log a warning (*"Cleared encoding specific datatype coder cache [..]"*) when the cache size was exceeded.

`org.firebirdsql.nativeResourceShutdownDisabled`

Set to true to disable automatic shutdown and unload of native libraries and other native resources. Normally you should only use this if the automatic shutdown misbehaves and causes application errors. If you need to set this to true, we'd appreciate it if you post a message to the firebird-java Google Group [https://groups.google.com/g/firebird-java] with details on why you needed to enable this, so we can improve or fix this feature.

These properties need to be set before Jaybird is loaded and used. Technically, `org.firebirdsql.jna.syncWrapNativeLibrary` is dynamic, but a native library will usually be loaded once.

## B.11. Useful Java system properties

The following Java system properties are relevant for Jaybird.

`jdk.net.useFastTcpLoopback`

***Since:*** *Firebird 3.0.2* ***Removed in:*** *Firebird 5.0* ***Since:*** *Jave 8 update 60* ***Since:*** *Windows 8 / Windows Server 2012* Set to true on Windows to enable "TCP Loopback Fast Path" (`SIO_LOOPBACK_FAST_PATH` socket option). "TCP Loopback Fast Path" can improve performance for localhost connections.
Java only has an 'all-or-nothing' support for the "TCP Loopback Fast Path", so Jaybird cannot enable this for you: you must specify this property on JVM startup. This has the benefit that this works for all Jaybird versions, as long as you use Java 8 update 60 or higher (and Firebird 3.0.2 or higher).



Microsoft has deprecated the `SIO_LOOPBACK_FAST_PATH` and recommends not to use it.

Support was removed in Firebird 5.0.

# Appendix C: Data Type Conversion Table

## C.1. Mapping between JDBC, Firebird and Java Types

The below table describes a mapping of the JDBC data types defined in `java.sql.Types` class to the Firebird data types. Also, for each JDBC data type a class instance of which is returned by `ResultSet.getObject` method is provided.

| JDBC Type | Firebird Type | Java Object Type |
|---|---|---|
| CHAR | CHAR | String |
| VARCHAR | VARCHAR | String |
| LONGVARCHAR | BLOB SUB_TYPE TEXT | String |
| NUMERIC | NUMERIC<br>INT128 | java.math.BigDecimal |
| DECIMAL | DECIMAL | java.math.BigDecimal |
| SMALLINT | SMALLINT | Integer |
| INTEGER | INTEGER | Integer |
| BIGINT | BIGINT | Long |
| REAL | [1] | |
| FLOAT | FLOAT | Double |
| DOUBLE | DOUBLE PRECISION | Double |
| BINARY | CHAR CHARACTER SET OCTETS<br>BINARY | byte[] |
| VARBINARY | VARCHAR CHARACTER SET OCTETS<br>VARBINARY | byte[] |
| LONGVARBINARY | BLOB SUB_TYPE BINARY | byte[] |
| DATE | DATE | java.sql.Date |
| TIME | TIME | java.sql.Time |
| TIME_WITH_TIMEZONE | TIME WITH TIME ZONE | java.time.OffsetTime |
| TIMESTAMP | TIMESTAMP | java.sql.Timestamp |
| TIMESTAMP_WITH_TIMEZONE | TIMESTAMP WITH TIME ZONE | java.time.OffsetDateTime |
| BLOB | BLOB SUB_TYPE < 0 | java.sql.Blob |
| BOOLEAN | BOOLEAN | Boolean |
| DECFLOAT (***Since:*** *JDBC 4.5/Java 26*)<br>JaybirdTypeCodes.DECFLOAT[2] | DECFLOAT | java.math.BigDecimal |

## C.2. Data Type Conversions

This table specifies the compatible conversions between the Firebird and Java types.

| | SMALLINT | INTEGER | BIGINT | REAL | FLOAT | DOUBLE | DECIMAL | NUMERIC | CHAR | VARCHAR | BLOB SUB_TYPE 1 | BLOB SUB_TYPE 0 | BLOB SUB_TYPE < 0 | DATE | TIME | TIMESTAMP | BOOLEAN | DECFLOAT |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| String | X | X | X | X | X | X | X | X | X | X | X | X | X | X | X | X | X | X |
| BigDecimal | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| Boolean | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| Integer | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| Long | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| Float | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| Double | X | X | X | X | X | X | X | X | X | X | | | | | | | X | X |
| byte[] | | | | | | | | | | | X | X | X | | | | | |
| Blob | | | | | | | | | | | X | X | X | | | | | |
| Date | | | | | | | | | | | | | | X | | X | | |
| Time | | | | | | | | | | | | | | | X | | | |
| Timestamp | | | | | | | | | | | | | | X | | X | | |

[1] A Firebird `REAL` is an alias for `FLOAT` and by default handled as `java.sql.Types.FLOAT`

[2] JDBC 4.4 and earlier do not yet define a `java.sql.Types` code for `DECFLOAT`

# Appendix D: Character Encodings

Character encodings and their correct use with Firebird from Java is an important topic, that initially seems to be complex, but in fact can be formulated by just a few rules. This appendix provides information on this topic.

## D.1. Encodings Types

Firebird uses character encodings in two different areas:

- The database encoding defines the character set in which `CHAR`, `VARCHAR` and `BLOB SUB_TYPE TEXT` fields are physically stored on the disk. There is a default database encoding that is specified during database creation. It is also possible to specify character sets on a per-column basis.
- The client connection encoding defines a character set in which client will send and expects to receive character data. This encoding might or might not match the database default encoding.

Firebird performs translation between character sets of the client connection and the character set of the content. The list of allowed character sets as well as the allowed translations between them are specified in the `fbintl` shared library located in the `intl/` directory of the Firebird installation.[1] There is also a special character set NONE that tells Firebird not to interpret the contents of the character field.

Firebird uses the following algorithm when performing translations:

- If source and target character sets match, send the content unchanged.
- If the target character set is `NONE`, send source content unchanged.
- If the source character set is `NONE`, send source content unchanged.
- If there is a direct translation rule between source and target character sets, use that rule.
- If there is no direct translation rule, check if there is rule to translate the source character set into the UTF8 character set and a rule to translate from `UTF8` into the target character set. If yes, use these two rules for translation.
- If no suitable translation rule can be found, throw an exception.

## D.2. Encodings in Java

The Java programming language is based on the Unicode character set and uses the UTF-16 encoding, in which each character is represented by one or two 16-bit units. Firebird, on the other side, is not based on Unicode and allows different character sets to be assigned to different database objects. Additionally, Firebird requires a connection character set to be specified in connection options, which forces Firebird to convert data from the character set of the database object into the character set of the client application.

There are two boundary cases that we will consider here, one when Firebird database was created with default[2] character set `UTF8`, another when the Firebird database was created without specifying the character set (i.e. character set `NONE`).

The character set UTF8 is a Unicode character set that uses UTF-8 encoding and occupies from one to four 8-bit units. Firebird has supported Unicode character set for a long time, however its implementation was deficient in Firebird 1.5 and earlier — it did not support proper uppercasing and correct sorting. These issues were addressed in Firebird 2.0 and at the moment nothing prevents developers from using Unicode in the database and on the client side, which greatly simplifies the internationalization and localization of the applications.

### D.2.1. The UTF8 character set

A developer must ensure two things to enable use of Unicode characters in the database and the application:

1. The database objects must be defined with the UTF8 character set; this can be done by either creating the database with a default character set of UTF8, or by adding the `CHARACTER SET UTF8` clause to column or domain definitions.
2. The `encoding` connection property in the JDBC driver has to be set to UTF8; this can be done in several ways: the easiest one is to add the appropriate parameter to the JDBC URL (see the first example), another possibility is to use appropriate method of the `DriverManager` class (see the second example). Applications that use `DataSource` interface to obtain the database connections also have access to the `encoding` property.[3]

*Specifying the connection encoding in JDBC URL*

```
Connection connection = DriverManager.getConnection(
    "jdbc:firebirdsql:localhost/3050:employee?encoding=UTF8",
    "SYSDBA", "masterkey");
```

*Specifying connection encoding in the connection properties*

```
Properties props = new Properties();

props.setProperty("user", "SYSDBA");
props.setProperty("password", "masterkey");
props.setProperty("encoding", "UTF8");

Connection connection = DriverManager.getConnection(
    "jdbc:firebirdsql:localhost/3050:employee", props);
```

There are a few limitations related to using the UTF8 character set:

- It is not possible to create Unicode columns longer than 8191 Unicode characters; this limitation is caused by the fact that the longest possible `VARCHAR` column can occupy 32765 bytes (32767 for `CHAR` columns) and a single UTF8 character can occupy up to four bytes.
- It is not possible to index Unicode columns longer than 2047 characters — this requires a page size of 32 kilobytes; this limitation is caused by the fact that the longest index key cannot be longer than a quarter of the database page, which has a maximum of 32k and the fact that each UTF8 character can occupy up to four bytes.

Using the UTF8 character set might cause noticeable performance degradation when the database is accessed over wide-area networks. This mainly applies to the cases when non-latin characters are stored in the database, as those characters will require two or more bytes, which in turn might cause additional roundtrips to the server to fetch data.

### D.2.2. The NONE character set

Java introduces additional complexity when the NONE character set is used. The reason for this is that Java internally stores all strings in Unicode format, and the application must define the character encoding for the byte contents to the JVM. When the NONE character set is used, Jaybird does not know how to interpret the received data. The only choice that is left to Jaybird is to construct a string using the default character set of the JVM, which usually matches the regional settings of the operating system and can be accessed from within the JVM through the `file.encoding` system property.

With connection character set NONE, Jaybird[4] uses the explicit character set of CHAR, VARCHAR and BLOB SUB_TYPE TEXT columns for the conversion. This addresses most of the problems described in this section, except for columns without an explicit character set (i.e. their character set is NONE).

A conversion using the default character set of the JVM can lead to errors when the same content is accessed from two or more Java Virtual Machines that have different configuration. One application running on a computer with, for example, Russian regional settings saves Russian text (default character set of the JVM is Cp1251), and another application running on a computer with German regional settings (default character set is Cp1252) will read in such case some special or accented characters. However, when all client applications run on the same OS with the same regional settings, in most cases this will not have any severe consequences, except maybe wrong sorting order or uppercasing on the server side.

On Linux and other Unix platforms, it might have more severe consequences as it is very common that regional settings are not configured and that the default "C" locale is used and the non-ASCII characters will be replaced with question marks ("?").

Since Java 18 (JEP 400 [https://openjdk.org/jeps/400]), this is less of a concern as the default is now always UTF-8. On the other hand, this can cause unexpected behaviour when upgrading from older Java versions.

Therefore, an application should only use NONE character encoding as an encoding for a database and a connection when at least one of the following is met:

- The database will contain only ASCII characters,
- It is guaranteed that all Java Virtual Machines accessing the database will have the same default encoding that can correctly handle all characters stored in the database,
- All columns have an explicit character set. When columns have an explicit character set (other than NONE) and connection character set NONE is used, Firebird will send an identifier of the character set of each column, and Jaybird will use that character set for the conversion.

As a partial workaround, you can specify the encoding that should be used to interpret bytes coming from the server in the `charSet` connection property. The following rules are used when interpreting the `encoding` and `charSet` properties:

- When only `encoding` property is specified, Jaybird uses the default mapping between server and Java encodings. When `encoding` property is not set or set to `NONE` and `charSet` property is not set, the default JVM encoding is used to interpret bytes coming from the server.
- When only `charSet` property is specified, Jaybird uses the reverse mapping to specify the connection encoding for the server and interprets byte stream according to the value of the property.
- When both `encoding` and `charSet` property are specified, Jaybird sets the connection encoding according to the value of the `encoding` property, but interprets the byte stream according to the `charSet` property. With Jaybird 3 and higher, this option has limitations when `encoding=NONE`: the conversion using `charSet` will only be applied for columns that don't have an explicit character set, otherwise that explicit character set is used for the conversion.

The last case is most powerful, but also is the most dangerous in use. When used properly, it can solve problems with legacy databases; when used incorrectly, one can easily trash the content of the database.

## D.3. Available Encodings

The below table lists the available character encodings in the default Firebird distribution and their mapping to the Java ones:

| **Firebird encoding (`encoding` property)** | **Java encoding (`charSet` property)** | **Size in bytes** | **Comments** |
|---|---|---|---|
| NONE | | 1 | Raw bytes, no interpretation of the content is possible. |
| ASCII | ASCII | 1 | |
| BIG_5 | Big5 | 2 | Traditional Chinese |
| DOS437 | Cp437 | 1 | MS-DOS: United States, Australia, New Zeland, South Africa |
| DOS737 | Cp737 | 1 | MS-DOS: Greek |
| DOS775 | Cp775 | 1 | MS-DOS: Baltic |
| DOS850 | Cp850 | 1 | MS-DOS: Latin-1 |
| DOS852 | Cp852 | 1 | MS-DOS: Latin-2 |
| DOS857 | Cp857 | 1 | IBM: Turkish |
| DOS858 | Cp858 | 1 | IBM: Latin-1 + Euro |
| DOS860 | Cp860 | 1 | MS-DOS: Portuguese |
| DOS861 | Cp861 | 1 | MS-DOS: Icelandic |
| DOS862 | Cp862 | 1 | IBM: Hebrew |
| DOS863 | Cp863 | 1 | MS-DOS: Canadian French |

| **Firebird encoding (`encoding` property)** | **Java encoding (`charSet` property)** | **Size in bytes** | **Comments** |
|---|---|---|---|
| DOS864 | Cp864 | 1 | IBM: Arabic |
| DOS865 | Cp865 | 1 | MS-DOS: Nordic |
| DOS866 | Cp866 | 1 | IBM: Cyrillic |
| DOS869 | Cp869 | 1 | IBM: Modern Greek |
| EUCJ_0208 | EUC_JP | 2 | JIS X 0201, 0208, 0212, EUC encoding, Japanese |
| GB_2312 | EUC_CN | 2 | GB2312, EUC encoding, Simplified Chinese |
| ISO8859_1 | ISO-8859-1 | 1 | ISO 8859-1, Latin alphabet No. 1 |
| ISO8859_2 | ISO-8859-2 | 1 | ISO 8859-2 |
| ISO8859_3 | ISO-8859-3 | 1 | ISO 8859-3 |
| ISO8859_4 | ISO-8859-4 | 1 | ISO 8859-4 |
| ISO8859_5 | ISO-8859-5 | 1 | ISO 8859-5 |
| ISO8859_6 | ISO-8859-6 | 1 | ISO 8859-6 |
| ISO8859_7 | ISO-8859-7 | 1 | ISO 8859-7 |
| ISO8859_8 | ISO-8859-8 | 1 | ISO 8859-8 |
| ISO8859_9 | ISO-8859-9 | 1 | ISO 8859-9 |
| ISO8859_13 | ISO-8859-13 | 1 | ISO 8859-13 |
| KSC_5601 | MS949 | 2 | Windows Korean |
| UNICODE_FSS | UTF-8 | 3 | 8-bit Unicode Transformation Format (deprecated since Firebird 2.0) |
| UTF8 | UTF-8 | 4 | 8-bit Unicode Transformation Format (Firebird 2.0+) |
| WIN1250 | Cp1250 | 1 | Windows Eastern European |
| WIN1251 | Cp1251 | 1 | Windows Cyrillic |
| WIN1252 | Cp1252 | 1 | Windows Latin-1 |
| WIN1253 | Cp1253 | 1 | Windows Greek |
| WIN1254 | Cp1254 | 1 | Windows Turkish |
| WIN1255 | Cp1255 | 1 | Windows Hebrew |
| WIN1256 | Cp1256 | 1 | Windows Arabic |
| WIN1257 | Cp1257 | 1 | Windows Baltic |

[1] On Windows this library is represented by `fbintl.dll`, on Linux, `libfbintl.so`

[2] The default character set simplifies the explanation, since we do not have to consider the cases when different columns with different character sets are used within the same connection. The statements made here, obviously, can be applied to those cases as well.

[3] See https://github.com/FirebirdSQL/jaybird/wiki for configuration examples of the most popular application servers.

[4] Since Jaybird 3

# Appendix E: Supported JDBC Scalar Functions

The JDBC Specification defines an escape syntax for numeric, string, time, date, system and conversion functions. Jaybird will try to provide an equivalent of the JDBC function using the built-in capabilities of Firebird. When no equivalent is available, Jaybird will pass the function call "as is" assuming that the database contains the necessary UDF, UDR or stored function declaration.

Below you will find the list of JDBC functions and if they are supported by Jaybird.

## E.1. Numeric Functions

| JDBC | built-in | Description |
|---|---|---|
| `ABS(number)` | X | Absolute value of `number` |
| `ACOS(float)` | X | Arccosine, in radians, of `float` |
| `ASIN(float)` | X | Arcsine, in radians, of `float` |
| `ATAN(float)` | X | Arctangent, in radians, of `float` |
| `ATAN2(float1, float2)` | X | Arctangent, in radians, of `float2` / `float1` |
| `CEILING(number)` | X | Smallest integer >= `number` |
| `COS(float)` | X | Cosine of `float` radians |
| `COT(float)` | X | Cotangent of `float` radians |
| `DEGREES(number)` | X | Degrees in `number` radians |
| `EXP(float)` | X | Exponential function of `float` |
| `FLOOR(number)` | X | Largest integer <= `number` |
| `LOG(float)` | X | Base e logarithm of `float` |
| `LOG10(float)` | X | Base 10 logarithm of `float` |
| `MOD(integer1, integer2)` | X | Remainder for `integer1` / `integer2` |
| `PI()` | X | The constant pi |
| `POWER(number, power)` | X | `number` raised to (integer) `power` |
| `RADIANS(number)` | X | Radians in `number` degrees |
| `RAND(integer)` | | Random floating point for seed `integer` |
| `ROUND(number, places)` | X | `number` rounded to `places` places |
| `SIGN(number)` | X | -1 to indicate `number` is < 0; 0 to indicate `number` is = 0; 1 to indicate `number` is > 0 |
| `SIN(float)` | X | Sine of `float` radians |
| `SQRT(float)` | X | Square root of float |
| `TAN(float)` | X | Tangent of `float` radians |
| `TRUNCATE(number, places)` | X | `number` truncated to `places` places |

*Legend*

**X** available

## E.2. String Functions

| JDBC | built-in | Description |
|---|---|---|
| `ASCII(string)` | X | Integer representing the ASCII code value of the leftmost character in `string` |
| `CHAR(code)` | X | Character with ASCII code value `code`, where `code` is between 0 and 255 |
| `CHAR_LENGTH(string [,CHARACTERS])` | X | Returns the length in characters of the string expression |
| `CHAR_LENGTH(string, OCTETS)` | X | Returns the length in bytes of the string expression whose result is the smallest integer not less than the number of bits divided by 8, alias for `OCTET_LENGTH` |
| `CHARACTER_LENGTH( string [,CHARACTERS])` | X | Alias for `CHAR_LENGTH` |
| `CHARACTER_LENGTH(string, OCTETS)` | X | Alias for `CHAR_LENGTH`, `OCTET_LENGTH` |
| `CONCAT(string1, string2)` | X | Character string formed by appending `string2` to `string1` |
| `DIFFERENCE(string1, string2)` | | Integer indicating the difference between the values returned by the function `SOUNDEX` for `string1` and `string2` |
| `INSERT(string1, start, length, string2)` | X | A character string formed by deleting `length` characters from `string1` beginning at `start`, and inserting `string2` into `string1` at `start` |
| `LCASE(string)` | X | Converts all uppercase characters in `string` to lowercase |
| `LEFT(string, count)` | X | The `count` leftmost characters from `string` |
| `LENGTH(string [,CHARACTERS])` | X[1] | Number of characters in `string`, excluding trailing blanks |
| `LENGTH(string, OCTETS])` | X[2] | Number of characters in `string`, excluding trailing blanks |
| `LOCATE(string1, string2 [,start])` | X | Position in `string2` of the first occurrence of `string1`, searching from the beginning of `string2`; if `start` is specified, the search begins from position `start`. 0 is returned if `string2` does not contain `string1`. Position 1 is the first character in `string2` |
| `LTRIM(string)` | X | Characters of `string` with leading blank spaces removed |

| JDBC | built-in | Description |
|---|---|---|
| `OCTET_LENGTH(string)` | X | Returns the length in bytes of the string expression whose result is the smallest integer not less than the number of bits divided by 8 |
| `POSITION(substring IN string [,CHARACTERS])` | X | Returns the position of first occurrence of `substr` in `string` (with optional `CHARACTERS`) |
| `POSITION(substring IN string, OCTETS)` | X[3] | Returns the position of first occurrence of `substr` in `string` (with caveat) |
| `REPEAT(string, count)` | X | A character string formed by repeating `string` `count` times |
| `REPLACE(string1, string2, string3)` | X | Replaces all occurrences of `string2` in `string1` with `string3` |
| `RIGHT(string, count)` | X | The `count` rightmost characters in `string` |
| `RTRIM(string)` | X | The characters of `string` with no trailing blanks |
| `SOUNDEX(string)` | | A character string, which is data source-dependent, representing the sound of the words in `string`; this could be a four-digit SOUNDEX code, a phonetic representation of each word, etc |
| `SPACE(count)` | X | A character string consisting of `count` spaces |
| `SUBSTRING(string, start, length)` | X | A character string formed by extracting `length` characters from `string` beginning at `start` |
| `UCASE(string)` | X | Converts all lowercase characters in `string` to uppercase |

*Legend*

**X** available

## E.3. Time and Date Functions

| JDBC | built-in | Description |
|---|---|---|
| `CURRENT_DATE[()]` | X | Synonym for `CURDATE()` |
| `CURRENT_TIME[()]` | X | Synonym for `CURTIME()` |
| `CURRENT_TIMESTAMP[()]` | X | Synonym for `NOW()` |
| `CURDATE()` | X | The current date as a date value |
| `CURTIME()` | X | The current local time as a time value |
| `DAYNAME(date)` | X[4] | A character string representing the day component of `date`; the name for the day is specific to the data source |
| `DAYOFMONTH(date)` | X | An integer from 1 to 31 representing the day of the month in `date` |

| JDBC | built-in | Description |
|---|---|---|
| DAYOFWEEK(date) | X | An integer from 1 to 7 representing the day of the week in date; 1 represents Sunday |
| DAYOFYEAR(date) | X | An integer from 1 to 366 representing the day of the year in date |
| EXTRACT(field FROM source) | X | Extract the field portion from the source. The source is a datetime value. The value for field may be one of the following: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND |
| HOUR(time) | X | An integer from 0 to 23 representing the hour component of time |
| MINUTE(time) | X | An integer from 0 to 59 representing the minute component of time |
| MONTH(date) | X | An integer from 1 to 12 representing the month component of date |
| MONTHNAME(date) | X[5] | A character string representing the month component of date; the name for the month is specific to the data source |
| NOW() | X | A timestamp value representing the current date and time |
| QUARTER(date) | X | An integer from 1 to 4 representing the quarter in date; 1 represents January 1 through March 31 |
| SECOND(time) | X | An integer from 0 to 59 representing the second component of time |
| TIMESTAMPADD( interval, count, timestamp) | X | A timestamp calculated by adding count number of interval(s) to timestamp |
| TIMESTAMPDIFF( interval, timestamp1, timestamp2) | X | An integer representing the number of interval by which timestamp2 is greater than timestamp1 |
| WEEK(date) | X | An integer from 1 to 53 representing the week of the year in date |
| YEAR(date) | X | An integer representing the year component of date |

*Legend*

**X** available

## E.4. System Functions

| JDBC | built-in | Description |
|---|---|---|
| DATABASE() | X[6] | Name of the database |
| IFNULL(expression, value) | X | value if expression is null; expression if expression is not null |

| JDBC | built-in | Description |
|---|---|---|
| `USER()` | X | Username in the DBMS |

*Legend*

**X** available

## E.5. Conversion Functions

| JDBC | built-in | Description |
|---|---|---|
| `CONVERT(value, SQLtype)` | X | `value` converted to `SQLtype` where `SQLtype` may be one of the following SQL types:<br>• `BIGINT`<br>• `BINARY`<br>• `BLOB`<br>• `CHAR`<br>• `CLOB`<br>• `DATE`<br>• `DECFLOAT`<br>• `DECIMAL`<br>• `DOUBLE`<br>• `DOUBLE PRECISION`<br>• `FLOAT`<br>• `INTEGER`<br>• `LONGNVARCHAR`<br>• `LONGVARBINARY`<br>• `LONGVARCHAR`<br>• `NCHAR`<br>• `NCLOB`<br>• `NVARCHAR`<br>• `REAL`<br>• `SMALLINT`<br>• `TIME`<br>• `TIME_WITH_TIMEZONE`<br>• `TIME_WITH_TIME_ZONE`<br>• `TIMESTAMP`<br>• `TIMESTAMP_WITH_TIMEZONE`<br>• `TIMESTAMP_WITH_TIME_ZONE`<br>• `TINYINT`<br>• `VARBINARY`<br>• `VARCHAR`<br>These type names can also be prefixed with `SQL_`. |

*Legend*

**X** available

The `CONVERT` escape provides some additional features, or deviates from the JDBC defined behaviour:

- Contrary to the JDBC specification, we allow explicit length or precision and scale parameters
- `(SQL_)VARCHAR`, `(SQL_)NVARCHAR` (and *value* not a parameter (?)) without explicit length is converted using `TRIM(TRAILING FROM value)`, which means the result is `VARCHAR` except for blobs where this will result in a blob; national character set will be lost. If *value* is a parameter (?), and no length is specified, then a length of 50 will be applied (cast to `(N)VARCHAR(50)`).
- `(SQL_)CHAR`, `(SQL_)NCHAR` without explicit length will be cast to `(N)CHAR(50)`
- `(SQL_)BINARY`, and `(SQL_)VARBINARY` without explicit length will be cast to `(VAR)CHAR(50) CHARACTER SET OCTETS`. With explicit length, `CHARACTER SET OCTETS` is appended.
- `(SQL_)LONGVARCHAR`, `(SQL_)LONGNVARCHAR`, `(SQL_)CLOB`, `(SQL_)NCLOB` will be cast to `BLOB SUB_TYPE TEXT`, national character set will be lost
- `(SQL_)LONGVARBINARY`, `(SQL_)BLOB` will be cast to `BLOB SUB_TYPE BINARY`
- `(SQL_)TINYINT` is mapped to `SMALLINT`
- `(SQL_)ROWID` is not supported as length of `DB_KEY` values depend on the context
- `(SQL_)DECIMAL` and `(SQL_)NUMERIC` without precision and scale are passed as is. In current Firebird versions, this means the value will be equivalent to `DECIMAL(9,0)`/`NUMERIC(9,0)`, which is equivalent to `INTEGER`.
- Unsupported/unknown *SQLtype* values (or invalid length or precision and scale) are passed as is to cast, resulting in an error from the Firebird engine if the resulting cast is invalid

[1] The `CHARACTERS` parameter only determines that characters are counted, the ignored blanks (space (0x20) or NUL (0x00)) are not determined by the parameter but by the underlying type

[2] The `OCTETS` parameter only determines that bytes are counted, the ignored blanks (space (0x20) or NUL (0x00)) are not determined by the parameter but by the underlying type

[3] Parameter `OCTETS` is ignored

[4] Always returns English full names (e.g. “Sunday”)

[5] Always returns English full names (e.g. “January”)

[6] Either the full path of the database or the alias. See documentation of `RDB$GET_CONTEXT('SYSTEM', 'DB_NAME')` for details.

# Appendix F: Jaybird versions

This appendix lists the distribution files and supported specifications of recent Jaybird versions.

## F.1. Jaybird 6

### F.1.1. Java support

Jaybird 6 supports Java 17 and higher (JDBC 4.3 - 4.5).

Given the limited support period for non-LTS Java versions, we limit support to Java 17 and the LTS versions after Java 17, and the latest Java release. Currently, that means we support Java 17, Java 21, Java 25, and Java 26.

Jaybird 6 provides libraries compiled against Java 17, which also work on higher Java versions.

Jaybird 6 is modularized, and provides the following modules:

`org.firebirdsql.jaybird`
: main Jaybird driver (artifact `org.firebirdsql.jdbc:jaybird`)

`org.firebirdsql.jaybird.chacha64`
: ChaCha64 wire encryption implementation (artifact `org.firebirdsql.jdbc:chacha64-plugin`)

`org.firebirdsql.jna`
: native and embedded protocol implementation using JNA (artifact `org.firebirdsql.jdbc:jaybird-native`)

### F.1.2. Firebird support

Jaybird 6 supports Firebird 3.0, Firebird 4.0, and Firebird 5.0.



**No promises on Firebird 6.0 support**

Firebird 6.0 is still in development, and includes changes — like schema support — that impact Jaybird.

As a result — at this time — we make no promises on the support for Firebird 6.0 by Jaybird 6. We may delay support for Firebird 6.0 until Jaybird 7, or we may improve support during maintenance releases of Jaybird 6.

Jaybird 6 will — by default — not connect to Firebird 2.5 or older. To connect to earlier versions, unsupported protocols have to be enabled explicitly, see Enabling unsupported protocol versions for details.

Formal support for Firebird 2.5 has been dropped, and some things may no longer work (especially `DatabaseMetaData` methods), if features of Firebird 3.0 have been used. The Type 2 and embedded server JDBC drivers require a separate artifact, `jaybird-native`, and JNA to access the Firebird client or embedded library.

#### Notes on Firebird 3.0 support

Jaybird 6 supports the Firebird 3.0 and higher zlib compression.

Jaybird 6 supports the Arc4 wire encryption plugin out-of-the-box.

#### Notes on Firebird 4.0 support

Jaybird 6 supports the protocol improvements of Firebird 4.0 for statement timeouts and server-side batch execution. Jaybird 6 does not support sending blobs as part of batch execution.

Jaybird 6 supports the Arc4 and ChaCha wire encryption plugin out-of-the-box. To support the ChaCha64 wire encryption plugin, the artifact `org.firebirdsql.jdbc:chacha64-plugin` has to be added to the classpath.

See also Notes on Firebird 3.0 support

#### Notes on Firebird 5.0 support

Jaybird 6 supports Firebird 5.0 server-side scrollable cursors and multi-row `RETURNING`.

See also Notes on Firebird 4.0 support

### F.1.3. Supported Specifications

Jaybird 6 supports the following specifications:

| Specification | Details |
|---|---|
| JDBC 4.5 | All JDBC 4.5 methods for features supported by Firebird, except arrays. |
| JDBC 4.4 | If a security manager is available, the driver still performs `SQLPermission` checks. |
| JDBC 4.3 | All JDBC 4.3 methods for features supported by Firebird, except arrays. |
| JTA | The driver provides an implementation of the `javax.transaction.xa.XAResource` interface via a `javax.sql.XADataSource` implementation. |

As we don't have access to the JDBC TCKs, Jaybird is not officially JDBC compliant.

### F.1.4. Distribution

Jaybird has no required runtime dependencies.

For native or embedded connections, the `jaybird-native` artifact and `jna-jpms` 5.19.1 need to be on the classpath.

For ChaCha64 wire encryption, the `chacha64-plugin` artifact and `bcprov-jdk18on` 1.86 (Bouncy Castle JCE provider) need to be on the classpath.

### Distribution package

The latest version of Jaybird can be downloaded from https://firebirdsql.org/en/jdbc-driver/

The following files can be found in the distribution package:

| File name | Description |
|---|---|
| `jaybird-6.0.7.jar` | The main JDBC driver, the Services API and event management classes. Provides only pure Java connections. |
| `chacha64-plugin-6.0.7.jar` | Optional dependency, support for the ChaCha64 wire encryption for pure Java connections. |
| `lib/bcprov-jdk18on-1.86.jar` | Optional dependency, required for ChaCha64 support (with `chacha64-plugin`) |
| `jaybird-native-6.0.7.jar` | Optional dependency, support for native and embedded connections. Requires `lib/jna-jpms-5.19.1.jar`. |
| `lib/jna-jpms-5.19.1.jar` | Optional dependency, required if you want to use Type 2 native or embedded protocols (with `jaybird-native`) |

### Maven

Alternatively, you can use Maven to automatically download Jaybird and its dependencies.

Jaybird 6 is available from Maven central:

**groupId** `org.firebirdsql.jdbc`

**artifactId** `jaybird`

**version** `6.0.7`

For example:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird</artifactId>
  <version>6.0.7</version>
</dependency>
```

If you want to use ChaCha64 wire encryption, you need to explicitly include `chacha64-plugin` as a dependency:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>chacha64-plugin</artifactId>
  <version>6.0.7</version>
</dependency>
```

If you want to use Type 2 support (native or embedded), you need to explicitly include `jaybird-native` as a dependency:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird-native</artifactId>
  <version>6.0.7</version>
</dependency>
```

In addition, for native connections you can use the `org.firebirdsql.jdbc:fbclient` dependency to provide the client library (`fbclient.dll`/`libfbclient.so`) on Windows and Linux. See Maven dependency for native client for details.

#### Gradle

See also Maven.

Examples:

*Main Jaybird artifact*

```
implementation 'org.firebirdsql.jdbc:jaybird:6.0.7'
```

*Jaybird native artifact (native and embedded protocol)*

```
implementation 'org.firebirdsql.jdbc:jaybird-native:6.0.7'
```

*ChaCha64 wire encryption plugin*

```
implementation 'org.firebirdsql.jdbc:chacha64-plugin:6.0.7'
```

## F.2. Jaybird 5



Although Jaybird 5 is still supported, we recommend upgrading to Jaybird 6 if you target Java 17 or higher and Firebird 3.0 and higher.

### F.2.1. Java support

Jaybird 5 supports Java 8 (JDBC 4.2), and Java 9 and higher (JDBC 4.3 - 4.5).

Given the limited support period for Java 9 and higher versions, we limit support to Java 8, 11, 17 and the LTS versions after Java 17 and the latest Java release. Currently, that means we support Java 8, Java 11, Java 17, Java 21, Java 25, and Java 26.



Jaybird 5 is the last version to support Java 8 and 11. Jaybird 6 uses Java 17 as the baseline (minimum) version. We highly recommend upgrading to Java 17 or

higher and to Jaybird 6.

Jaybird 5 serves as a "long-term support" version for Java 8 and 11, with maintenance releases at least until the release of Jaybird 7.

It is possible future maintenance releases of Jaybird 5 stop expanding Java support to include newly released Java versions. A reason can be — but is not limited to — if adding support for a newer Java version results in breaking support for Java 8 or Java 11, or JDBC 4.2 or JDBC 4.3, or require changes that would limit testability on those versions. We may also simply stop expanding Java support to limit the number of versions we need to test.

Jaybird 5 provides libraries for Java 8 and Java 11. The Java 8 builds have all JDBC 4.3 related functionality and can be used on Java 9 and higher as well, but the Java 11 version has additional features, like the ChaCha wire encryption.

Jaybird 5 is not modularized, but all versions declare the automatic module name `org.firebirdsql.jaybird`.

## F.2.2. Firebird support

Jaybird 5 supports Firebird 2.5, Firebird 3.0, Firebird 4.0, and Firebird 5.0.



**No promises on Firebird 6.0 support**

Firebird 6.0 is still in development, and includes changes — like schema support — that impact Jaybird.

As a result — at this time — we make no promises on the support for Firebird 6.0 by Jaybird 5. We may delay support for Firebird 6.0 until Jaybird 7, or we may improve support during maintenance releases of Jaybird 5 and/or Jaybird 6.

Jaybird 5 is the last version to support Firebird 2.5.

Formal support for Firebird 2.0 and 2.1 has been dropped, and some things may no longer work (especially `DatabaseMetaData` methods), if features of Firebird 2.5 have been used. The Type 2 and embedded server JDBC drivers use JNA to access the Firebird client or embedded library.

### Notes on Firebird 3.0 support

Jaybird 5 supports the Firebird 3.0 and higher zlib compression.

### Notes on Firebird 4.0 support

Jaybird 5 supports the protocol improvements of Firebird 4.0 for statement timeouts and server-side batch execution. Jaybird 5 does not support sending blobs as part of batch execution.

Jaybird 5 only supports the ChaCha wire encryption plugin using the Java 11 version of the driver. Jaybird 5 does not support the ChaCha64 wire encryption plugin. Support for ChaCha64 was added in Jaybird 6.

See also Notes on Firebird 3.0 support

#### Notes on Firebird 5.0 support

Jaybird 5 supports Firebird 5.0 server-side scrollable cursors and multi-row `RETURNING`.

See also Notes on Firebird 4.0 support

### F.2.3. Supported Specifications

Jaybird 5 supports the following specifications:

| Specification | Details |
|---|---|
| JDBC 4.5 | All JDBC 4.5 methods for features supported by Firebird, except arrays. |
| JDBC 4.4 | If a security manager is available, the driver still performs `SQLPermission` checks. |
| JDBC 4.3 | All JDBC 4.3 methods for features supported by Firebird, except arrays. |
| JDBC 4.2 | All JDBC 4.2 methods for features supported by Firebird, except arrays. |
| JTA | The driver provides an implementation of the `javax.transaction.xa.XAResource` interface via a `javax.sql.XADataSource` implementation. |

As we don't have access to the JDBC TCKs, Jaybird is not officially JDBC compliant.

### F.2.4. Distribution

Jaybird has no required runtime dependencies. For native or embedded connections, JNA 5.19.1 needs to be on the class path.

#### Distribution package

The latest version of Jaybird can be downloaded from https://firebirdsql.org/en/jdbc-driver/

The following files can be found in the distribution package:

| File name | Description |
|---|---|
| `jaybird-5.0.13.<java>.jar` | An archive containing the JDBC driver, the Services API and event management classes. Where `<java>` is either `java11` or `java8`. |
| `lib/jna-5.19.1.jar` | Optional dependency, required if you want to use Type 2 native, or embedded protocols |

#### Maven

Alternatively, you can use maven to automatically download Jaybird and its dependencies.

Jaybird 5 is available from Maven central:

| | |
|---|---|
| **groupId** | `org.firebirdsql.jdbc` |
| **artifactId** | `jaybird` |
| **version** | `5.0.13.<java>` (where `<java>` is either `java11` or `java8`) |

For example:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird</artifactId>
  <version>5.0.13.java11</version>
</dependency>
```

If you want to use Type 2 support (native or embedded), you need to explicitly include JNA as a dependency:

```
<dependency>
  <groupId>net.java.dev.jna</groupId>
  <artifactId>jna</artifactId>
  <version>5.19.1</version>
</dependency>
```

For native connections you can use the `org.firebirdsql.jdbc:fbclient` dependency to provide the client library. See Maven dependency for native client for details.

#### Gradle

See also Maven.

Equivalent declarations for Gradle:

*Main Jaybird artifact*

```
implementation 'org.firebirdsql.jdbc:jaybird:5.0.13.java11'
```

*JNA dependency for native and embedded*

```
implementation 'net.java.dev.jna:jna:5.19.1'
```

## F.3. Jaybird 4



Jaybird 4 is end-of-life and will not receive further updates. We recommend upgrading to Jaybird 6.

### F.3.1. Java support

Jaybird 4 supports Java 7 (JDBC 4.1), Java 8 (JDBC 4.2), and Java 9 and higher (JDBC 4.3).

Given the limited support period for Java 9 and higher versions, we will limit support on those versions to the recent LTS versions and the latest release. Currently, that means we support Java 7, 8, 11, 17, and 21[1].

Jaybird 4 provides libraries for Java 7, Java 8, and Java 11. The Java 8 builds have the same source and all JDBC 4.3 related functionality and can be used on Java 9 and higher as well.

Jaybird 4 is not modularized, but all versions declare the automatic module name `org.firebirdsql.jaybird`.

### F.3.2. Firebird support

Jaybird 4 supports Firebird 2.5 and higher.

Formal support for Firebird 2.0 and 2.1 has been dropped (although in general we expect the driver to work). The Type 2 and embedded server JDBC drivers use JNA to access the Firebird client or embedded library.

#### Notes on Firebird 3 support

Jaybird 4 supports the Firebird 3.0 and higher zlib compression.

#### Notes on Firebird 4 support

Jaybird 4 supports the protocol improvements of Firebird 4.0 for statement timeouts, but does not implement the new batch protocol.

Jaybird 4 provides time zone support.

Jaybird 4 supports the extended numeric precision types `NUMERIC` or `DECIMAL` with a precision higher than 18 up to 38. Since Jaybird 4.0.1, the `INT128` type is also supported.

Jaybird 4 does not support the ChaCha wire encryption plugin. Support for ChaCha has been added in Jaybird 5.

### F.3.3. Supported Specifications

Jaybird supports the following specifications:

| Specification | Details |
|---|---|
| JDBC 4.3 | Jaybird supports most of JDBC 4.3, in as far the features are required or supported by Firebird. It is not officially JDBC compliant, because we currently don't have access to the TCK. |

| Specification | Details |
|---|---|
| JCA 1.5 | Jaybird provides an implementation of `javax.resource.spi.ManagedConnectionFactory` and related interfaces. CCI interfaces are not supported. |
| JTA | The driver provides an implementation of the `javax.transaction.xa.XAResource` interface via the JCA framework and a `javax.sql.XADataSource` implementation. |

## F.3.4. Distribution

The Jaybird driver has compile-time and run-time dependencies to JCA 1.5.

### Distribution package

The latest version of Jaybird can be downloaded from https://firebirdsql.org/en/jdbc-driver/

The following files can be found in the distribution package:

| File name | Description |
|---|---|
| `jaybird-4.0.10.<java>.jar` | An archive containing the JDBC driver, the JCA connection manager, the Services API and event management classes. Where `<java>` is either `java7`, `java8` or `java11`. |
| `jaybird-full-4.0.10.<java>.jar` | Same as above, but including the JCA 1.5 dependency. Where `<java>` is either `java7`, `java8` or `java11`. |
| `lib/connector-api-1.5.jar` | Required dependency; part of `jaybird-full`, not necessary when deploying to a Java EE or Jakarta EE 8 or lower application server |
| `lib/jna-5.5.0.jar` | Optional dependency, required if you want to use Type 2 native, local or embedded protocols |

### Maven

Alternatively, you can use maven to automatically download Jaybird and its dependencies.

Jaybird 4 is available from Maven central:

**groupId** `org.firebirdsql.jdbc`

**artifactId** `jaybird`

**version** `4.0.10.<java>` (where `<java>` is either `java11`, `java8` or `java7`)

For example:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird</artifactId>
  <version>4.0.10.java11</version>
```

```
</dependency>
```

If your application is deployed to a Java EE or Jakarta EE 8 or lower application server, you will need to exclude the `javax.resource:connector-api` dependency, and add it as a provided dependency:

```
<dependency>
  <groupId>org.firebirdsql.jdbc</groupId>
  <artifactId>jaybird</artifactId>
  <version>4.0.10.java11</version>
  <exclusions>
    <exclusion>
      <groupId>javax.resource</groupId>
      <artifactId>connector-api</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>javax.resource</groupId>
  <artifactId>connector-api</artifactId>
  <version>1.5</version>
  <scope>provided</scope>
</dependency>
```

If you want to use Type 2 support (native, local or embedded), you need to explicitly include JNA as a dependency:

```
<dependency>
  <groupId>net.java.dev.jna</groupId>
  <artifactId>jna</artifactId>
  <version>5.5.0</version>
</dependency>
```

For native and local you can use the `org.firebirdsql.jdbc:fbclient` dependency to provide the client library. See Maven dependency for native client for details.

[1] The Jaybird 4 support for Java 21 comes with one caveat: some tests weren't run due to lack of support for Java 21 in test libraries used for testing Jaybird, and we did not want to invest time to upgrade or replace those libraries.

# Appendix G: License

The contents of this Documentation are subject to the Public Documentation License Version 1.0 (the "License"); you may only use this Documentation if you comply with the terms of this License. A copy of the License is available at https://firebirdsql.org/en/public-documentation-license/.

The Original Documentation is “Jaybird JDBC Driver Java Programmer’s Manual”. (Previous name: “Jaybird 2.1 JDBC driver Java Programmer’s Manual”.)

The Initial Writer of the Original Documentation is Roman Rokytskyy Copyright © 2004-2008. All Rights Reserved. (Initial Writer contact(s): roman (at) rokytskyy (dot) de).

Contributor(s): Mark Rotteveel.

Portions created by Mark Rotteveel are Copyright © 2014-2026. All Rights Reserved. (Contributor contact(s): mark (at) lawinegevaar (dot) nl).